

分 类 号_____

学号 D201577736

学校代码 10487

密级 _____

华中科技大学

博士学位论文

深度学习系统内存管理和通信优化
关键技术研究

学位申请人：刘 博

学 科 专 业：计算机系统结构

指 导 教 师：金 海 教授

答 辩 日 期：2020 年 7 月 16 日

A Dissertation Submitted in Partial Fulfillment of the Requirements
for the Degree of Doctor of Philosophy in Engineering

**Research on Key Technologies of Memory Management and
Communication Optimization for Deep Learning System**

Ph. D. Candidate : Bo Liu
Major : Computer Architecture
Supervisor : Prof. Hai Jin

Huazhong University of Science & Technology
Wuhan 430074, P.R.China

July 16, 2020

摘要

深度学习作为目前人工智能领域的前沿技术，在处理现实世界复杂问题时优势显著。与传统的人工神经网络相比，深度神经网络拥有更多的隐藏层及神经元，同时产生了大量不同种类的中间数据。这些数据对训练与推理任务的执行效果能够起到至关重要的作用，但与此同时，也在系统层面上带来庞大的计算体量、存储开销以及通信负载。目前深度学习系统优化领域存在不少问题，训练运行时效率较为低下，主要问题集中在两方面，其一是高性能加速硬件的内存容量有限，深度学习系统对内存资源的利用率较低，难以完成大规模迭代训练任务；其二是分布式加速硬件集群的互联带宽有限，集群节点间频繁的梯度通信成为性能瓶颈。解决上述多重资源受限的问题主要涉及到神经网络中特征映射、参数等中间数据的内存管理与稀疏通信优化技术等方面的研究。具体来说，针对深度学习系统优化的研究工作分别从以下三个角度开展。

针对加速硬件存储资源不足以满足训练运行时内存需求的问题，考虑以神经网络模型层结构为中心的内存空间重用思想，提出一种基于空间复用的特征映射数据内存管理机制 Layrub。首先对反向计算的执行序列进行细粒度的重排，然后针对卷积神经网络特征映射数据规模较大的特性，提出神经网络层敏感的内存复用策略，实现高内存利用率的特征映射数据组织方式。通过对数据在加速硬件与主存之间进行合理迁移与放置，使得训练运行时的内存消耗明显减少，能够对深度神经网络的形态设计和进一步研究提供系统级的支撑。实验表明，与原生未优化的深度学习框架 Caffe 相比，Layrub 可以使系统内存使用率平均降低 58.2% 以上，最高可达 98.9%，同时达到适中的时间代价，训练执行时间平均增加 24.1%。实验结果还表明，Layrub 明显优于一些具备内存优化能力的主流深度学习系统，并且可以执行极限规模的深度学习任务。

针对分布式训练中高性能计算节点间梯度交换的通信瓶颈问题，提出了一种基于陈旧补偿的参数梯度稀疏通信优化机制 GradSA。首先提出基于神经网络层结构的梯度稀疏化思想，分析并结合参数梯度数据的特点，减少梯度传输规模，显著降低跨节点通信开销；其次，利用近似梯度进行历史梯度的积累，以此来加快收敛速度。随后，

针对历史梯度累积后的稀疏梯度进行高效编码。实验结果表明，即使在通信网络带宽较低的极端环境下，所提出的梯度优化算法也能够获得良好的吞吐量和加速性能。GradSA 能够在保证迭代计算性能不下降的前提下，将参数梯度通信量减少 514 倍。通过与 8-bit 量化、TernGrad 和 DGC 等主流方法进行比较，进一步证明了 GradSA 在各种神经网络模型和数据集上的优势。

针对分布式训练中计算、内存、通信带宽等资源消耗巨大、协同优化困难的问题，考虑系统内存与通信资源协同优化的思想，提出一种内存高效的分布式稀疏通信机制 LaySA。首先对内存管理策略进行数据对象的增广与重新定义，解决由稀疏通信引起的内存膨胀问题，其次提出镜像参数更新机制，解决参数梯度数据的内存优化与稀疏通信优化难以共存的矛盾，填补分布式深度学习系统在多重资源优化方面存在的空缺，最终实现内存与通信两类优化思路的深度融合、协同执行。实验结果表明，经过系统资源协同优化后，计算节点的内存压力得到有效缓解，在资源利用率及分布式训练效率方面取得了良好效果。与仅采用 Layrub 和 GradSA 的训练系统相比，LaySA 最高能够节约 80.5% 的系统内存使用量，神经网络模型在单 GPU 上整体训练时间缩短了 12.25% 左右，除此之外，LaySA 能够使用更大批量数据进行分布式训练，系统吞吐量整体提升 150% 以上，明显优于单独使用内存或通信优化机制的训练系统。

综上所述，针对深度学习系统优化这个研究命题，从体系结构层面上考虑使用更合理的中间数据存储管理机制，从分析数据结构、系统调优等思路进行，对深度神经网络的训练执行效率进行提升。与此同时，从系统运行时层面上优化增强中间数据（其中包括特征映射数据及参数梯度），从分析中间数据特性、数据体量的思路出发，对深度神经网络的训练质量进行提升。通过上述研究，能够在硬件环境的计算、存储、带宽等资源有限的情况下，尽可能的挖掘大规模深度学习的效率和性能。

关键词： 深度学习 分布式训练 中间数据 内存管理 通信优化

ABSTRACT

Deep learning is one of the popular fields in artificial intelligence, which outperforms the traditional machine learning when dealing with complex problems of real world. Compared with artificial shallow neural networks, deep neural networks have much more hidden layers and neurons, as well as generate enormous amounts of intermediate data with different types. These data play important roles for the efficiency on the training or inference tasks. However, they also bring mammoth amounts of computation load, storage cost, and communication overhead for the GPU-based deep learning systems. There still exist many issues on deep learning systems being addressed. First, due to the limited memory capacity of accelerated hardware (i.e., GPU, FPGA), as well as the low utilization of memory resources, it is difficult to accomplish the large-scale iterative training tasks. Second, since the interconnection bandwidth of distributed accelerated cluster is limited, the frequent gradient communication between cluster nodes becomes the performance bottleneck. When solving these above problems, it mainly involves core techniques on the memory management and sparse communication optimization for the intermediate data (i.e., feature maps, weights, and their gradients) of the training. Specifically, this research focuses on the runtime optimization for the deep learning systems, and develops the corresponding strategies as follows.

To address the memory issue on the training program, we first identify the memory usage characteristics for deep and wide convolutional neural networks and demonstrate the opportunities for memory reuse at both the intra-layer and inter-layer levels. We then present Layrub, a runtime GPU memory management strategy that is specific for the feature map data of convolutional neural networks. First, it orchestrates the execution of the backward phase on the training process. Then, it proposes an idea of layer-centric memory reuse to reduce memory consumption based on the memory characteristics of network models and achieves a proper data organization for the feature map with high memory utilization. By reasonably transferring and placing the data between the acceleration hardware and the main memory, the memory consumption during the training process will be significantly reduced, which can provide system-level support for the morphological design and further research of deep neural networks. Experiments show that, compared to the original Caffe, Layrub can

cut down the memory usage rate by an average of 58.2% and by up to 98.9%, at the moderate cost of 24.1% higher training execution time on average. Results also show that Layrub outperforms some popular deep learning systems with memory optimization strategy and can perform extreme-scale training tasks.

To tackle the communication bottleneck on the distributed training, we propose a staleness-compensated sparse gradient optimization mechanism, GradSA, to improve communication efficiency. First, layer-level gradients are sparsified to reduce the communication overhead, which conforms to characteristics of the network structure. Second, the historical accumulations of the approximated gradients are utilized to speed up convergence. Then, an efficient encoding mechanism is proposed to compress the sparse accumulated historical gradients. Experimental results show that the proposed gradient optimization algorithm can obtain ideal throughput and acceleration performance even in extreme environments with low communication network bandwidth. Furthermore, extensive experiments indicate that GradSA can reduce gradient size by $514\times$ without performance degradation. By comparing with competitors such as 8-bit quantization, TernGrad, and DGC, it further proves the advantages of GradSA in various neural network models and data sets.

To reduce the huge consumption of the multiple resources (i.e., computing, memory, and communication bandwidth) and tackle the conflict between different optimizations, we consider the idea of collaborative optimization on both system memory and bandwidth resources and propose a memory-efficient distributed sparse communication mechanism, called LaySA. First, to tackle the memory ballooning issue caused by sparse communication, the memory reuse strategy of Layrub is refined, and the data object of the memory optimization is augmented and redefined. Second, a mirror parameter update mechanism is proposed to solve the contradiction between the memory management and the sparse communication optimization for weight gradients. The deep integration and collaborative execution of these two types of strategies can fill the gap in the distributed GPU-based training system in terms of multiple resource optimization. Experimental results show that the proposed collaborative optimization can significantly alleviate the memory pressure of computing nodes and improve both the resource utilization and the system throughput of the distributed training systems. Compared with the baseline system using either Layrub or GradSA, LaySA can help to save the system memory usage by up to 80.5%, and the overall

training time of the neural network models on a single GPU is reduced by about 12.25%. Furthermore, LaySA can extremely scale up the batch size of datasets during the distributed training, and the overall throughput has increased by more than 150%, which outperforms the current training systems that use memory optimization or communication optimization mechanisms alone.

In summary, this research studies the memory management and sparse communication optimization policies for the intermediate data of deep neural networks, aiming to guarantee the efficiency of the runtime training program. At the same time, it also proposes to enhance and refine the intermediate data by considering both the data characteristics and quantity, aiming to keep the convergence quality of the training process. Through the above research, it can excavate the efficiency and performance of large-scale deep learning as much as possible with the restricted hardware resources such as computing, memory, and bandwidth.

Keywords: Deep Learning Distributed Training Intermediate Data Memory Management Sparse Communication Optimization

目 录

摘 要	I
ABSTRACT.....	III
1 绪 论	
1.1 研究背景	(1)
1.2 国内外研究现状	(11)
1.3 研究内容和主要贡献	(15)
1.4 论文组织结构	(17)
2 深度学习系统运行时内存管理机制	
2.1 研究动机	(18)
2.2 基于空间复用的数据放置机制	(24)
2.3 Layrub 原型系统设计与实现	(29)
2.4 实验评估与分析	(34)
2.5 本章小结	(42)
3 深度学习系统运行时稀疏通信优化机制	
3.1 研究动机	(45)
3.2 分层稀疏筛选机制	(51)
3.3 基于误差反馈的近似梯度累加机制	(53)
3.4 高效的稀疏编码压缩机制	(57)
3.5 有效性评价与分析	(59)
3.6 本章小结	(68)
4 深度学习系统运行时内存与通信协同优化机制	
4.1 研究动机	(70)

华中科技大学博士学位论文

4.2 内存高效的参数稀疏通信机制	(74)
4.3 LaySA 原型系统设计与实现	(79)
4.4 有效性评价与分析	(83)
4.5 本章小结	(89)
5 总结与展望	
致 谢	(93)
参考文献	(96)
附录 1 攻读博士学位期间发表的主要论文	(106)
附录 2 攻读博士学位期间申请发明专利	(107)
附录 3 攻读博士学位期间参与的主要科研项目	(108)

插图目录

图 1-1 残差网络结构	4
图 1-2 循环神经网络结构	5
图 1-3 神经网络语言模型结构	6
图 1-4 深度学习系统优化	12
图 1-5 研究内容与论文组织结构	16
图 2-1 卷积神经网络结构特性	19
图 2-2 测试精度优势和内存使用情况	20
图 2-3 卷积神经网络的训练过程	22
图 2-4 基于层内空间复用的数据分时放置	26
图 2-5 基于跨层空间复用的数据迁移	27
图 2-6 混合的空间复用策略	29
图 2-7 同步 barrier 对内核执行的影响	33
图 2-8 深度模型内存分配对比	35
图 2-9 宽度模型内存分配对比	36
图 2-10 多个模型内存分配对比	37
图 2-11 训练系统运行时内存消耗对比	38
图 2-12 系统规范化性能对比	39
图 3-1 基于历史梯度积累的稀疏梯度训练	47
图 3-2 不同的筛选粒度对模型训练性能的影响	49
图 3-3 三阶段梯度通信机制	51
图 3-4 基于索引的梯度编码	59
图 3-5 ResNet-56 学习曲线	62

图 3-6 ResNet-56 测试精度与训练时间对比.....	63
图 3-7 不同模型的时间开销对比.....	63
图 3-8 不同 GPU 集群训练吞吐量对比.....	66
图 3-9 低带宽环境下训练加速对比.....	68
图 4-1 基于稀疏通信的训练过程.....	72
图 4-2 内存分配情况对比.....	73
图 4-3 内存复用策略重定义.....	76
图 4-4 内存高效的参数通信.....	78
图 4-5 LaySA 训练执行流程图.....	82
图 4-6 训练运行时内存分配情况.....	84
图 4-7 训练系统吞吐量对比.....	88

表格目录

表 1-1 模型定量分析	8
表 1-2 AlexNet 各层参数规模统计	10
表 2-1 现有训练系统对比	21
表 2-2 Layrub 函数调用	31
表 2-3 系统配置描述	34
表 2-4 不同训练系统的训练时间对比	40
表 2-5 极端规模模型内存消耗对比	41
表 2-6 分布式训练效率对比	42
表 3-1 最新的 NVIDIA GPU 的特征介绍	46
表 3-2 互连网络的特征介绍	46
表 3-3 压缩比例对比	58
表 3-4 编码解码时间对比	58
表 3-5 现有蜂窝移动网络制式总结	60
表 3-6 梯度优化策略的训练精度对比	61
表 3-7 梯度优化策略的压缩比对比	61
表 4-1 系统配置描述	83
表 4-2 批量尺寸及内存消耗对比	86
表 4-3 批量处理规模与训练耗时	87

1 绪 论

当前,以深度神经网络(Deep Neural Network, DNN)为基础的深度学习技术在语音、视觉、自然语言处理等领域取得了巨大成功。例如在图像分类、机器翻译、棋牌对抗等多项任务上,相关算法已经超过一般人类水平,为语音服务、汽车自动驾驶、金融支付、工业制造、安保等多个行业带来了革命性的产业升级。深度学习作为一门综合了脑科学、思维科学、统计数学、计算科学及工程科学等的前沿学科,近几年来在高性能硬件强大算力的驱动下,迎来了新一轮的研究热潮。而与传统的人工神经网络相比,深度网络拥有更多的隐藏层及神经元,训练与推理阶段的计算体量庞大,同时也会产生大量的中间数据。这些特性使得大多数的深度学习应用对训练过程的存储空间及计算平台的高可扩展性有着较大需求。因此,针对深度学习训练过程开展中间数据内存管理与分布式通信等优化研究已经成为热点方向。

本章首先介绍深度学习的概念,分析深度学习中间数据的特性,指出运行时中间数据内存管理与通信优化等方面所面临的挑战,然后分析国内外研究现状及存在的不足,最后给出研究内容、主要贡献和组织结构。

1.1 研究背景

作为目前人工智能领域的前沿技术,深度学习在处理复杂大数据问题时优势显著,是近期关注度最高的机器学习方法之一。本节首先介绍深度学习的概念、基本原理和发展近况,然后根据深度学习训练过程中各类数据的功能特性,对相关的中间数据进行分类,最后引出对中间数据进行内存管理与分布式通信优化的重要意义。

1.1.1 深度学习简介

深度学习的理论研究开始于 20 世纪 40 年代,它植根于生物神经网络的基本工作原理,使用具备基本逻辑运算功能的人工神经元来构建人工神经网络^[1]。换言之,深度学习其实是一种通过模拟大脑的神经突触连接结构进行复杂信息处理的方法,在多

层感知的设计架构下，能够从现实世界的海量原始数据中自主地进行层次化的特征学习，构建低层—中层—高层特征的抽象模式，提取出稳定的、不易变的特征映射，用于分类、预测等领域特定任务。与传统神经网络、逻辑回归、支持向量机、决策树等传统的、浅层的机器学习方法相比，深度神经网络的网络结构深度更深、宽度更宽，非线性表征能力更强，模型学习能力更强。同时，深度学习方法避免了对特征描述算子的手工设计，更好的利用原始数据本身的无规则特征去挖掘分类预测任务所需信息，并且通过层次化的端对端构建方式，解除了对数据预处理过程如图像分割或语义提取等操作的依赖，实现过程更加的便捷、易操作。

尽管目前深度学习在多个应用领域取得了空前的成功，但是回顾历程，深度学习的发展也经历了多次的大起大落。但随着计算机体系结构的不断发展，硬件计算能力的不断提升，深度学习终于迎来空前繁荣的阶段。

1) 深度学习发展历程

Rosenblatt 等人于 1957 年提出单层神经感知机^[2]，为人工神经网络的发展奠定了基础，然而由于缺少合适的训练算法，深度学习暂时未得到进一步的关注。八十年代中后期，Hopfield 以及 LeCun 等人提出了训练神经网络的反向传播算法^[3]（Back-Propagation, BP），多种神经网络随之被提出，如受限玻尔兹曼机^[4]（Restricted Boltzmann Machine, RBM）等，深度学习迎来了第一轮热潮。但是，由于 CPU 等计算硬件的存储空间及算力等资源所限，再加之训练阶段存在梯度消失（爆炸）、难以求得全局极值等算法缺陷，相对于支持向量机^[5]（Support Vector Machine, SVM）、随机森林^[6]（Random Forest）等传统机器学习算法，神经网络在多种应用中均表现不佳，因此深度学习的热潮也随之迅速冷却降温^[7]。

LeCun 等人于 1998 年提出 LeNet-5^[3]，开启了卷积神经网络（Convolutional Neural Network, CNN）的时代，这一神经网络模型在手写数字识别任务中取得了突破性的胜利。LeNet-5 在神经网络架构设计中提出采用局部感受野、权重共享以及池化等思想，它所使用的网络层结构一直沿用至今，给后续研究者带来了无数的启发与灵感^[8]。Hinton 等学者于 2006 年提出深度置信网络^[9]（Deep Belief Net, DBN），网络模型中的每一层被视为一个无监督的受限玻尔兹曼机，然后使用有监督的反向传播算法对模

型权重进行微调。作者指出多层的神经网络具备更强的学习能力，并且通过无监督预训练及有监督反向传播等方法，有效解决了传统神经网络因网络深度过深而极易陷入局部最小的问题^[9]，显著改善多层模型的性能。随后，Krizhevsky 等学者于 2012 年提出 AlexNet^[11]，该模型以 top-1 错误率为 37.5%，top-5 错误率为 17.0% 的成绩，一举夺得了 2012 年的目标物体分类竞赛冠军，深度学习再一次迎来了发展的热潮。自此之后，大量以卷积神经网络为核心思想的神经网络模型问世，这其中包括 VGGNet^[12]，Inception-v3^[13]，GoogLeNet^[14]，Inception-v4^[15]，ResNet^[16,17]，其中，ResNet 的出现让神经网络不断加深这一概念成为现实，残差网络结构利用恒等映射和跳跃式传递的方式进行残差学习，将神经网络成功拓展到 1000 多层，在获取丰富的高层次特征的基础上，图像分类任务准确率达到了当时的最高水平。

总的来说，在人工智能领域，一场关于深度学习的技术变革正在不知不觉中开启。究其所以然，除了神经网络架构设计的不断推陈出新之外，也包含如下的原因：相机与手机等手持移动设备的普及，使得获取大量的高清图像数据变得容易，大规模带标签的训练数据集得以出现，神经网络能够处理的任务也慢慢变得丰富而多样化。除此之外，更深层次的原因^[18]在于：图形处理单元（Graphics Processing Unit，GPU）^[19,20]、现场可编程逻辑门阵列（Field Programmable Gate Array，FPGA）^[21,22]、以及应用特定集成电路（Application Specific Integrated Circuit，ASIC）^[23,24]等加速硬件在计算能力方面实现了快速增长，使得大规模的并行数据处理成为可能。计算机体系结构的快速发展满足了深度神经网络对硬件算力等资源的需求，有力地推动着学界与工业界的持续研究^[25,26]，使深度学习领域正在达到前所未有的繁荣景象。可以说，如果没有强有力的通用^[27]或专有加速硬件^[28,29]作为基础支撑，帮助深度学习系统完成训练与推理任务，很多模型算法设计相关的研究工作将无法开展并得到验证，深度神经网络强大的表征作用也就无法体现出来^[30]。因此，面向深度学习的计算机体系结构优化研究对深度学习的发展具有重要意义。

2) 深度学习基本理论

这一部分主要介绍本文所涉及到的各类深度神经网络模型，同时对深度学习的重要理论——深度神经网络的训练过程进行简要介绍。具体的，首先对深度神经网络进

行介绍，主要描述其类型、结构特征等概要性内容。随后会对前向—后向误差传播算法的基本原理进行介绍。

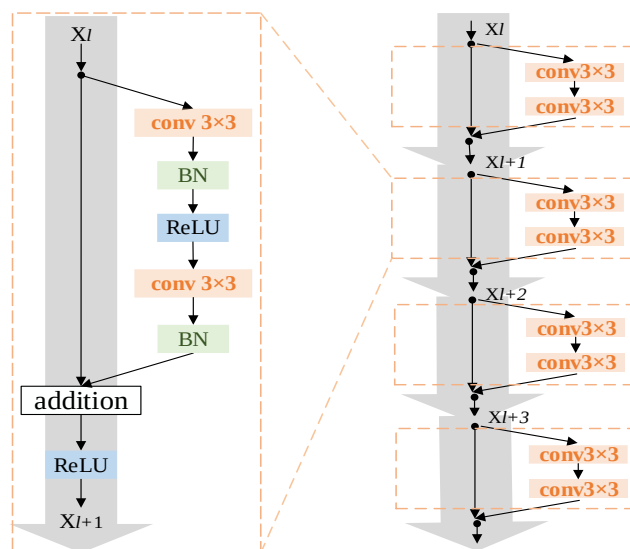


图 1-1 残差网络结构

(1) 深度神经网络模型

下面主要介绍以卷积神经网络、循环神经网络以及神经网络语言模型等为代表的深度神经网络模型，这些模型现已被应用到包括机器视觉、语音识别、以及自然语言处理等众多领域。

深度卷积神经网络：卷积神经网络的设计思想基于如下认知：图像的视觉特征分布在整张图像上；卷积操作可以借助少量可学习参数，在图像的不同位置上提取出相似的特征映射。作为多层神经感知机的推广形式，卷积神经网络在多个方面对其进行优化改进。卷积神经网络的基本结构一般包括卷积层（Convolutional Layer）、池化（降采样）层（Pooling/Sub-sampling Layer）、非线性映射层（Non-linear Layer）和全连接层（Fully Connected Layer）。卷积层对输入图像进行局部感知，提取低层次的视觉特征，并在更高的层次进行组合得到全局信息表征。池化（降采样）层一般位于连续的卷积层之间，使用不同的特征提取方式（如最大池化、均值池化、卷积降采样等）进行特征降维，同时能够保证特征不变性，防止过拟合、提高模型泛化性。在非线性映射层中，输入特征通过非线性函数映射得到输出特征，由于卷积网络中存在连续的线性运算，引入非线性映射层更加符合生物学的定义。全连接层一方面能够充分整合全

局的图像特征信息，一方面对特征进行维度变换，起到分类器的作用，得到分类类别所对应的概率得分。举例说明，如在著名的手写识别网络 LeNet-5 中包含两个卷积层、两个池化（降采样）层、三个全连接层和多个非线性映射层。

作为卷积神经网络在近年来最大的设计突破，残差（Residual）结构的基本思想是引入跨层连接操作，即能够跳过一层或多层的恒等映射（Identity Mapping）。图 1-1 左侧展示了基本的残差结构^[16]。利用恒等映射和跳跃式传递的方式进行残差学习，以 ResNet 为代表的一系列神经网络得以深度加深的研究方向继续探索。图 1-1 右侧展示了深度卷积神经网络的基本结构。已知当前最深的残差网络^[16,17]能够成功拓展到 1200 多层，在获取丰富的高层次特征的同时，达到非常理想的鲁棒性和计算效率。

深度循环神经网络：卷积神经网络主要处理没有时间特性的计算机视觉问题，相比较来说，循环神经网络则能够较好地完成时间序列任务。近几年，循环神经网络^[31]（Recurrent Neural Network, RNN）在语音识别、视频理解等领域被广泛应用并取得巨大成功。循环神经网络在传统人工神经网络的基础上引入了定向循环的思想，在隐藏层中增加了一条信息传输通道，使得隐藏节点定向连接成环。如图 1-2 所示，左侧的 RNN 可以具体展开为右侧部分^[31]。从图中可以观察模型的前向传播过程，其中， X 为输入信号， Y 为输出信号， W 为连接各层间神经元的参数权重， S 为神经元激活值， t 代表当前时刻，从模型结构图和前向传播原理可知，RNN 隐藏层的激活值 S_t 不仅仅取决于当前输入信号 X_t ，还取决于上一个时刻隐藏层的激活值 S_{t-1} 。这样的内部结构有助于网络模型对时间标签的传递，使得模型能够对时间特性建模，表征训练样本的动态时序行为，从而挖掘时间序列特征，最后得到合理的分类或预测结果。

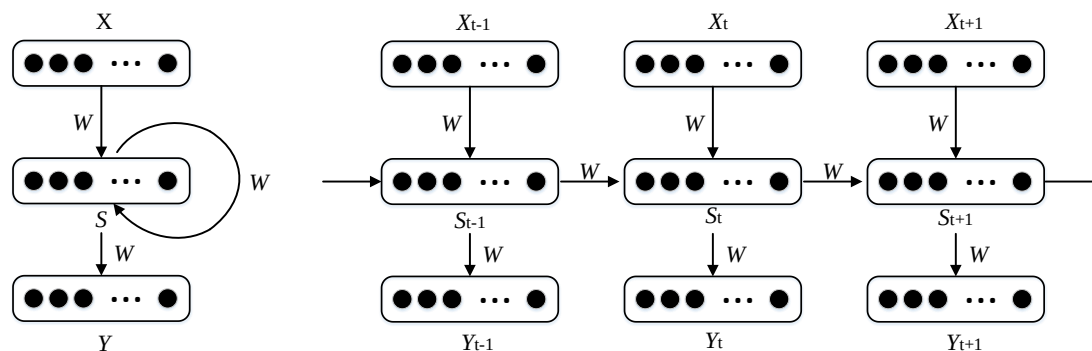


图 1-2 循环神经网络结构

神经网络语言模型：与卷积神经网络、循环神经网络相比，神经网络语言模型^[32,33]（Neural Network Language Model, NNLM）由于其具备长距离建模能力，弥补了其他模型不能合理表征大规模自然语言文本的缺点，而词向量^[34,35]（Word Embedding）作为神经网络语言模型的副产物，一经出现便成为自然语言处理领域的研究基础，在各类应用中起到至关重要的作用。

与传统的基于统计信息的语言模型相比，NNLM 将语料中的任意词项用词向量 $e(m) \subseteq F^m$ 表示， $e(m)$ 是一个长度为 m 的实值向量，该模型的基础是神经概率语言模型，通过神经网络来计算语言模型中的参数，在训练过程中得到词向量。该模型可以克服数据稀疏问题，同时可以使用词向量表征词语间的相似性。整体结构^[30]如图 1-3 虚线左侧所示，主要包含输入层（Input Layer）、投影层（Projection Layer）、隐藏层（Hidden Layer）、以及输出层（Output Layer）。对于任意单词 w_t ，将 $Context(w_t)$ 定义为 w_t 上下文相邻的若干个单词，即 $Context(w_t) = \{w_{t-k}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+k}\}$ ，输入是给定的上下文中各个词语的词向量 $e(Context(w_t))$ 。其中输入层到投影层的映射过程是将所有上下文相关的词向量进行串联，作为单词 w_t 的特征向量。投影层到隐藏层的映射过程使用 Tanh 激活函数，对串联后向量的每一个分量完成激活操作。隐藏层到输出层属于线性过程，使用 Softmax 函数进行归一化操作得到最终的目标概率函数，这里目标函数的意义是通过 w_t 的上下文的若干个单词，能够最大化当前单词是 w_t 的概率。

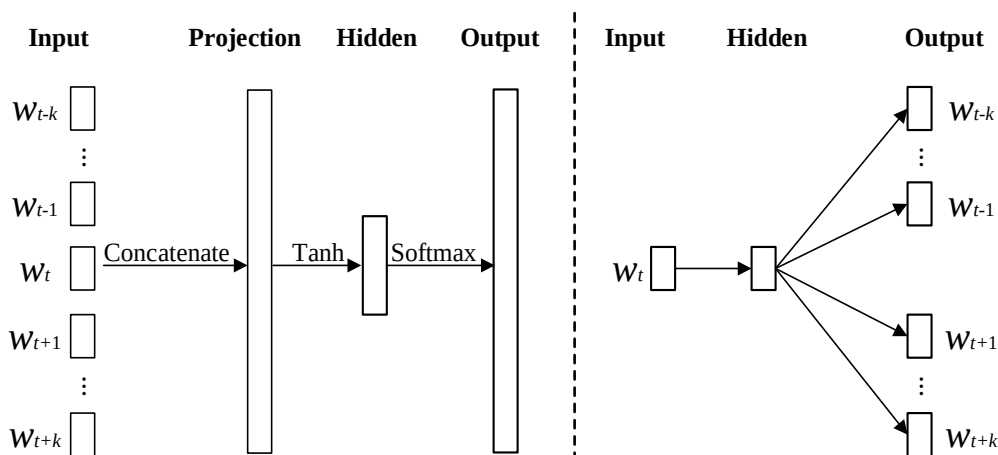


图 1-3 神经网络语言模型结构

为对海量自然语言文本进行合理表征，近年来研究者们纷纷尝试基于 NNLM 提

出改进,首先在隐藏层到输出层的部分引入层级思想^[33],随后提出代表性神经网络模型 Skip-Gram 和 CBOW 来训练词向量^[34,35], Word2Vec 作为该模型的开源实现,得到了学界及企业界高度的关注。Skip-Gram 模型具体结构如图 1-3 的虚线右侧所示。近几年,NNLM 家族出现以 BERT 为代表的双向语言模型^[36,37],引入 Transformer 编码器^[38]作为主体模型结构,使用多头注意力(Multi-Head Self Attention)机制进行特征抽取,动态编码变长序列,获取到上下文相关的双向特征表示,在多项下游任务中获得理想效果。

(2) 神经网络训练方法

在上述提到的各类神经网络中,神经元均从输入层开始,接收前一层的输入信号,并输出信息到下一层,直至传播到输出层,最后进行分类或预测,这一过程称为前向推理(传播)阶段。由于神经网络希望可以通过某种方式使得输出尽可能等于正确分类,因此通常利用目标值与输出层的误差(也称损失函数)来更新参数,使用链式法则将误差沿网络反方向传播到前一层,再利用随机梯度下降法^[3](Stochastic Gradient Descent, SGD),沿着梯度下降的方向,迭代求解函数的极小值,从而得到最小化的损失函数、以及使其最小化的模型参数,最终完成参数的更新。这一过程被称为后向训练(传播)阶段,而该训练方法称为反向传播算法^[40](BP 算法)。

目前对于各种深度神经网络(包括卷积神经网络及神经网络语言模型等)的训练,基本上使用的都是 BP 算法。而对于具有时间属性的循环神经网络,训练方法为随时时间的反向传播算法^[41](Back Propagation Through Time, BPTT)。该方法思路与传统的反向传播思路略有不同,由于循环神经网络针对时间序列进行建模,所以 BPTT 算法需要对各个时间步长求和,从最后一个时间标签将累积误差反向传递,最终完成损失函数的最小化,得到梯度并更新参数。

1.1.2 深度学习中间数据的特性及分类

作为最有前途的机器学习技术之一,深度神经网络能够分析现实世界的复杂信息,并在各种实际应用领域达到较高的预测精度。随着现实世界数据规模的日渐增长,如何能够在大规模数据集下提升应用精度同时保证训练效率,是一个值得持续研究的问题。

题。表 1-1 展示了在包含语义理解、语音识别、图像识别等在内的多个应用领域中，各种神经网络模型所需训练数据集、以及模型数据规模等内容，并且记录了各个神经网络模型训练运行时内存使用量。可以观察到，在各类卷积神经网络中，特征映射类数据消耗了较大比例的内存空间，参数类数据在 GPU 内存中总占比不超过 10%。而相对地，各类基于语言模型的神经网络中，参数类数据则占据了较大比例的内存空间。

表 1-1 模型定量分析

应用领域	网络模型架构	训练数据	模型量化指标	
			模型规模 (参数/特征映射)	训练过程 计算量
语义理解	Word2Vec ^[34,35] (vector dimension=500)	17.3GB 文本 (Wikipedia corpus ^[42])	4.16GB/4.16GB	0.2GFLOPS
	BERT-LARGE ^[36] (sequence length=512, batch size=16)	33 亿单词 (Wikipedia corpus ^[42] , BooksCorpus ^[36])	1.5GB/23.8GB	0.23TFLOPS
语音识别	4-layer LSTM-RNN ^[43] (sequence length=800)	2975 小时 (LID corpus ^[44])	148MB/49.2GB	0.09TFLOPS
图像分类	19-layer VGGNet ^[12] (batch size=64)	60,000 张图片 (Cifar-10 ^[45])	548MB/7.24GB	3.75TFLOPS
	152-layer ResNet ^[16] (batch size=32)	1,000,000 张图片 (ImageNet ^[46])	60.2MB/11.61GB	11.3GFLOPS
	1001-layer ResNet ^[16] (batch size=32)	1,000,000 张图片 (ImageNet ^[46])	40.8MB/67.2GB	0.15TFLOPS
行为检测	199-layer P3D ResNet ^[47] (batch size=128)	1000000 条视频 (Sports-1M ^[48])	232.5MB/23.69GB	0.07TFLOPS

注：模型名称下面括号内的数字表示训练时使用的批量处理尺寸或模型默认配置。模型计算规模的计量单位是 GFLOPS，即每秒十亿（ 10^9 ）次浮点运算；或 TFLOPS，即每秒万亿（ 10^{12} ）次浮点运算。

由于本工作的关注点为如何改善训练时的系统效率问题，同时尽可能地保持神经网络的学习能力，因此需要分析模型推理与训练时的中间数据组成成分，对不同类型中间数据的特性进行简单阐述。具体来说，在神经网络的推理与训练过程中会产生包括模型参数在内的多种中间数据，包括如下：

（1）权重（Weight）：由于神经网络结构中神经元符合逐层分布的规律，相邻层之间神经元相互连接，每个连接上都有一个权重参数，用来决定输入信号对于神经元的影响程度。权重按层排列成矩阵形式，使得神经网络每层的前后向计算都可以用矩阵

计算的形式表达。这里将“Weight”直译为权重，也可用“参数”一词来替代，本文后续内容不再对其进行区分，统一使用“参数”一词表示相同含义。

(2) 参数梯度 (Weight Gradient): 参数梯度是对目标损失函数中的每个参数求偏导得到的，从其物理含义上看，当前参数的梯度是目标函数在该参数上变化最快的位置。训练过程使用梯度下降法得到参数梯度，以损失函数作为目标函数，沿梯度的负方向进行计算，能够更快地得到目标函数最小值，随后使用带方向的梯度值对参数进行迭代地累加更新，最终获得学习后的最新参数。

(3) 特征映射 (Feature Map): 特征映射是指在前向推理过程中，训练数据经过神经网络逐层的线性及非线性计算所得到的输出信号，由于反向训练阶段进行参数梯度计算时需要每一层特征映射的参与，因而需要在前向推理完成后将该类数据存储在内存中，等待反向训练阶段的使用。在一个 n 层的神经网络中，除了最底层的数据输入层和最高层的目标函数层之外，其余各层在前向推理过程中都会产生特征映射数据，并将其存储起来，因此特征映射数据的内存开销与层数 n 成线性关系。

(4) 梯度映射 (Gradient Map): 在反向传播过程中，特征映射梯度（也可称之为梯度映射）是神经网络的最高层损失值对各层特征映射值进行微分求得的结果。损失函数的梯度由对损失值求导得到，其余每层的梯度映射由链式求导法则逐层计算得出。一般而言，梯度映射与特征映射一一对应，且规模相同，因此，梯度映射的内存开销也与神经网络的层数 n 成线性关系。

1.1.3 对深度学习中间数据放置与优化的必要性

加深网络深度^[16,17]被看作是构建深度卷积神经网络过程中最重要的设计原则之一。深度越深，卷积、池化（下采样）、以及非线性激活层则越多，深度神经网络所拥有的参数数量也会不断增加。庞大的参数规模使得深度神经网络拥有强大的表征能力，这样可以有效降低超参数的搜索空间，同时提高模型应用时的鲁棒性。然而，网络模型加深不仅带来参数规模的增加，同时也在无形中增加了各种逐层计算所需中间数据的规模。这里以卷积神经网络 AlexNet^[11]为例，表 1-2 统计了该网络结构的参数规模，然而，值得注意的是，实际运行该神经网络的训练任务时，参与计算的各种中间数据

的规模则远远大于参数自身的规模，这意味着训练运行时所需的内存空间会大于参数本身的存储空间。而 AlexNet 运行时存储开销大的主要根源在于其使用的卷积结构。AlexNet 的前 5 层为卷积层，其结构特点是相邻两层的所有神经元共享连接上的参数权重，因此可学习的参数权重数量非常有限，所需存储空间也相对较少。但是在实际迭代计算中，这些共享的参数权重对神经元输入值所执行的卷积计算并不共享，对应每个神经元的输入（即中间数据中的一种类型——特征映射数据）规模各不相同，所以计算时的内存开销会比存储开销大很多，而实际的计算规模也会大得多。这一情况给深度学习系统带来的最直接的影响就是训练阶段系统对计算、内存、互联带宽等各类资源的过度需求。因此如何能够对各类参与计算的中间数据进行合理的内存管理和稀疏优化，有效地改善训练时各类资源的过度消耗问题，成为众多深度学习系统研究工作的重中之重。

表 1-2 AlexNet 各层参数规模统计

AlexNet	Conv1	Conv2	Conv3	Conv4	Conv5	FC6	FC7	FC8	总量	规模
参数数量	35K	0.6M	0.86M	1.3M	0.86M	36M	17M	4M	60.65M	242.6MB

注：计量单位中 1K=1000，1M=1000K。

总的来说，尽管目前已经有一些深度神经网络成功应用到多个领域中，并且收效显著，然而，其中依然存在不少缺陷。其一是深度神经网络训练运行时对内存空间的需求量较大，内存资源的利用率比较低；其二是深度神经网络训练运行时对计算资源的需求日益增长，分布式计算加速已成趋势，而加速硬件之间的互联带宽非常有限，当前分布式训练效率十分欠佳。

解决上述问题主要涉及到特征映射、参数等中间数据的内存管理及优化增强等方面的研究。具体来说，主要有以下三个方面：

首先，深度学习神经网络的学习能力依赖于特征映射的抽取表征机制，而在神经网络训练过程中，特征映射等中间数据的生成往往需要大量的存储空间，然而当前高性能加速硬件的存储资源远不足以满足训练时的内存需求。具体来说，由于神经网络模型结构日趋复杂，朝着层数更深的方向发展，但与之相对的，高性能加速硬件如 GPU 等的内存容量增长缓慢，这两者之间的矛盾成为了限制神经网络模型发展的主要瓶颈。可以这么说，神经网络模型层数越深，特征映射等中间数据越多，在训练时对 GPU 内

存的需求就越大。因此，如何减少加速硬件上训练运行时中间数据的内存开销是目前亟待解决的问题之一。

其次，深度神经网络的快速训练依赖于分布式高性能加速硬件的并行处理，而加速硬件之间的通信带宽不足以满足训练迭代时中间数据的跨节点交换需求。具体来说，作为分布式深度学习的常用并行方式，数据并行要求每个计算节点计算本地参数梯度，并与其他工作节点交换梯度，最终通过服务节点统一地更新模型参数。这种更新模式带来了集中爆炸式的梯度数据交换，然而相比于海量的数据通信规模，计算节点间的互连网络带宽的发展速度远达不到所需，更无法赶超加速硬件计算能力的发展速度。因此，在使用新型加速器作为分布式工作节点时，通信开销日益成为分布式训练的瓶颈。尤其是在包括移动设备或边缘设备在内的低通信带宽环境下，训练神经网络更是困难重重。因此，如何优化梯度数据、提升待交换梯度的质量、降低待传递梯度的规模、减少通信带宽需求，是目前亟待解决的另一类问题。

最后，尽管当前已有工作分别在运行时内存优化及分布式通信优化两个方面展开研究，但是随着深度学习应用实践的不断深入，训练数据体量与神经网络的模型规模进一步扩张，资源受限往往同时体现在多个维度上，单一地从某角度进行优化，无法从根本上改善训练运行效率。然而，直接对两类优化策略进行简单合并，并不能实际解决多种资源受限的问题，甚至还会阻碍训练效率的提升。例如，在某些神经网络模型中使用参数通信优化策略会进一步的增加内存压力，而与此同时，内存优化策略的引入可能会影响分布式迭代计算的正确性。因此，如何能够应对神经网络训练任务所带来的内存资源、通信带宽资源等方面的挑战，对内存与通信两类优化方案进行深度融合、协同执行，是目前有待解决的重点问题。

1.2 国内外研究现状

针对深度学习系统优化这个命题，可以从多个角度展开研究工作：图 1-4 展示了深度神经网络模型的中间数据与训练内存消耗/运行速度/模型推理精度之间的对照关系，可以直接优化图 1-4 左侧部分（用“自变量”来表示深度神经网络模型的各项指标）来得到右侧部分（用“因变量”来表示深度学习训练系统运行时的各项指标）的

变化，除此之外，也可以对图 1-4 右侧部分的因变量着重考虑，结合系统特点针对性地优化左侧部分的自变量。可知，两种优化思路均可以通过优化自变量（即深度神经网络模型的模型结构及中间数据类型等指标）来达成。而具体来说，针对自变量的优化工作，可以直接按照数据结构、系统优化的思路进行，对深度神经网络达到工程性加速。除此之外，也可以从优化中间数据体量、种类的思路出发，对神经网络的表征和学习能力进行保障和提升。总的来说，要达成优化训练系统性能、提升模型训练精度的目标有两种路线可以走。一是在系统层面上引入更合理的运行时中间数据存储管理与通信机制。二是同时在系统和算法两个层面上优化增强中间数据（其中包括特征映射数据、参数数据、及梯度数据）。

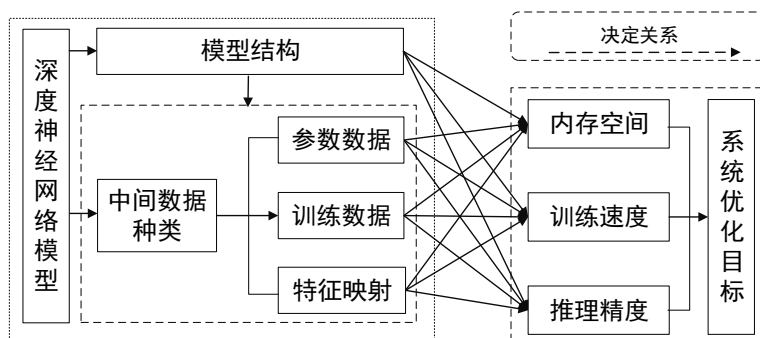


图 1-4 深度学习系统优化

通过对国内外相关工作进行调研和分类，将面向深度神经网络的训练系统运行时优化研究大致分成 2 个大方向，分别是：运行时中间数据内存管理、以及参数梯度数据优化约减。

1.2.1 运行时中间数据内存管理

由于深度学习在多个领域如火如荼的展开，已有包括 GPU、ASIC 等在内的多种先进硬件加速器被广泛部署到学界及工业界，以加快深度神经网络的训练和推理^[49,50]。然而，当前这些加速硬件的内存都相对有限，难以满足训练运行时对内存资源的需求。无法把训练过程中规模庞大的中间数据全部放入内存中，这将会直接导致训练失败。因此，有必要提出针对中间数据的内存管理与数据放置机制，减少训练运行时对系统内存资源的依赖。关于如何合理地在训练系统中管理与放置中间数据这个研究问题，

目前主要有以下几类解决方案:

模型并行化 (Model Parallelism): 当前一些主流的深度学习系统, 其中包括谷歌公司的 DistBelief^[51]、微软公司的 Adam^[52]、以及新加坡国立大学的 SINGA^[53]等等, 试图通过对深度神经网络模型进行切分, 尽可能的将模型均匀地划分到多个 GPU 上, 来完成分布式的并行训练过程。这种类型的并行执行通常被称为模型并行, 可以显著减轻每个 GPU 执行训练任务时的内存压力。然而, 由于神经网络模型被切分到多个节点上, 在单次推理和训练迭代内, 模型内部需要在多个计算节点之间频繁通信同步以获取相关神经元的赋值, 巨大的模型通信同步开销导致该方法的实用性较差, 并行加速比很不理想。

时间换空间 (Trading Time for Space) 思想: 华盛顿大学的陈天奇等人在 MXNet^[54]系统的基础上提出了引入额外计算 (Extra-Computing) 的 GPU 内存优化方法^[55], 将卷积神经网络前向计算过程中得到的特征映射及时丢弃以节约空间消耗, 在反向计算过程中使用额外前向计算的方式将其恢复出来, 使得训练过程能够正常进行下去。该方法本质上是用计算时间换取存储空间, 与正常的迭代计算相比, 该方法引入额外的前向计算, 在显著降低内存使用量的同时也带来了明显的额外时间开销。布朗大学的 Wang 等人^[56]在该思想的基础上, 针对非线性结构的神经网络模型, 提出了一种动态 GPU 内存管理方法——SuperNeurons^[56], 通过构建 GPU 缓存池的方式, 将训练过程中产生的中间数据尽可能多的存储在 GPU 中, 最大限度地保持计算性能, 在 GPU 显存超限后, 引入额外计算方法来缓解显存压力, 训练速度下降的问题得到一定缓解。

数据迁移 (Data Migration) 思想: NVIDIA 公司的 Rhu 等人提出的 vDNN^[57]和卡内基梅隆大学的 Cui 等人提出的 GeePS^[58]主要关注于训练过程中的卷积神经网络特征映射数据迁移机制研究。该方法主要是考虑将 CPU 内存看作是容量无限的内存缓冲池, 将前向过程的特征映射数据暂存到该缓冲池中, 随后通过 PCI-E 总线从 GPU 显存中迁移至 CPU 主存中, 反向计算过程时使用数据预取技术将所需的特征映射再次迁移至 GPU 内存中。该类方法主要基于 CPU-GPU 数据迁移的思想, 能够应对极限尺寸神经网络训练任务所带来的挑战, 但是该类方法对训练性能造成较为显著的影响, 额外时间开销无法忽视。

1.2.2 参数梯度数据优化约减

相对于在单机或单节点上完成深度学习任务，分布式架构能够显著加快深度神经网络训练和预测的速度^[59,60]，因此在学界和产业界均有广泛应用。参数梯度约减方法的目的是削减分布式训练中待传递参数梯度的规模，减轻参数更新阶段跨节点间的通信带宽压力。该类策略都基于统一的观察结论^[61]，即深度网络中的梯度更新存在大量冗余，每次迭代只有极少数梯度对参数更新有贡献。主要分为梯度稀疏化及量化两大类，稀疏化方法直接删去全体梯度中“贡献程度”较小的部分梯度。量化方法对梯度分布的值域空间进行最小化处理。以下分别介绍这些方法。

梯度稀疏化 (Gradient Sparsification): 梯度稀疏化删去神经网络中部分节点的参数梯度，将每次迭代的梯度更新操作限制在一小部分参数中。稀疏化可以分为随机删除和阈值删除两类方法。随机删除方法不区分梯度的差异性，而是随机筛选出一部分参数梯度并继续训练模型，该方法与 dropout 的思想^[16]很相似，在降低梯度通信负载的同时保证深度网络模型的健壮性。另一大类方法根据梯度对参数更新的“贡献程度”（也可称之为当前梯度值在梯度全体中的显著程度），根据不同的度量标准评估相关梯度的贡献度，并对其进行排序输出。随后依据排序结果设置阈值，淘汰贡献程度不高的一部分梯度，仅仅上传那些排序靠前的梯度值。Strom 等人首次尝试用静态阈值^[62]来筛选梯度值，低于阈值以下的梯度元素则不被发送。随后的工作对阈值的筛选机制^[63,64]进行了进一步的尝试，也有自适应阈值^[65]调整方案被提出。

梯度量化 (Gradient Quantization): 梯度量化方法通过值域区间近似的思路，压缩模型中参数梯度的取值区间，减少长尾及异常取值，从而减少梯度规模，进而降低通信开销^[66]。量化可以分为梯度共享和低精度定点化 (Fixed-Points) 两类方法。梯度共享的做法是将参数梯度直接映射到有限可选离散数值空间，使用密码书 (Codebook)^[67]实现有限的参数取值，并与神经元权重连接的一一对应。而低精度定点化方法通过压缩梯度数值的位宽 (bit-width) 变化范围来缩小梯度赋值区间。具体来说，压缩数值位宽的方法将参数梯度的数据精度类型由双精度浮点数 (Double) 变为单精度浮点数 (Float) 或是整数型^[68] (Int)，甚至变为二进制^[69,70] (Binary)，这样的做法实际上

也是在变相地缩小取值空间，有效降低模型交换梯度时的通信开销。

1.3 研究内容和主要贡献

从国内外研究现状可以得知，深度学习中间数据的优化与放置问题已经引起了研究人员的广泛重视，但是目前这类的研究还不够全面。

首先，当前关于深度学习中间数据内存管理优化方面的研究工作往往以内存节约为主要优化目标，而忽视了训练过程中时间性能的下降；除此之外，当前神经网络的结构多样、设计灵活，但当前研究工作尚不能全面兼顾多种类型的网络结构。其次，在深度学习的分布式扩展方面，虽然关于梯度压缩优化的研究工作已有很多，但由于其存在碎片化的问题，缺少整体解决方案，无法在低带宽、梯度数据体量大等的情况下完成分布式通信优化目标，难以满足训练系统对分布式高可扩展能力的要求。最后，以自然语言处理中的神经网络语言模型为出发点，继续开展研究，发现大规模预训练语言模型的训练阶段对计算、内存、通信带宽等多类资源需求强烈，往往需要从内存管理及分布式通信两方面进行协同优化，才能达到高效生成特征表示的研究目的。然而，当前绝大部分与训练系统优化相关的研究工作仅仅从某个单一硬件资源的维度出发，使用单一的内存优化或分布式优化策略，并不能很好的应对多重系统资源匮乏这一现实问题，难以达到理想的训练效率。

针对上述研究现状的不足之处，本文以深度学习系统中多种中间数据的内存管理与通信优化为中心展开研究。如图 1-5 所示，研究目标主要集中在两个方面，第一，如何提高分布式深度学习系统中加速硬件集群的资源利用率，进而达到高内存利用率、高性能计算的目的；第二，如何在兼顾深度学习系统性能的前提下，同时保证深度神经网络的分布式训练效率，加速分布式通信过程，实现模型的快速收敛。具体的研究内容主要包括以下三个方面：

(1) 深度学习系统运行时内存管理机制。由于深度神经网络模型的训练和特征生成阶段会给 GPU 等加速硬件带来巨大内存压力，为解决该问题，提出基于空间复用的特征映射合理放置与内存管理机制。首先对反向计算的执行序列进行细粒度的重排，然后提出神经网络层敏感的内存复用策略，实现高内存利用率的特征数据组织方

式，该研究内容通过在加速硬件与主存之间对特征映射数据进行合理迁移与内存管理，显著减少训练系统运行时的内存消耗，能够对深度神经网络的形态设计和进一步应用研究提供系统级的支撑。

(2) 深度学习系统运行时稀疏通信优化机制。由于深度神经网络的快速训练依赖于分布式加速硬件集群的部署，而加速硬件之间的互联通信带宽并不足以满足迭代的数据交换需求，为解决该问题，提出基于误差反馈的参数梯度稀疏通信优化机制。首先分析参数梯度的数据特性，提出基于神经网络层结构敏感的 Top-k 梯度筛选策略，通过历史累加机制保障训练精度，并执行高效的编码压缩，进一步完善面向稀疏梯度数据的通信优化机制，为梯度快速交换提供了全栈式支撑。该研究内容旨在训练阶段自动对梯度进行稀疏优化，节约运行时互联带宽的消耗，提高训练系统的可扩展性。

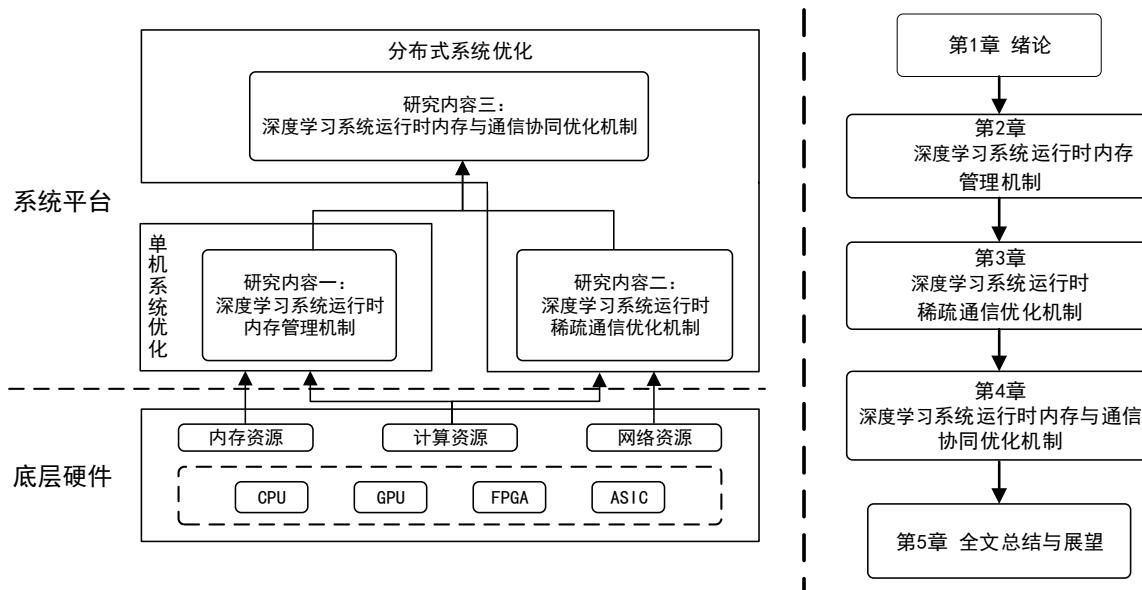


图 1-5 研究内容与论文组织结构

(3) 深度学习系统运行时内存与通信协同优化机制。由于神经网络语言模型等大规模神经网络在训练运行时往往存在体量大、计算及存储开销较大的特点，对训练系统的计算、内存等资源极度渴求。然而当前资源管理和优化方法仅仅从单一角度出发，尚未能够互相协同、互相融合。因此提出对内存与通信资源进行协同优化的思路，首先对内存优化策略进行数据对象增广与重新定义，解决由稀疏通信引起的内存膨胀问题，其次提出镜像参数更新机制，解决参数数据内存优化难以与稀疏通信优化共存

的矛盾。该研究内容旨在保障内存高效的分布式参数稀疏通信机制，兼顾不同类型的神经网络，支持超大规模网络的训练，且能够在一定程度上对训练性能有所保证。

1.4 论文组织结构

论文主要针对深度学习运行时中间数据的内存管理与稀疏通信优化关键技术进行展开，论文组织结构如下：

第一章为绪论部分。首先介绍深度学习的概念，分析深度学习中间数据的特性，并讨论对深度学习中间数据进行内存管理及稀疏优化的必要性；然后介绍国内外研究现状及当前工作存在的不足；接着指出本文的主要研究目标，并列出具研究内容与主要贡献；最后列出了全文组织结构。

第二章为深度学习系统运行时内存管理机制，目的是能够在训练时自动对深度神经网络的内存分配情况进行管理和优化，显著节约运行时显存消耗。

第三章为深度学习系统运行时稀疏通信优化机制，目的是能够在训练时自动对其参数梯度进行稀疏通信优化，显著节约运行时分布式节点间网络带宽消耗。

第四章为深度学习系统运行时内存与通信协同优化机制，目的是能够填补分布式训练系统在多重资源管理优化方面研究的空缺，实现内存与通信两类优化策略的深度融合、协同执行，保障分布式训练系统运行时的整体效率。

第五章归纳全文的创新点并展望未来工作。

2 深度学习系统运行时内存管理机制

在训练过程中，深度卷积神经网络的特征映射数据需要大量的存储空间，而高性能加速硬件的存储资源不足以满足训练时的内存需求。这两者之间的矛盾成为了限制卷积神经网络模型发展的主要瓶颈。本章引入以层为中心的内存重用思想，提出一种基于空间复用的特征映射数据内存管理机制 Layrub。首先对反向计算的执行序列进行细粒度的重排，然后提出神经网络层敏感的内存复用策略，实现高内存利用率的特征数据组织方式，通过在加速硬件与主存之间对特征数据进行合理迁移与放置，训练运行时的内存消耗明显减少，能够对深度神经网络的形态设计和进一步研究提供系统级的支撑。实验表明，与原生的 Caffe 相比，Layrub 平均降低了 58.2% 的内存使用率，最高可达 98.9%，而时间代价是适中的，平均增加了 24.1% 的训练执行时间。实验结果还表明，Layrub 优于一些流行的具备内存优化能力的深度学习系统，如 GeePS、vDNN、MXNet 和 Tensorflow。更重要的是，在解决极限规模的深度学习任务时，Layrub 的优势更加明显，可以在一个 12GB 内存的 GPU 上成功地训练 1517 层的超深卷积神经网络模型。

本章内容组织如下：第 2.1 节讨论并分析了模型训练运行时中间数据存储优化方面存在的问题，并阐述了研究思路；第 2.2 节详细介绍了 Layrub 的内存复用策略和实施流程；第 2.3 节详细介绍了 Layrub 原型系统的设计实现，并解决一些实现过程中的具体问题；第 2.4 节介绍了实验环境和评估设置，通过实验对 Layrub 的内存效率、训练性能等方面进行验证；第 2.5 节对本章研究内容进行总结。

2.1 研究动机

本节首先通过实验结果分析训练运行时深度卷积神经网络模型的内存分配行为特征，并指出对训练运行时特征映射数据进行内存优化的必要性，随后总结了当前相关研究工作的不足之处，最后提出解决该问题的基本思路和设计原则。

2.1.1 问题提出

深度神经网络的复兴使其跃居为最有前途的机器学习技术之一，可以用于分析复杂世界的信息，并在各种实际应用领域达到较高的准确性。通过对神经网络近期发展历程的分析，可知上述突破主要是基于以下两个原因：训练数据体量的不断增加和神经网络架构规模的不断增加。

由于神经网络架构设计是否合理对模型的最终质量起到了决定性作用，因此网络模型的设计原则是研究者重点关注的内容。而模型的深度^[16,17]和宽度^[71]被几乎所有的研究者认为是构建神经网络时最卓有成效的两个因素。“扩展网络深度”的思路最早是由卷积神经网络 VGGNet^[12]的作者提出，而后再由 ResNet 所继承^[16]，随后随机深度策略^[17]将其发扬光大，并成功训练出 1202 层的超深网络。网络深度越大，堆叠的功能层越多，超参数搜索空间的复杂度随之降低，模型参数对特定数据集的过拟合风险也会随之降低，从而最终提升了模型的鲁棒性。与此相提并论的是另一个思路，即“扩展网络宽度”，与 VGGNet 不同的是，GoogLeNet 使用 Inception 模块来实现宽度较宽的层，通过堆叠不同大小的卷积积累了更多的特征映射，此外，在 WRN^[71]和 ResNeXt^[72]中也有证据表明，在残差网络中宽度对模型质量的影响远大于深度对其的影响。用图 2-1 来表示这两种网络结构特性，2-1 (a) 为深度卷积神经网络^[16]，2-1 (b) 为宽度卷积神经网络^[71]。

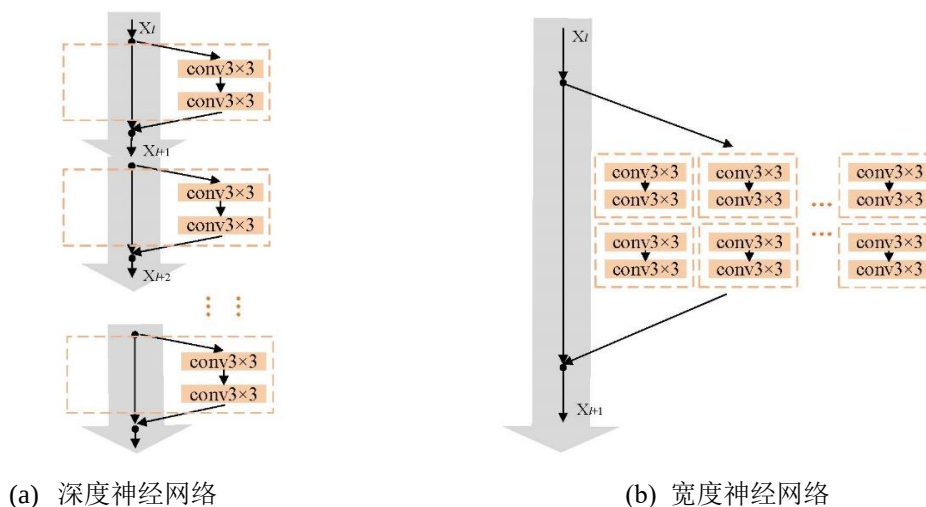


图 2-1 卷积神经网络结构特性

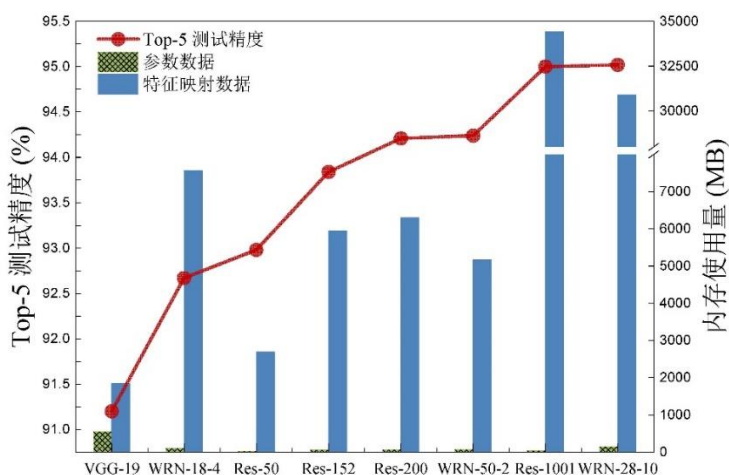


图 2-2 测试精度优势和内存使用情况

注：在 ImageNet 验证集上记录 top-5 的模型精度，图片批处理尺寸 batch size = 32。

随着深度和宽度的稳步增长，深度卷积神经网络模型的训练精度可以显著提高。如图 2-2 所示，实验结果表明，对于训练数据集 ImageNet^[46]，卷积网络模型 ResNet 的深度从 50 增加至 1001，宽度从 2 增加至 10，此时的测试精度可以从 91.2% 提升至 95% 以上。而与此同时，随着 ResNet 模型精度的提升，模型训练过程中的内存使用量将接近 60GB。可以这么说，深度学习的快速发展正在引发一场构建更深或更宽网络模型的竞赛，最终促使超大规模网络模型不断涌现。然而，超大模型的规模决定了它们在训练过程中需要更多的计算和内存资源。如表 1-1 所示，使用 ResNet 模型完成图像识别任务，152 层的 ResNet 需要占用 11.61GB 内存，每次对 32 张图像进行批处理需要 113 亿次浮点运算。

为解决超大模型难以训练、推理缓慢的问题，很多加速硬件，如 GPU、FPGA 和 ASIC 被引入到深度学习系统中，以加速训练过程。然而，深度神经网络的特征映射数据需要大量的存储空间，而当前主流加速硬件的内存资源十分有限。例如，现有的训练系统（如 Caffe^[73]，TensorFlow^[74]，以及 Angel^[75]等）无法在单一 GPU 上处理类似于 ResNet 的超大规模卷积神经网络，将会直接导致训练任务的失败。其次，为了避免内存不足的问题，当前的训练系统往往依赖于用户来决定模型内存开销是否适合当前 GPU 节点，这对于普通用户来说是非常困难的。因此需要找到一种系统优化方法，能够以弹性方式管理与放置模型训练运行时所产生的中间数据。

2.1.2 现有技术分析

当前已有一系列研究工作致力于设计内存管理和数据放置策略来实现单 GPU 上的卷积神经网络模型训练，在表 2-1 中列出了实现内存优化技术的深度学习系统，并总结了它们所使用的内存管理和数据放置策略。除了 Caffe 是直接为整个网络分配内存空间之外，几乎所有其他系统都是以层为中心、为前馈计算所涉及的相关层整体地分配内存空间。具体来说，基于 Caffe 的 vDNN^[57]和 GeePS^[58]都引入了 GPU 内存池来临时存储中间数据，并在训练过程中在 GPU 和 CPU 内存之间迁移数据以减少 GPU 内存消耗。然而，GPU 内存池会导致双倍的 GPU 内存消耗和显著的性能损失，这将在后面小节中具体讨论。此外，基于 MXNet 上的 Memonger^[55]方法提出通过执行额外的前向计算来提升内存效率，但是同时导致了计算资源的浪费。此外，从表格中可以看出，上述的内存优化策略均无法很好地应对超深网络以及超宽网络。这是因为当前的研究工作都倾向于进行保守的内存分配，所分配的空间可以容纳当前层的所有中间数据，这时内存资源的浪费问题仍然存在。而与此同时，现有的方案几乎都将特征映射数据与其梯度共同驻留在内存中，而实际上这中间有 50%左右的内存空间被浪费，具体问题将在 2.1.3 小节进一步讨论。

表 2-1 现有训练系统对比

代表性系统	数据放置机制	内存优化策略	性能开销	支持特性			
				数据复用	数据迁移	支持宽度网络	支持极限规模训练
Caffe ^[73]	整个网络 (d+g)	就地放置	-	×	×	×	×
TensorFlow ^[74]	相关的层 (d/g)	最佳适应	-	×	×	×	×
GeePS ^[57]	相关的层 (d+g)	内存池	大	×	√	×	×
vDNN ^[58]	相关的层 (d+g)	内存池	大	×	√	×	×
MXNet ^[54,55]	相关的层 (d/g)	重新计算	中等	×	×	×	√

注：d：特征映射数据；g：对应的梯度；d+g：为特征数据和对应梯度分配内存；d/g：为其中之一分配内存；因为 Caffe 和 TensorFlow 没有显式的内存优化策略，所以它们的性能开销不具备可比性。

2.1.3 解决方案设计

2.1.2 小节讨论了当前关于中间数据存储优化与放置方面的研究工作，然而，由于

神经网络的结构多样、设计灵活，现有策略不能很好地兼顾多种类型的卷积神经网络结构，内存优化效果不明显。不仅如此，由于其往往以内存节约为首要优化目标，而忽视了训练过程中时间性能的下降，也是当前研究工作存在的不足之处。

回顾 1.1 节中对中间数据的分类与特性总结，并分析相关的反向传播算法，遵循以层为中心的链式法则，从算法原理和体系结构两个层面出发，分析研究模型在训练过程中的内存分配行为规则，并提供三个重要观察结果。根据这些观察结论获得启发，进而提出解决方案，即基于空间复用的内存管理机制，兼顾神经网络的深度和宽度特性、支持超大规模网络的训练，且能够在一定程度上对训练性能有所保证。

第一个观察结论：特征映射数据和其对应梯度数据的内存占用情况同样值得关注。首先，可以注意到后向阶段需要的内存空间大约是前向阶段的两倍，这是由于后向阶段的计算过程不仅使用每个中间层（输入数据层除外）的特征映射和参数，而且还使用它们的对应梯度。其次，回顾反向传播算法，注意到这样一个现象： $|P|=|\nabla P|$ 且 $|D|=|\nabla D|$ 。这意味着参数 P 及其梯度 ∇P 具有相同的内存占用规模，同时，特征映射数据 D 及其梯度 ∇D 也具有相同特点，即两者的规模是相等的，具体情况如图 2-3 所示，如 Conv 和 ∇ Conv、FC 和 ∇ FC。该观察结论给在这二者（即特征映射数据及其梯度映射）之间进行层内的内存空间重用提供了可能性，研究动机由此产生。

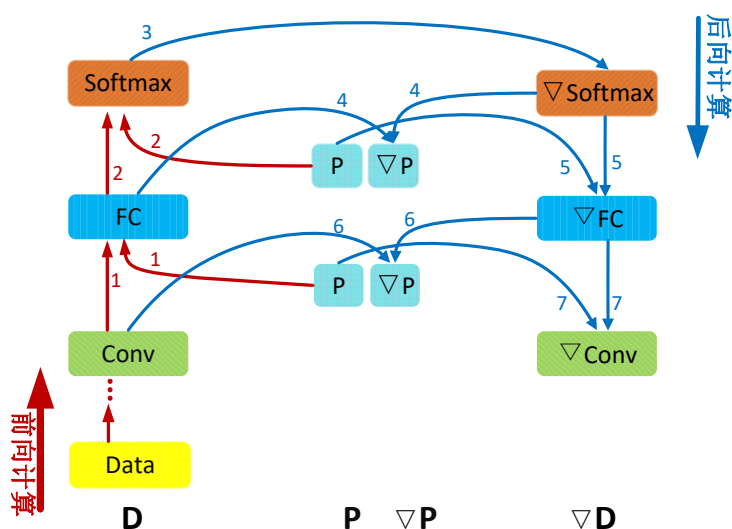


图 2-3 卷积神经网络的训练过程

注：这里的 P 和 ∇P 指参数及其梯度， D 和 ∇D 指特征映射数据及其梯度。

第二个观察结论：模型中不同层之间的内存占用情况是相互独立的。首先，从图 2-3 所示的训练阶段执行流可知，无论网络的深度或宽度如何，无论网络的结构是线性还是非线性结构^[12]，训练过程都是逐层进行的^[16]。其次，在整个训练过程中，实际的计算过程是以层为粒度，而计算所需内存仅为单层网络的内存占用数量，因而不需要将所有层的数据都放置到 GPU 内存中。而这些观察结论给予以下启发：可以考虑在那些内存占用情况相互独立的层之间进行跨层的内存重用，在保证层内前向计算正确性的前提下，保持最少的内存使用规模。

第三个观察结论：卷积神经网络的特征映射数据及其梯度占用了较大比例的 GPU 内存空间。值得注意的是，尽管空间重用的思想也可以应用到网络模型参数及其梯度数据的优化机制中，本章并未对参数及其梯度进行内存优化。这主要是由于以下原因，首先，如表 1-1 及图 2-2 所示，在众多卷积神经网络中，参数的规模大小远远小于特征映射及其梯度的规模，对其进行内存优化，并不会给整体内存节约效果带来显著提升。其次，在分布式的训练过程中，参数梯度需要频繁的在不同节点之间交换和聚合，将参数数据长期驻留在内存中能够保障计算正确性，并且尽可能保持训练速度。

总的来说，受上述一系列观察结论的启发，在卷积神经网络训练阶段，提出一种基于空间复用的数据放置与内存优化机制，并基于该内存空间复用的思想，实现一个兼顾内存效率与训练性能的训练系统 Layrub。Layrub 的几个重要设计目标简要描述如下：

同时考虑神经网络的深度和宽度特性：首先，针对深度网络，提出了一种基于跨层空间复用的 CPU-GPU 数据迁移放置策略，无论神经网络结构设计的深度如何增加，在所有的训练迭代中，GPU 的内存消耗都能保持一定的比例。其次，针对宽度网络，提出了一种基于层内空间复用的数据合理放置策略，特征映射及其对应梯度数据可以分时复用同一组 GPU 内存空间，能够有效降低层内内存空间占用比例。

能够支持超大规模网络的单机单卡训练：考虑将层内和层间的两种内存空间复用策略进行结合，提出混合的空间复用策略，分析以层为中心的顺序执行过程，挖掘不同层之间计算和内存访问操作的独立性，并通过重排这些操作来复用内存空间，减少内存消耗。该策略能够与大多数形态各异的神经网络协同工作，并在单个 GPU 上获

得最大化的内存节约效果，显著改善了极限规模的卷积神经网络无法在单个 GPU 上运行的现状。

能够在一定程度上对训练性能有所保证：使用 CUDA 流水线技术，在 GPU 内存与 CPU 主存间调度放置特征映射数据，对迭代计算流和数据迁移流并发执行，尽可能在计算过程中隐藏数据通信时延，达到最大程度时间重叠，把总运行时间几乎缩短到单纯计算时间，达到与现有主流深度学习系统相当的性能。

2.2 基于空间复用的数据放置机制

回顾 2.1.3 小节可知，Layrub 的主要目标是管理训练运行时中间数据的分时及分区域放置，尽可能的减少卷积神经网络对 GPU 内存资源消耗。在本节中，首先对迭代计算执行过程的操作序列进行协调和重排，以此来支撑层结构敏感的空间重用策略，然后从算法和架构的角度介绍所提出的三种策略：层内的数据分时放置、跨层的数据迁移放置、以及混合策略，以此来最大程度的复用 GPU 的内存空间。

2.2.1 训练过程的操作执行序列重排

作为深度学习的算法核心，以层为中心的前馈计算通常在多个迭代中反复地执行前向计算和后向计算两个阶段。“以层为中心”这个概念可以理解为，前馈计算过程是由一系列以层结构为执行粒度的、保持顺序执行的操作序列组成，具体如图 2-3 所示。也就是说，模型其中一个层的前向操作不能比前一层的前向操作发生得更早，因为它需要前一层的输出数据作为其输入数据。类似地，某一层的后向操作也依赖于它的后面一层的数据。而多个操作之间的这些数据依赖关系直接决定了每次迭代中数据的放置方式和内存消耗情况。值得注意的是，本节策略的核心思想是对内存空间的复用，其动机是基于前文总结的观察结论：一旦成功解耦前后访存操作之间的依赖关系，当前层神经元数据的存储空间就可以被其他层的神经元数据所复用。换句话说，只要能够保证为多组数据分配内存空间的操作序列之间彼此独立，那么这些数据就可以重复利用当前的内存空间。因此，为了重新组织输入或输出数据的放置位置，可以尝试

挖掘这些顺序操作之间的独立性，对执行序列进行适当的重排。

如图 2-3 所示，以一个简单的卷积神经网络模型训练过程为例，该模型仅包含一个输入层、一个卷积层、一个全连接层、一个 softmax 输出层，这里对原始的后向阶段运算操作（4）和运算操作（5）进行观察和分析。前者用于计算全连接层（FC）的参数梯度 $\nabla \mathbf{P}$ ，后者用于计算全连接层神经元对应的特征映射梯度 $\nabla \mathbf{FC}$ 。这些类型的操作顺序是可以互换的，不会影响迭代计算的正确性。因此，如果以层为计算的基本粒度，首先进行参数梯度的计算，随后再进行相应层神经元特征映射梯度的计算，那么这两种操作的内存占用将会是相互独立的。经过适当的调整重排，将后向计算阶段的操作执行顺序固定：任意一个中间层的参数梯度计算操作优先于同层的神经元特征映射对应梯度的计算操作。也就是说，操作（4）被重排在操作（5）之前，操作（6）被重排在操作（7）之前，其余同类型的操作顺序也可以按照同样的方式进行重排。

2.2.2 基于层内空间复用的数据分时放置策略

本节提出基于层内空间复用的数据分时放置策略（Intra-Layer Memory Reuse），旨在解决模型训练过程中存在的层结构内部的内存膨胀问题，特别是针对宽度模型来说，由于大量神经元集中的、稠密的分布在某些层的内部，这些层的卷积操作会产生海量的特征映射数据，对 GPU 的内存消耗体量非常巨大。在 2.2.1 节中，为保持运行时层内部两个主要操作数（求得特征映射及其梯度的运算操作）相互独立，对操作执行序列进行重新组织排列。在此基础上，本节提出一种层内空间的复用策略，该策略依赖于层内部前后向计算之间的操作独立性，在后向计算时特征映射的梯度使用前向计算中其对应层特征映射的存储空间。因此，训练过程的数据放置机制被重新定义：特征映射的梯度能够重用相同层特征映射的 GPU 内存空间。

如图 2-4 所示，运算操作（1）存在于全连接层的前向计算阶段，运算操作（4）和（5）则位于全连接层的后向计算阶段。分时放置策略具体为：操作（1）执行运算得到神经元特征映射数据 $\mathbf{FC}$ ，放在该值预先分配的内存空间中；该值首先会被操作（4）访问并使用，但是由于此后该值将不会再被使用，因此该值所在内存空间的占用状态被解除；随后，运算操作（5）使用该内存空间来放置计算得到的梯度数据 $\nabla \mathbf{FC}$ 。

同理可知，其余的操作序列也可以以相同的方式执行：例如，操作（7）将重用卷积层特征映射 Conv 的内存空间，用以计算梯度数据 ∇Conv 。

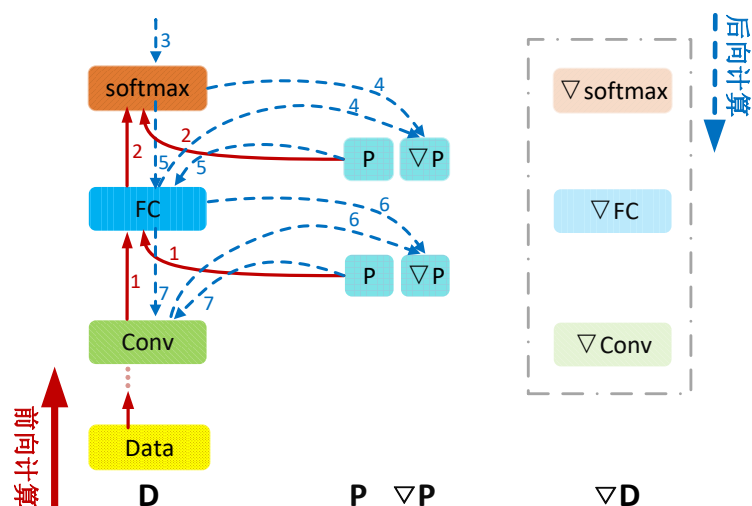


图 2-4 基于层内空间复用的数据分时放置

本节所提出的层内空间复用策略以层结构为基本操作单位，将前向计算阶段所需的内存空间与后向计算阶段所需的内存空间进行重复利用，这意味着在模型相应层中节省的内存比例可以高达 50%。可以看出，基于层内空间复用的方法，对宽度卷积神经网络模型的训练过程最为有利。对于超宽模型，现有的保守方法只能减少约 10% 的内存使用量，与之相比，本节所提出的层内策略在内存节省方面可以达到更为显著的效果。第 2.3 节将会详细描述层内策略的实现细节，并在第 2.4 节中通过实验证明该策略的有效性。

2.2.3 基于跨层空间复用的 CPU-GPU 数据迁移放置策略

尽管基于层内复用的数据放置策略能够在任一层内部进行空间的重用，以此节省内存空间，但是它的作用范围仅仅在任一层的内部，无法在不同层之间跨层重用内存空间。对于另一大类沿深度加深趋势发展的卷积神经网络模型（如 VGGNet、ResNet 等），在使用层内空间复用的优化方法之后，该类模型对 GPU 内存资源的消耗情况仍然非常严重。为了解决这个问题，引入另一种跨层的内存重用和数据迁移策略（Inter-Layer Memory Reuse and Data Migration），该策略在不同层的连续操作序列之间插入

数据迁移操作，在 CPU 主存和 GPU 内存之间迁移相关数据，可以重用一定数量的 GPU 内存空间。该思路既不违背训练过程的算法正确性，又考虑到了基于异构多核硬件的深度学习系统的体系结构特性。具体来说，本节的迁移放置策略主要是依据下面的两点观察结论而提出的。

首先，观察图 2-5，在前向和后向计算序列中，当某一层的参数梯度计算还远远未轮到执行时，也就是如图所示的操作（7）的执行时序是远落后于操作（1）的，那么，该层的特征映射 Conv 可以暂时从 GPU 内存迁移到 CPU 内存进行存储，该卷积层特征映射 Conv 的内存空间可以被其它层重新利用。而此后，在执行操作（6）和操作（7）之前，卷积层的数据 Conv 将被迁移回到 GPU 内存。这与 2.2.1 节得出的结论一致：运算操作和内存占用都遵循逐层执行的方式。因此，来自不同层的、相互独立的操作序列重复地利用一定数量的内存空间这一思路是可行的。

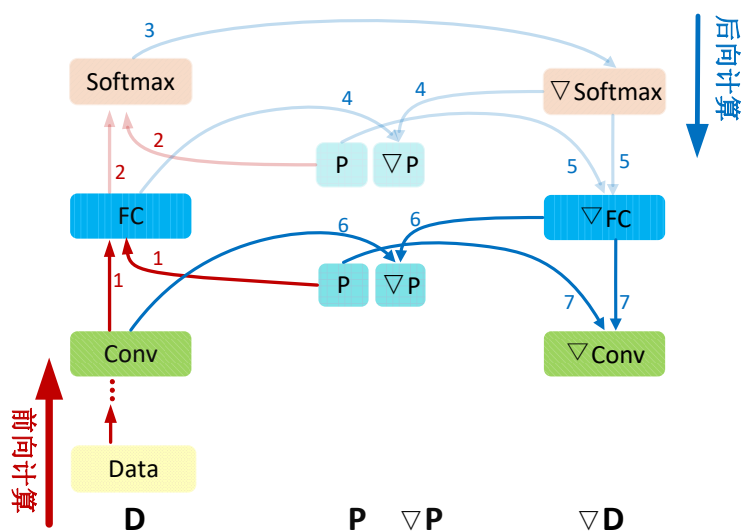


图 2-5 基于跨层空间复用的数据迁移

其次，考虑到前馈计算的算法原理限制，任一层计算参数梯度的方式如图 2-5 所示，想要计算得到卷积层的参数梯度 ∇P ，必须同时使用到当前卷积层的特征映射数据 Conv 和前面全连接层的神经元特征映射梯度数据 ∇FC 。这意味着单次训练迭代中只需要保留那些将来会在后向计算时被使用的中间数据，在不需要时迁出 GPU 内存，在需要时提前迁回 GPU 内存。在这里，由于 CPU 主存远比 GPU 内存容量要大，CPU 内存可以被作为临时存储这些中间数据的缓冲区。

从系统资源调度的角度来看,这种基于跨层空间复用的策略其核心思想实际上是经典问题“生产者-消费者”问题的一个变体。而相对应于本节的实际需求而言,从训练过程中内存消耗的角度考虑,有限的 GPU 内存可以理解为上述的“资源”,深度神经网络模型中以层为中心的运算操作可以理解为“消费者”,“消费者”需要足够的 GPU 内存来存储神经元特征映射数据。与原始的内存分配计划(给“消费者”一次性分配完他所需的所有资源)相比,更适合的做法应该是:在“消费者”真正需要这些资源的时候,再将对应的资源分配给他们,并且在使用完这些资源后立即释放它们。

因此本节提出了一种数据迁移放置策略,使“消费者”能够及时的重用 GPU 内存资源。然而,如何选择合适的时间点上开始在 GPU 和 CPU 之间迁移数据是一个值得讨论的关键问题。此外,卷积神经网络模型往往呈现不同形态(线性或非线性的网络结构),这将进一步加剧这一问题的严重性。考虑到这一点,应该首先记录每层数据的内存访问顺序,跟踪该数据是否被其他层引用,并记录对应的引用计数。通过这种方式,可以建立训练过程的细粒度内存访问序列,用以确定何时迁移数据。此外,考虑到数据迁移的开销,启用了两个 CUDA 流:一个是默认流,作为 cuDNN 加速库的应用接口^[76,77],接管向前和向后的所有计算;而另一个流专门用于管理数据迁移操作,通过 PCI-E 在 CPU 和 GPU 之间进行数据迁移。这些流可以在 GPU 上并发执行,使得计算与迁移两个任务能够重叠,尽可能的隐藏迁移数据造成的时间延迟。实现细节在 2.3 节中描述。值得注意的是,在任意网络执行以层为中心的前馈计算时,每一层对内存的使用规模是不同的。因此,为了简单起见,选择所需内存片段中尺寸最大的两片区域(如 Conv 和 ∇ Conv、FC 和 ∇ FC)作为反复重用的资源,迭代训练时每一层会连续地请求和重用它们。

2.2.4 混合的空间复用策略

回顾 2.2.2 及 2.2.3 小节,层内策略能够在层内级别上重用内存空间,而跨层策略在层间级别上重用内存空间。它们分别在宽度和深度两种形态的神经网络中发挥各自的优势。然而,当网络模型的宽度和深度形态并存时,单一使用某一种策略,则无法解决混合形态模型的内存爆炸问题。也就是说,两种策略在应对超大尺寸的网络模型

时存在着各自的局限性，具体的对比结果将在 2.4 节中进行阐述。

因此，需要同时考虑这两种策略，提出一种混合的以层为中心的内存重用策略（Hybrid Layer-Centric Memory Reuse），从而完美地考虑几乎所有的网络结构。经过分析可知，只要预先定义并对前后向计算操作序列进行重排（如 2.2.1 节所述），这两种策略在混合执行的过程中不会发生冲突。由于计算序列重排，每个层的参数梯度计算将会优先于神经元特征映射的梯度计算，因此层内空间复用策略不会再影响连续多层之间的操作独立性。层与层之间内存占用的独立性能得到保持，从而可以保证跨层策略的成功执行。如图 2-6 所示，在使用层内策略时，仍然可以执行以层为中心的梯度 ∇Conv 的计算任务，并且也只需要两块 GPU 的内存空间（一块用于存储 ∇FC ，另一块用于存储 ∇Conv ）。通过这种方式，跨层策略可以很自然地实现。总的来说，本节提出以层为中心的混合策略，在层内策略的基础上构建跨层策略，这样迭代计算所需的 GPU 内存空间将会大大减少。对比实验结果证明了该方法的有效性，该策略能够在单 GPU 上为超大规模的训练任务获得最大程度的内存收益。

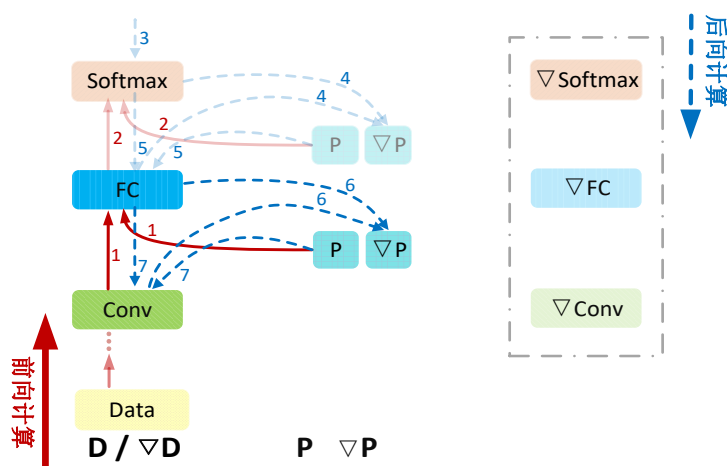


图 2-6 混合的空间复用策略

2.3 Layrub 原型系统设计与实现

本节首先描述 Layrub 的设计细节和算法实现，以深度学习系统 Caffe 为基础进行代码实现，然后解决一些实现过程中的具体问题，对 ReLU 激活函数进行适当改良，并完成并行任务之间的同步控制。

2.3.1 算法实现

Layrub 作为一个 c++库，用于管理基于 GPU 的深度学习系统的数据放置机制，可以实现到不同的系统上，比如 MXNet 和 TensorFlow。下面以深度学习系统 Caffe 为例来说明 Layrub 的实现细节。神经网络模型通常部署在单个 CPU 进程中，该进程启动 NVIDIA 库函数调用或 CUDA 内核在 GPU 上执行具体的计算。这个过程将会调用 Layrub 函数来管理数据放置和内存空间分配，并接管正常的计算迭代。

算法 2-1: Layrub 训练迭代

```
1.  function Train(net)
2.      //L:一个网络(net)的所有层; T: 总的迭代次数
3.      for  $l = 1 \rightarrow L$  do
4.          Tracer[l].addRef() //初始化一个网络，为每一层  $l$  的数据设置引用计数(Ref)
5.      end for
6.      for  $i = 1 \rightarrow T$  do //在迭代  $i$  中，Tracer 跟踪每个层的 active Ref
7.          for  $l = 1 \rightarrow L$  do //前向计算过程
8.              layer[l].Forward(default_stream) //前后向计算由 default_stream 执行
9.              Tracer[l].subRef()
10.             //当  $l$  的数据与其他层无依赖关系时，它可以被 migrate_stream 迁移
11.             if Tracer[l].Ref == 0 then
12.                 InterLayer[l].Offload(migrate_stream)
13.                 Synchronize() //同步每个层的 default_stream 和 migrate_stream
14.                 //数据迁移后，当前层的空间将被下一层重用
15.                 InterLayer[l].Reuse()
16.             end if
17.          end for
18.          for  $l = L \rightarrow 1$  do //后向计算过程
19.              //内存空间可以在层内层次上前后重用
20.              layer[l].Backward(IntraLayer[l].Reuse(), default_stream)
21.              //需要的数据会被 migrate_stream 提前迁移到 GPU
22.              InterLayer[l].Loadin(migrate_stream)
23.              Synchronize() //同步每个层的 default_stream 和 migrate_stream
24.          end for
25.      end for
26. end function
```

算法 2-1 对应对其进行具体实现，给出了一个使用 Layrub 进行训练迭代的示例。

在正式训练迭代开始前，对神经网络进行一次初始化迭代来获取各层的数据依赖信息。初始化迭代不涉及到真实的训练数据，并不进行正式的参数更新，只在神经网络模型中逐层地完成内存分配，并进行前向和反向计算。

训练一个 n 层线性卷积神经网络，特征映射及参数等各类中间数据的内存成本均与层数 n 呈线性关系，一般而言，梯度与各类中间数据一一对应，因此，特征映射梯度的内存成本与神经网络的层数 n 也成线性关系，整体的内存成本可看作为 $O(2n)$ 。在 2.2 小节提出了对中间特征映射数据的内存复用策略，可以大大减少深度卷积神经网络训练运行时的内存使用量。对于该 n 层神经网络而言，只需要在 GPU 内存中分配两个适当大小的内存块，在迭代内循环地对其复用，就可以在迭代计算中正常保存这些特征映射梯度，因此可以将特征映射相关数据的内存使用量降低到 $O(1)$ ，此时内存成本为 $O(1)$ ，即内存用量为常数级别。

表 2-2 总结并具体解释了 Layrub 用于管理中间数据内存分配的函数调用情况，对完成数据依赖关系构建（Tracer）、层内空间分配（IntraLayer）、层间数据迁移（InterLayer）等功能的函数调用分别进行了封装。

表 2-2 Layrub 函数调用

函数 API	所属类	变量	具体描述
addRef()	Tracer	层的索引	遍历网络模型，设置各层数据的引用计数
subRef()	Tracer	层的索引	追踪前向阶段的引用
reuse()	IntraLayer	层的类型，指向特征数据及其梯度的指针	共享特征数据与其梯度的指针
Offload()	InterLayer	层的索引，引用计数	将数据从 GPU 内存转移到 CPU
Loadin()	InterLayer	层的索引，引用计数	将数据从 CPU 内存加载到 GPU
reuse()	InterLayer	层的类型，指向特征数据及其梯度的指针	跨层共享指针

2.3.2 内存高效的 ReLU 激活函数

在提出的层内空间重用策略中，神经元特征映射数据的存储空间被同一层的梯度所重用。因此，神经元特征数据和相应的梯度数据相互独立。当前几乎所有类型的功能层都在遵守这种独立性，然而，在激活函数 ReLU^[78]上，由于在求得反向误差 $\delta^{(ReLU)}$

时需要神经元特征数据 $z^{(ReLU)}$ ，具体如公式 (2-1) 所示，数学原理决定了层内复用策略很难实现到 ReLU 层上。换句话说，执行 ReLU 层时无法在前后向计算中复用空间。

$$\delta^{(ReLU)} = \delta^{(l+1)} * f_{(ReLU)}'(z^{(ReLU)}) = \begin{cases} 0, & z^{(ReLU)} < 0 \\ \delta^{(l+1)}, & z^{(ReLU)} \geq 0 \end{cases} \quad (2-1)$$

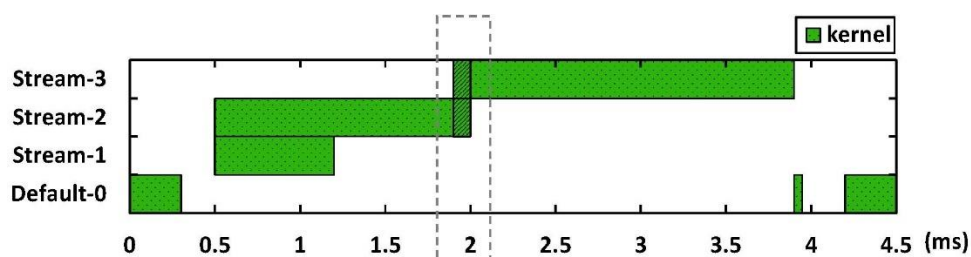
为解决上述问题，进一步分析数学原理，可以得知：这里使用 $z^{(ReLU)}$ 的目的只是对其进行正负值的判断，也就是说， $z^{(ReLU)}$ 并不参与 32 位浮点计算，因此考虑提出一个内存高效的 ReLU 激活函数（Memory-Efficient ReLU），压缩原始数据，并将其存储在一个额外的 CPU 内存空间。在将压缩后的数据转移到一个额外的区域后，神经元特征数据的原始存储空间可以被相应的梯度数据重用，用以完整实现层内策略。在具体实现中，由于 char 类型是 CUDA 函数支持的最短长度（8 位），因此选择 char 类型作为压缩后神经元数据 $z^{(ReLU)}$ 的数据类型，额外的内存消耗减少到原始数据大小的 1/4，而且几乎没有额外的时间消耗。

2.3.3 任务的同步控制

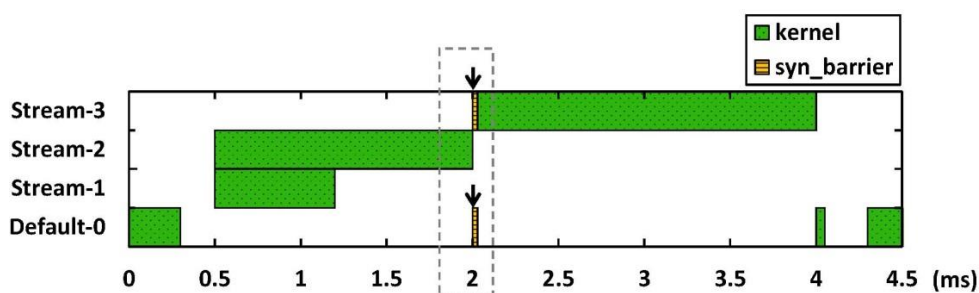
流行的深度学习系统如 Caffe 和 TensorFlow 都是调用一系列基于 CUDA^[77] 的运算加速库，在此基础上执行 GPU 训练任务。这些加速库包括通用的数学库如 cuBLAS^[79]，领域特定库如 cuDNN^[76]。由于 cuDNN 指定的训练任务只能通过流水线技术（CUDA stream）来执行，因此使用 cuDNN 加速库的深度学习系统也都在流中执行训练任务。图 2-7 显示了在卷积层执行后向计算时 NVIDIA Profiler^[77] 捕获的四个内核的执行时间线。对于卷积层，cuDNN 使 3 个流（图 2-7 中的 stream-1、2、3）分别获得偏置梯度、参数梯度和神经元特征数据梯度。此外，Caffe 还启用了默认流（图 2-7 中的 default-0）来控制调度其余的 CUDA 操作，这些操作与后向过程中的梯度计算无关。

由于不同流的内核执行状态是完全异步的，因此并发内核之间使用的数据应当保证相互独立，一旦出现依赖关系，将会直接影响内核所执行任务的正确性。以层内空间重用策略为例，特征数据的梯度将会重用特征数据的存储空间，在计算获取偏置梯

度和参数梯度时（分别为 stream-1 和 stream-2），需要读取特征数据；当计算获取特征数据的梯度时（为 stream-3），需要向特征数据的内存空间写回新数据。这时可以发现 stream-2 和 stream-3 均涉及到特征映射数据，两者存在相互依赖关系。而这种数据依赖关系将会导致后向计算出现错误。想要在使用 cuDNN 加速库的训练框架上实现计算操作顺序的重排，就不得不面对这个问题。例如，图 2-7（a）中绿色重叠条带就是指由 stream-2 和 stream-3 之间并发执行内核而引起的数据冲突。在实现层内空间重用策略时，如果不对这种数据冲突问题进行提前干预，当训练完成时，模型的训练精度将会大幅降低，可能从正常的 75% 下降到 56%。而这种模型精度的急剧下降是不可接受的。因此，本节提出一种基于栅栏操作的同步控制方法，能够完全避免精度下降，保证计算的正确性。



(a) 无同步 barrier 的 kernel 执行时间线



(b) 插入同步 barrier 的 kernel 执行时间线

图 2-7 同步 barrier 对内核执行的影响

任务的同步控制：为了解决精度下降的问题，同时保证训练的内存高效性，可以把相互依赖的任务从有参层（如卷积层）的所有派生任务中分离出来。同步栅栏操作（图 2-7（b）中的两个带箭头的黄色条带）被插入到相应的流中，用来隔离这些相互依赖的任务。同步栅栏能够对 CUDA 调用的执行顺序进行管理，因此在所有其他流中的全部操作完成之前，当前流中被指定的 CUDA 操作将不会启动。在本节的示例

中，在 default-1 和 stream-2 的偏置和参数的梯度计算完成之前，stream-3 中特征数据的梯度计算不会开始。因此，即使内核启动是异步的，也不会再发生不同流的内核重叠执行，由并发内核执行造成的数据依赖问题也不复存在，最终可以完全避免精度下降的问题。

2.4 实验评估与分析

本节评估 Layrub 在基于 GPU 的深度学习系统中的性能表现，选择了具有代表性的深度神经网络模型和训练数据集，目前这些模型已经广泛应用于图像分类、语言模型和视频描述等领域。

2.4.1 实验设置

实验环境设置：本节使用两种 GPU 进行实验，具体型号与配置如表 2-3 所示。GPU 通过 PCI-E 总线（gen3）与 CPU 通信，该总线提供最高 16GB/秒的数据传输带宽。除另有说明外，所有实验均在 NVIDIA K40M 上进行。在分布式实验中，机器之间通过 1Gbps 的以太网相互连接，每台机器安装一个 K40M。Layrub 的实现和部署均在深度学习框架 Caffe 上进行。

表 2-3 系统配置描述

GPU 配置	机器配置	软件环境
NVIDIA GeForce GTX TITAN Z GPU, 6082MB	Intel Core i7-920 (2.67GHz 8-core), 64GB RAM	64-bit Ubuntu 14.04, CUDA toolkit 7.5, cuDNN v4
NVIDIA Tesla K40M GPU, 11580MB	Intel Xeon E5-2620 (2.40GHz 24-core), 256GB RAM	64-bit Ubuntu 14.04, CUDA toolkit 7.5, cuDNN v4

模型和数据集：实验中使用了一批代表性的卷积神经网络模型，包括 LeNet^[3], Cifar10_quick^[45], AlexNet^[11]及其变体 CaffeNet, VGGNet^[12], GoogLeNet^[15], ResNet^[17], WRN^[71], 其中 LSTM-RNN^[80]为神经网络语言模型, LSTM-CNN^[81]为视频描述类模型。模型的所有配置都遵循默认超参数（学习率、动量等）。此外，使用了两个著名的图像分类数据集：Cifar-10^[45]和 ImageNet^[46]；一个典型的视频数据集：UCF101^[82]。

基线深度学习系统：为了验证所提出的三种策略的内存效率，首先把 Caffe 的原

始版本（无内存优化策略）作为基准。实验目的是测量中间数据的内存开销。由于在大多数系统（如 Caffe 和 MXNet）中，在开始迭代计算之前会默认给所有的中间数据静态的分配内存空间。因此可以准确地测算出特征数据及其相应梯度的内存消耗。其次，为了验证 Layrub 在各方面的性能优势，同时也与其他的代表性系统进行对比，如基于 Caffe 的内存优化系统 GeePS^[57]和 vDNN^[58]，MXNet^[54]，与基于 MXNet 的内存优化系统 memonger^[55]，TensorFlow^[74]。在实验中使用 NVIDIA 系统管理接口(nvidia-smi) 测算运行时的内存使用情况。

2.4.2 内存效率评估

Layrub 在深度网络模型训练中的优势：为了证明 Layrub 在深度网络模型训练中的优势，在 ResNet 模型构建环节中不断增加网络层数，设计得到的残差网络原型包括 ResNet-34/50/101/152/1001。图 2-8 的结果显示，当网络的深度从 50 增加到 1001，GPU 的内存分配量仍然能够保持在 759.5MB 左右。也就是说，不管网络的深度如何增加，内存消耗情况均能够保持在一个基本恒定的水平。Layrub 增加了探索更深网络模型的可能性，使设计人员无需考虑 GPU 的内存限制。

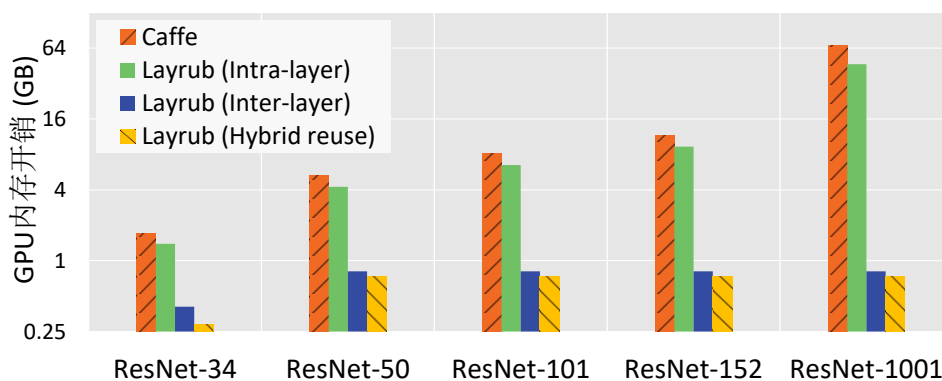


图 2-8 深度模型内存分配对比

注：使用 ImageNet 数据集，批处理尺寸 batch size=32。

图 2-8 表明与基线系统（Caffe）相比，跨层空间复用（Inter-Layer Memory Reuse）策略明显更加适合于深度模型，优化效果更为显著，例如，在 ResNet-1001 上能够获得高达 98.9%的内存节省比例。然而，跨层策略也存在自身的局限性，并不能普适于所有形态的网络结构，例如 LeNet、Cifar10_quick，由于其深层特征并不明显，因此

优化效果也不够显著。因此提出了混合复用策略来弥补上述两种策略的不足。

Layrub 在宽度网络模型训练中的优势：为了证明 Layrub 在宽度网络模型方面的优势，将 WRN 模型宽度从 10 扩展到 256。注意，宽度为 1 的 WRN-16-1 实际上是一个深度为 16 的原始 ResNet 模型。Zagoruyko 和 Komodakis 研究了卷积层的宽度对分类精度的影响^[71]，通过增加更多的过滤器到卷积层，如 WRN-16-10 从 64 个过滤器增加到 640 个过滤器。本节采用类似的方法来增加 WRN 的宽度，得到图 2-9 中的 WRN 配置。为了排除由网络深度带来的潜在影响，将模型深度从 16 降低到 8，可以观察到层内内存复用策略（Intra-Layer Memory Reuse）在宽度模型方面具有潜力。在 WRN-8-128 和 WRN-8-256 中，跨层内存复用策略表现较差，平均内存使用量的节约幅度仅为 13.6%。相反的，层内策略可以将内存使用量降低到接近 50%，将内存用量稳定到一定的水平。

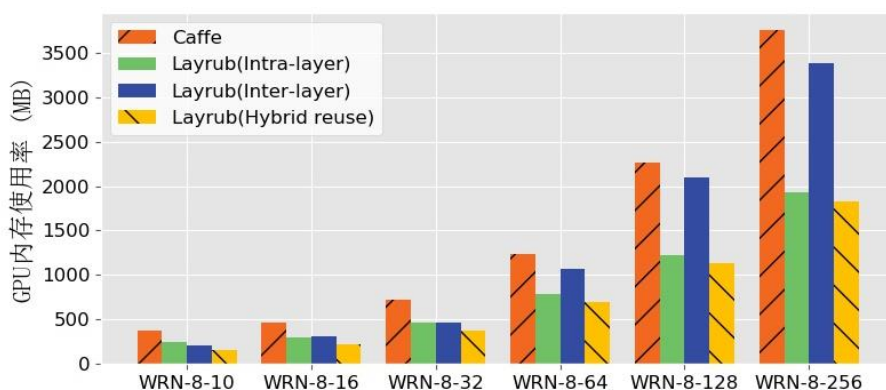


图 2-9 宽度模型内存分配对比

注：使用 Cifar-10 数据集，批处理尺寸 batch size=128。

注意，原生 Caffe 使用的就地优化（In-Place Optimization）操作在 Layrub 中也有对应做法，但是它与层内策略有本质的不同，因为就地优化只适用于元素级操作，并不适用于复杂的矩阵操作，如卷积操作等。除此之外，层内策略还优于 MXNet 所使用的内存共享优化（Memory Sharing Optimization），因为内存共享通常发生在任意两个数据之间，而不考虑它们的内存占用是否相等。而层内策略是在两个数据大小相同的情况下进行的，可以实现更精确的内存节省，更多的对比实验见下一小节。

混合内存重用策略的优势：通过分析可以得出结论：层内策略和层间策略各有优

势，但同时也都存在一定的缺陷。因此本节提出了混合内存重用策略，使两种策略相辅相成。图 2-10 中的实验结果表明，这种混合策略适用于几乎所有类型的网络，训练多种模型时的内存使用量均能从 55% 减少到 98.9%。对于 GoogLeNet 来说，混合策略的内存消耗比例略高于层间策略，这是由于 GoogLeNet 有着特殊的正则化 LRN 层，消耗了最多的内存资源，简单地部署层内策略无法达到最优的结果，在这种情况下，混合策略退化成层间策略。结果表明，混合复用策略在各个形态的模型上性能都优于单一层内或者单一跨层复用策略。

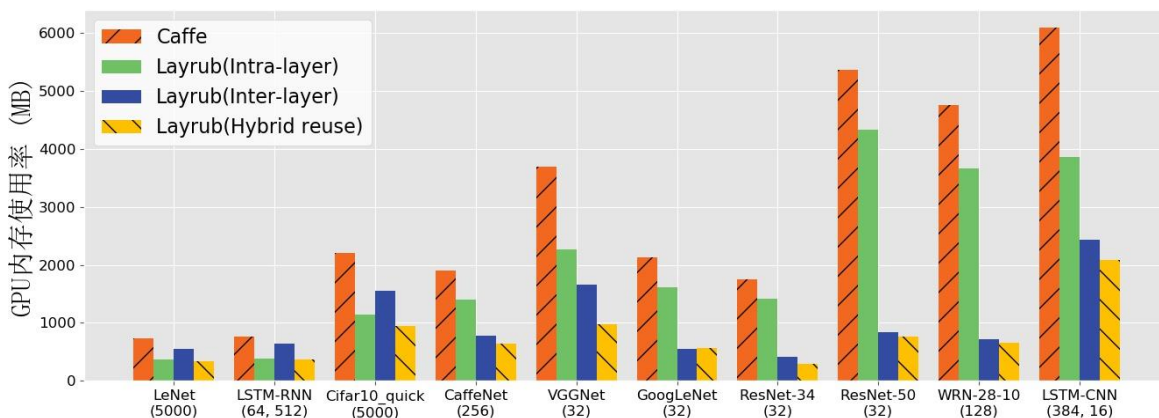


图 2-10 多个模型内存分配对比

注：训练测试时 LeNet、VGGNet、Cifar10_quick 以及 WRN 使用 Cifar10 数据集，CaffeNet、AlexNet、GoogLeNet 及 ResNet 使用 ImageNet 数据集，LSTM-CNN 使用 UCF101 数据集，模型名称下面括号内的数字表示训练时使用的批量处理尺寸。

小结：为了体现 Layrub 的内存效率，在 K40M GPU 上使用 Layrub 的三种策略分别进行训练，并将所有结果与 Caffe 系统进行比较。层内重用和层间重用两种内存优化策略有效地缓解了训练深度和宽度两大类卷积神经网络模型时的内存压力。混合策略将这两种策略结合起来，在大多数网络模型中实现了几乎恒定的内存消耗比例。

2.4.3 与多个代表性训练系统性能对比

本节将 Layrub 与其他代表性的训练系统进行了性能对比，这些系统都具备一些显式的内存优化策略，其中有 GeePS、基于 MXNet 的 memonger（以下简称 MXNet）以及 TensorFlow。为了获得最大的内存节约比例，除非另有说明，将在随后的实验中默认使用 Layrub 的混合重用策略。

内存效率对比：由于这些系统的内存优化策略各有不同，这里评估所有模型在四个系统上的运行时内存消耗。为了实现数据迁移，GeePS 使用内存池完成双重缓冲，所需的存储空间大小是特征数据实际所需内存空间的两倍。因此与 Layrub 相比，GeePS 特征映射数据所消耗的内存几乎是后者的两倍。从图 2-11 可以看出，在训练 ResNet-34 时，Layrub 消耗了 1482MB 的 GPU 运行时内存，而 GeePS 使用了 2705MB。由此可知，由于实验所用 GPU 的内存容量限制，图 2-12 的 GoogLeNet（128）和 VGGNet-16（64）是无法在 GeePS 系统中进行训练的。同时，可知有些训练系统无法对实验涉及到的所有模型都执行训练过程（例如 GeePS 及 MXNet 缺少对 WRN-28-10 模型的支持），因此仅展示其中 7 个模型的结果来证明 Layrub 的内存效率。Layrub 策略对内存的消耗比例比 GeePS 下降了 40.6%，同时，比 TensorFlow 下降了 50.1%。与 MXNet 相比，Layrub 在 GoogLeNet 上可以节省 35.2% 的运行内存，表现良好。在众多相关的内存优化系统中，只有 MXNet 和 Layrub 可以在一个内存 12GB 的 GPU 上成功训练 ResNet-1001，而包含 TensorFlow 和 GeePS 在内的系统则由于内存优化效率较低，因而在训练时超出内存容量限制，最终导致任务失败。2.4.4 节将会对极限训练任务进行更详细的评估。

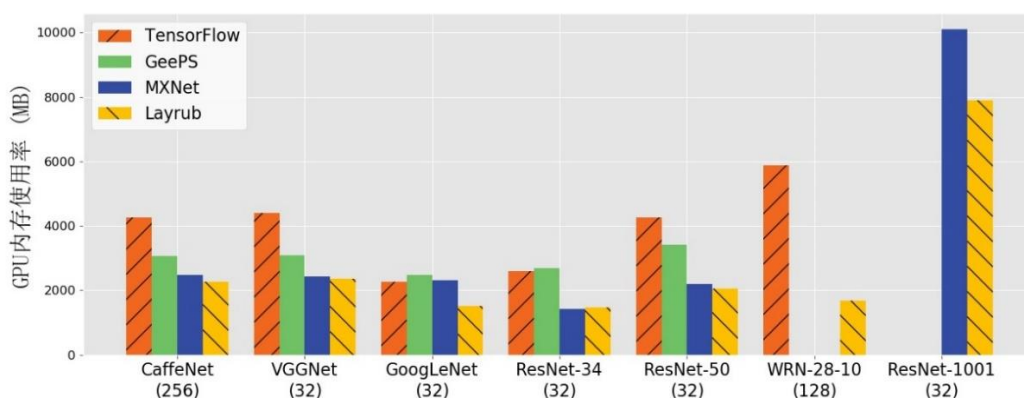


图 2-11 训练系统运行时内存消耗对比

注：训练测试时 VGGNet 及 WRN 使用 Cifar10 数据集，CaffeNet、AlexNet、GoogLeNet 及 ResNet 使用 ImageNet 数据集，模型名称下面括号内的数字表示训练时使用的批量处理尺寸。

时间性能对比：为了证明 Layrub 的时间性能，将其与最近的相关工作 GeePS 和 vDNN 进行了比较，后者与 Layrub 相似，也利用了 CPU-GPU 的数据传输策略来提升内存效率。上述的 GeePS 和 vDNN 均使用到了 GPU 内存池技术，GeePS 使用双重缓

冲内存池来实现高效的数据迁移，vDNN 在内存池中异步的分配和释放数据内存。但是考虑到异步内存操作将会带来明显的时间开销，Layrub 并没有采用这种技术。相比之下，所提出的层内复用策略对时间开销的影响几乎为零。而所提出的层间复用策略可以通过跨相邻层重用空间来获得显著的内存节省效果。然而，伴随着显著提升的内存效率，当数据迁移时延与计算时延无法完全重叠时，将会产生额外数据等待，训练时间开销方面出现了轻微的增加。

为了直接比较 Layrub、GeePS 和 vDNN（并未开源），采用 performance-time 公式 $P_{opt} = \frac{T_{base} \times P_{base}}{T_{opt}}$ 来计算规范化后的时间开销，这与 vDNN 中使用的评估方法相同。由于 Layrub、GeePS 和 vDNN 都是基于原生 Caffe 进行研发的，所以将原生 Caffe 作为规范化性能对比的基准。基线（Caffe）的规范化性能等于 1，越接近基线，则该系统的性能越好。以图 2-12 中的 VGGNet-16（64）为例，vDNN 的内存优化策略（图 2-12 中的 vDNN（m））的规范化性能得分低于 50%，而 Layrub 的是 83%。也就是说，尽管 Layrub 和 vDNN（m）的内存效率基本相同，但 vDNN（m）的性能开销要显著得多。与 vDNN 的性能最优策略（图 2-12 中的 vDNN（p））相比，尽管该策略以牺牲内存效率的代价换取时间性能，但是在时间效率上 Layrub 仍然比 vDNN 有优势。

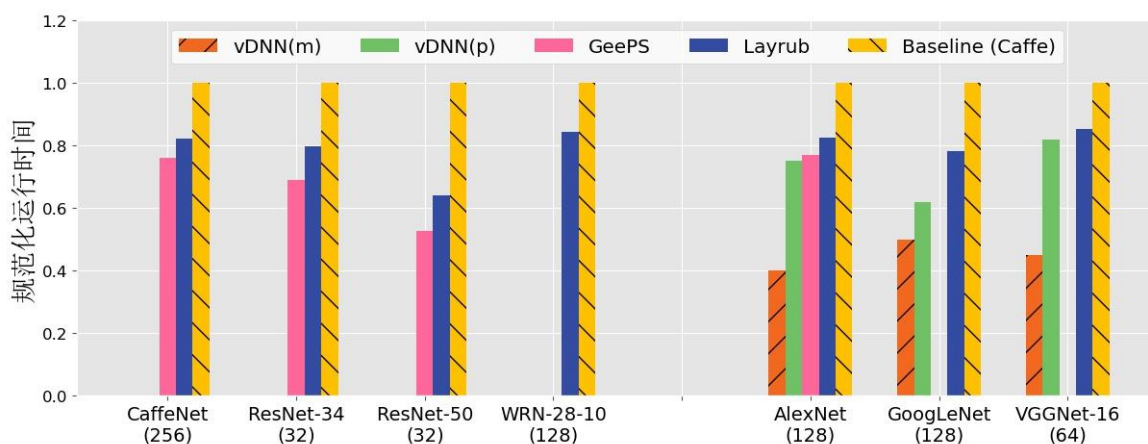


图 2-12 系统规范化性能对比

注：规范化到基线（Caffe），越接近 1 越好。训练测试时 VGGNet 及 WRN 使用 Cifar10 数据集，CaffeNet、AlexNet、GoogLeNet 及 ResNet 使用 ImageNet 数据集，模型名称下面括号内的数字表示训练时使用的批量处理尺寸。

众所周知，MXNet 和 TensorFlow 是目前最流行的、工业界应用最广泛的深度学

习系统，因此在表 2-4 中将 Layrub 与这两者进行了比较。由于不同框架的实现机制不同，为公平对比，这里记录一个 epoch（epoch 是整个训练集的单次遍历）的训练执行过程，并统计 20 次迭代的平均执行时间。以 Caffe、MXNet 和 TensorFlow 的原始版本为基准，这些版本没有做任何显式的内存优化策略。结果表明，Layrub 的额外时间开销的比例占 Caffe 原始训练时间的 9.05%，训练 8 个代表性模型时平均额外增加时间的比例为 24.1%。而 MXNet 在优化前后平均所增加的额外开销比例为 26.3%。与之相比，虽然在某些模型上，TensorFlow 的时间效率要优于其他模型，但由于缺乏有效的内存优化策略，其内存效率最差，像 ResNet-1001 这样极端规模的模型是无法在 TensorFlow 上进行训练的。

表 2-4 不同训练系统的训练时间对比

模型	MXNet			Layrub			TensorFlow
	T _{base} (M)	T _{opt} (M)	Ratio	T _{base} (L)	T _{opt} (L)	Ratio	T _{base} (T)
LeNet	2.373	2.783	+17.28%	2.148	2.342	+9.05%	5.143
Cifar10_quick	5.478	7.565	+31.61%	7.223	8.178	+13.23%	-
AlexNet	27.343	34.393	+25.78%	20.808	25.355	+21.85%	19.161
VGGNet	44.209	57.224	+29.44%	91.815	110.759	+20.63%	64.060
GoogLeNet	8.926	11.233	+25.85%	9.726	12.392	+27.41%	-
ResNet-34	13.897	17.849	+28.44%	37.387	46.928	+25.52%	10.101
ResNet-50	17.947	22.492	+25.32%	55.131	86.425	+56.76%	15.951
WRN-28-10	-	-	-	79.397	94.021	+18.42%	42.394
ResNet-1001	-	324	-	-	712.200	-	-
ResNet-1517	-	-	-	-	1148.860	-	-

注：单位：秒，在 K40M 上执行 20 个训练迭代。训练集与批处理尺寸设置同上图 2-10。

小结：为了实现时间效率，Layrub 利用 CUDA 流技术完成数据迁移，并隐藏延迟，尽可能的减少时间开销，提升内存效率。Layrub 可以以额外训练时间平均增加 24.1%为代价，在大多数网络模型训练时获得最佳的内存效率。总的来说，与以往的 vDNN、GeePS、MXNet 等相比，Layrub 在时间效率和内存效率上都有一定的优势。

2.4.4 极限规模训练性能评估

本小节以 ResNet 系列模型为例，探索了极限规模下深度模型的训练性能。关于

ResNet 模型深度的选择，遵循 ResNet 作者所制定的规则^[16,17]进行设置，如 1001 层或 1517 层，这里的每个 ResNet 模型至少需要几十 GB 的内存占用。由于除 MXNet 以外的其余训练系统内存消耗过大，无法在任意一个系统上使用单一 GPU 训练超过 1001 层的深层模型，因此仅选取 MXNet 系统作为对比对象。此外，这里选取了两种具有不同内存容量的 GPU，具体如表 2-5 所示：

表 2-5 极端规模模型内存消耗对比

模型	Titan Z			K40M		
	Caffe	MXNet	Layrub	Caffe	MXNet	Layrub
ResNet-34	4426MB	1554MB	1251MB	4430MB	1431MB	1257MB
ResNet-50	OOM	2298MB	1802MB	11425MB	2195MB	1805MB
ResNet-302	OOM	4873MB	3444MB	OOM	4911MB	3448MB
ResNet-695	OOM	OOM	5964MB	OOM	7885MB	5968MB
ResNet-1001	OOM	OOM	OOM	OOM	10104MB	7901MB
ResNet-1172	OOM	OOM	OOM	OOM	11455MB	9022MB
ResNet-1304	OOM	OOM	OOM	OOM	OOM	9872MB
ResNet-1517	OOM	OOM	OOM	OOM	OOM	11239MB

注：使用 ImageNet 数据集，批处理尺寸 batch size=32，OOM 表示内存不足（Out of Memory）。

表 2-5 记录了极端规模的神经网络模型在训练运行时的内存消耗情况，展示了 Layrub 在不同硬件环境下训练极端规模模型的能力。当模型的深度增加到 1517 时，在 K40M 上 Layrub 的训练过程仍然能够正常进行。观察实验结果可知，在 K40M 上，ResNet-1517 对应的运行时内存消耗为 11235MB。相比之下，在 ResNet 的网络深度不超过 1172 时，MXNet 系统仅能支持深度不超过 1172 的 ResNet。在 MXNet 系统中模型 ResNet-1172 执行训练的运行时内存消耗为 11455MB，这已经非常接近 K40M 内存的上限（即 11580MB），当网络继续加深，MXNet 的训练任务已经无法再进行下去。由此可知，Layrub 成为鲁棒性最强的系统，能够在多个 GPU 架构上对训练过程运行时内存开销进行优化，在 Titan Z 上 ResNet 的最大深度被扩展到 695 层，对应在 K40M 上能够扩展到 1517 层。

小结：为了打破 GPU 在训练极限模型时的内存限制，让设计者能够从更深或更宽的设计角度去探索网络架构，选择使用 Layrub 的混合空间复用策略，能够在训练过程达到最大的内存效率。

2.4.5 系统可扩展性评价

本小节评估了 Layrub 的系统可扩展性，在多节点的 Layrub 上评估其内存效率和加速性能。由于 Caffe-MPI¹是针对多节点 GPU 集群提出的 Caffe 的分布式版本，所以选择在 Caffe-MPI 上实现了 2.2 节的空间复用策略，来为大规模训练提供分布式支持。

表 2-6 给出了使用一个节点或两个节点（每个节点使用一个 K40M GPU）时，Layrub 的训练执行时间对比。此外，表格中还列举了分布式环境下 Layrub 在三种代表性神经网络模型上执行训练过程的加速比。如表 2-6 所示，与使用单个节点相比，使用两个节点的 Layrub 在 AlexNet 上的训练速度提高了 1.98 倍。对于这三种模型，Layrub 都能达到稳定的、接近线性的加速效果。虽然由于实验环境的限制并未使用更多的节点进行评估，但是仍然可以预见到，随着空间复用策略对内存效率的进一步优化，使用更多的节点时将会带来更高的加速比。

表 2-6 分布式训练效率对比

模型	分布式支持		
	Layrub (单节点)	Layrub (双节点)	加速比
AlexNet(256)	1h 47m	54m	1.98×
ResNet-50(32)	53h 16m	26h 54m	1.98×
ResNet-1001(32)	395h 25m	201h 3m	1.97×

注：所有模型都使用 ImageNet 数据集，在 K40M 上执行 1 个 Epoch。

小结：空间复用的内存优化策略能够在多节点的分布式环境下实现，除了解决极限规模下模型训练的内存消耗问题，同时也能够提升训练系统的可扩展性。实验结果证明，在扩展到多个节点时，本章所提出的空间复用策略对系统加速效果没有影响。

2.5 本章小结

深度卷积神经网络的特征映射数据需要大量的存储空间，然而，当前高性能加速硬件的存储资源不足以满足卷积神经网络训练时的内存需求。本章针对训练时内存受限问题进行研究，该问题目前存在两大主要挑战：第一是当前的内存优化策略无法兼

¹ <https://github.com/Caffe-MPI/Caffe-MPI.github.io>

顾多种类型的网络模型，内存优化效果不足以支撑大规模网络的训练；第二是由于这些策略往往集中于解决内存消耗问题，忽视了训练过程中时间性能的下降。为应对上述挑战，提出基于空间复用的特征映射数据放置机制，兼顾卷积神经网络的深度和宽度特性，可以支持超大规模网络的训练，且能够在一定程度上对训练性能有所保证。实验表明，Layrub 可以有效的重用 GPU 内存空间，甚至可以在单 GPU 卡上成功训练 1517 层的 ResNet 模型。通过与 GeePS、vDNN、MXNet 和 TensorFlow 进行比较，进一步展示了 Layrub 在各种模型和数据集上的优势。此外，Layrub 可以与 TensorFlow、MXNet 和其他框架很好的兼容，用于节约对应系统的内存使用。

然而，本章所讨论的内存优化技术同样具有一定的局限性。基于跨层复用思想的 CPU-GPU 数据迁移机制虽然能够显著降低神经网络训练时的内存使用量，但是不可避免地造成了训练性能下降，而另外一种基于时间换空间思想的 Extra-Computing 方法也面临着同样的性能下降问题，然而值得注意的是，两种优化策略引发训练性能损失的原因却不尽相同。分析可知，Extra-Computing 方法由于需要一次额外的前向计算，因而造成了训练性能的下降，而 CPU-GPU 迁移方法的性能下降则是由 GPU 和 CPU 内存之间的数据迁移引起的。经过观察得知，两类方法在不同类型的层上有着不同的性能表现，由此可以得到新的设计原则：属于计算密集型的层（如卷积层、全连接层）更推荐使用 CPU-GPU 迁移策略，而内存密集型的层（如激活层）更推荐使用 Extra-Computing 方法，自此本工作扩展得到一种新的层自适应策略选择机制^[83]，为卷积神经网络中不同的层选择合适的内存优化方式，尽可能的降低训练时性能开销。

3 深度学习系统运行时稀疏通信优化机制

第 2 章中介绍的基于空间复用的数据放置策略可以在单机环境下实现训练运行时的高内存利用率。然而在现实世界中,随着各类人工智能应用的迅速涌现,很多场景下单机加速硬件已经无法应对大规模的训练及预测任务^[17,36]。因此对深度学习系统开展分布式并行化研究变得十分有必要。由于分布式深度学习能够显著加快深度神经网络训练和预测的速度,因此在各类深度学习任务中均有广泛应用。然而,在基于数据并行的分布式训练中,节点间的梯度通信常常成为性能瓶颈,在低带宽环境下,梯度通信延迟进一步阻碍了分布式训练的效率。在近期提出的几种参数梯度约减算法中,梯度稀疏化是代表性方法之一,能够显著减小梯度通信规模。然而,对梯度应用稀疏策略后,梯度的陈旧老化不可避免,最终导致模型训练过程无法收敛。此外,近期关于梯度稀疏的工作仅仅考虑对整个深度网络进行稀疏化,从整个网络中筛选出一小部分梯度,而忽略了筛选策略本身对训练效率的影响。此外,如何高效地对稀疏后的梯度集合进行编码和解码这一问题还没有得到解决。因此,为了提高训练效率,本章提出了一种基于陈旧补偿的稀疏梯度优化策略 GradSA。首先结合网络结构的特点,提出分层的梯度稀疏化思想,能够显著减少通信开销。其次,利用近似梯度进行历史梯度的积累,以此来加快收敛速度。随后,提出针对稀疏梯度特征的高效编码机制。大量的实验表明,相对于现有工作,本章提出的整体通信优化机制具有明显优势,GradSA 方法能够在保证性能不下降的前提下,将参数梯度通信量最多减少 514 倍,能够有效提升系统吞吐量,加速训练迭代过程。

本章内容组织如下:第 3.1 节讨论并分析了分布式训练运行时通信优化方面存在的问题,并阐述了三阶段梯度通信优化的研究思路;第 3.2 节详细介绍了 GradSA 的梯度分层稀疏筛选机制;第 3.3 节详细介绍了基于误差反馈的近似梯度累加机制;第 3.4 节详细介绍了稀疏编码压缩机制;第 3.5 节介绍了实验环境和评估设置,通过实验对 GradSA 的时间效率、训练性能、系统可扩展性等方面进行验证;第 3.6 节对本章研究内容进行总结。

3.1 研究动机

本节首先分析互联带宽资源受限的环境下分布式深度学习系统的通信瓶颈问题，并指出对训练运行时参数梯度数据进行稀疏优化的优势所在，随后总结了当前关于梯度稀疏化相关研究工作的不足之处，通过实验观察分析梯度通信的三个必要阶段，得出对应的研究动机，最后提出解决该问题的基本思路和设计原则，并且提出一个三阶段的稀疏通信优化机制，在保证分布式训练质量的同时，有效减少通信集规模，提升系统吞吐率。

3.1.1 问题提出

随着对深度神经网络研究的深入，分布式深度学习系统已成为支持神经网络发展的强有力工具，备受学术界和工业界的关注。以自然语言处理类典型应用为例，在维基百科（Wikipedia）语料库上通过对 BERT 等语言模型进行预训练获取词向量的过程往往需要数天的时间^[36,37]，因此有必要使用分布式计算架构加速大规模模型的迭代执行。为了满足持续增长的训练需求，研究人员提出数据并行的思想，要求每个工作节点计算其局部梯度，并通过与训练系统中其他节点交换梯度来统一的更新模型参数。随着数据规模进一步扩大，更多的计算节点参与到分布式训练中来，因此产生了大量的梯度计算和梯度通信。然而，与此相比，节点间互连网络带宽的发展速度远远比不上计算硬件计算能力的发展速度，如表 3-1 和表 3-2 所示，最新的 NVIDIA V100 GPU^[85]具有 7TFlops 峰值性能和 900GB/s 的 HBM2 内存带宽，然而目前最新的 InfiniBand 网络只有 200GB/s 的互连带宽。由此可见，通信开销日益成为分布式训练系统的瓶颈。特别是在低带宽的环境下^[86]，例如，在包含大量的移动设备或边缘设备的蜂窝网络环境下训练深度神经网络，通信延迟将进一步增加，训练效率也会严重受限。因此，如何能够减轻训练系统对网络通信带宽的依赖，成为系统优化领域的一大挑战。

目前尝试通过约减参数梯度的规模来提升通信效率的研究工作主要分为两类：（1）梯度量化：这类方法侧重于根据梯度数据分布规律，将梯度量化为定点数，如使用二进制^[62,69]（binary）或三元^[87]（ternary）表示。（2）梯度稀疏化：这类方法^[88,89]在通信

过程中对梯度进行稀疏筛选，在每次迭代中，只筛选出一小部分梯度在节点间进行通信交换。然而，以上这些方法的基本思想都是减小梯度数据交换的规模，不可避免的会带来额外损失，影响模型训练的精度。

表 3-1 最新的 NVIDIA GPU 的特征介绍

GPU 型号	M2090	K40	P100	V100
设计架构	GF110	GK110	GP100	GV100
计算能力	0.665	1.68	4.67	7.014
内存带宽	192	288	720	900

注：表格列出在过去十年中 NVIDIA Tesla 系列 GPU 产品¹，该系列计算卡旨在加速深度学习和高性能应用。计算能力的计量单位是 TFLOPS。内存带宽的计量单位是 GB/s。

表 3-2 互连网络的特征介绍

通信协议	Ethernet	Fibre Channel	Omni-Path	InfiniBand
网络带宽	1-100	2-16	100-200	100
网络延迟	200 μ s	50 μ s	130ns	110ns
可靠性	较低	中等	较高	较高

注：表格列出互联通信网络的大致发展历程。网络带宽的计量单位是 GB/s。

为了解决上述问题，提高模型性能，基于参数梯度约减的分布式训练一般采用历史梯度累加的方式，对梯度损失进行反馈和恢复^[90]，将稀疏误差带入下一次迭代，累积误差以避免误差漂移。这个思路源于 Sigma-Delta 调节机制^[69]，在许多情况下被证明是卓有成效的。以基于参数服务器（Parameter Server, PS）通信模式^[84]的数据并行训练为例（如图 3-1 所示），每个工作节点（worker）维护一个本地参数和一个本地训练数据子集合，服务节点（server）从多个工作节点处收集所有局部参数梯度并更新全局参数。在本地的工作节点完成前向和后向计算之后，会将全体梯度上传（push 操作，图 3-1 中的右侧蓝色箭头）至服务节点，并及时从服务节点拉取（pull 操作，图 3-1 中的右侧橙色箭头）更新后的参数。远端的中心服务节点负责收取各个工作节点的梯度并汇总更新至全局参数，供工作节点进行拉取及本地覆盖。对于梯度稀疏化机制来说，工作节点并不会直接传送本地梯度矩阵的所有元素到参数服务器中，而是选择那些取值超过预定义阈值（在图 3-1 中用“ th ”来表示）的梯度元素，其余的历史梯度将积

¹ <https://developer.nvidia.com/cuda-gpus>

累反馈到下一次迭代计算所得的本地梯度矩阵。

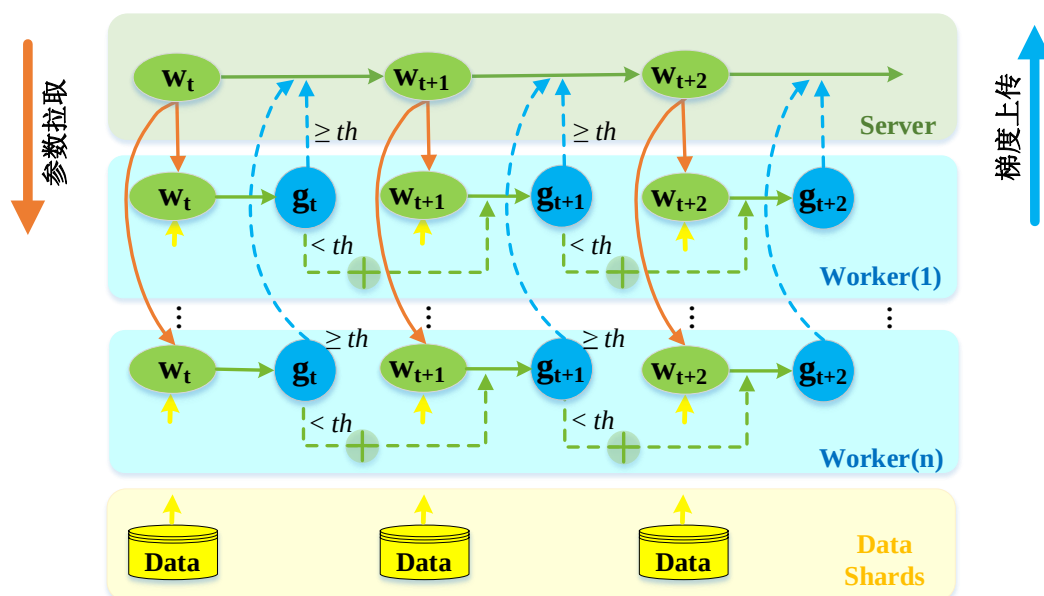


图 3-1 基于历史梯度积累的稀疏梯度训练

然而，当梯度稀疏比例设置为极低值时（如 0.1%），累积历史梯度的方法仍然无法满足模型的精度要求（例如，ResNet 的测试精度下降约 1-5%），因为它会导致另一个严重的问题，称为梯度老化（Gradient Staleness）。具体来说，较低的梯度稀疏比例意味着更多的网络带宽节约比例，这极大地减轻了训练中的通信瓶颈。但是，0.1%的稀疏率意味着大多数的参数每隔 1000 次迭代才能被上传至服务器端，而与此同时，未被上传的历史梯度将不断在本地进行累加。相对于全局参数来说，这些梯度早已过时或陈旧，但是他们仍然被累加并被传递到后续迭代更新中。这一问题带来的影响抵消掉了梯度累加策略对随机误差的反馈和恢复能力。近期工作中已有一些考虑到对梯度累加策略进行优化改良，例如在 DGC^[64]中通过使用修正动量（Momentum Correction）来实践这一想法。另外一些工作^[63,91]尝试通过改变模型架构、增加归一化层，来尝试对梯度值进行调节，还有其他工作^[64]尝试进行局部梯度裁剪（Gradient Clipping）和热身训练（Warm-up Training）。

然而，由于本节的目标是能够将训练过程部署在通信带宽非常低的边缘或移动网络上，因此希望将梯度的稀疏率设置得尽可能地低，以减少通信体量。而上述提到的保证稀疏 SGD 收敛的相关方法都无法直接满足低带宽环境下的高效通信需求，参数

更新过程仍然低效。除此之外，当前工作仅仅考虑对整个神经网络的梯度集合进行稀疏化，只从整个网络模型中筛选出一小部分梯度，而忽略了网络结构本身对参数更新的贡献程度，进而对训练效率产生负面影响。另外，当稀疏率定义为 0.1% 之后，传统的稀疏矩阵压缩方法不再适用，如何有效地编码和解码一个极度稀疏的梯度集合也仍然是一个有待解决的问题。

3.1.2 解决思路

上一小节讨论了当前关于参数梯度约减及历史梯度恢复方面的研究工作，然而，由于低带宽环境这一应用需求，当前策略不能很好地对梯度误差进行反馈和恢复，很难保证训练的高效收敛。不仅如此，由于相关工作往往以梯度约减为主要优化目标，而忽视了针对通信集合的高效压缩编码，这也是当前研究工作的不足之处。因此从梯度通信的阶段特性出发，分析研究梯度通信的三个必要阶段，并阐述本章的解决思路和研究动机。

解决思路一：梯度的分层稀疏筛选机制。梯度稀疏方法通过设置适当的阈值，对梯度进行筛选，上传那些对参数更新有着“重要贡献”的梯度，将梯度更新的规模降低到很小的比例。然而，最优阈值的设计并不容易。这是由于在网络结构的各个层上，梯度的分布特征是在不断变化的，因此很难从整体上有效表达。对于网络中的所有层使用统一的阈值筛选规则是不合适的。这种不恰当的筛选规则会引发连锁反应，最终影响训练效果。为了证实这一观点，本节针对一个在机器视觉领域具有代表性的卷积神经网络模型——ResNet，进行了一系列实验研究，并得出以下观察结论。

首先，神经网络模型中各个层的梯度数据分布规律不同。如果仅从整个网络的角度出发，制定统一的稀疏度、选择统一的筛选阈值，从整个网络的全部梯度中筛选出超过阈值的取值，这将会忽略掉很多变化幅度较低、但对参数更新贡献较大的梯度。其次，在相同的稀疏度下，针对同一模型选择不同的筛选粒度也会导致模型更新质量的差异，最终影响到训练的性能。例如，图 3-2 中的两个筛选粒度分别为网络粒度（Network-Level）和层粒度（Layer-Level），选择全尺寸梯度下降（Full-Size SGD）的方式作为基线。在这里将 Layer-Level 做如下定义：为每一层设置相同的稀疏比例，

并从每一层筛选出相同比例的梯度；同时对 Network-Level 做定义：为整个网络设置相同的稀疏比例，从整个网络的参数集合中筛选出一定比例的梯度。观察可知，当稀疏度设为 1% 时，两种筛选粒度的模型性能接近，而在稀疏度设置较低时，如 0.1%，Layer-Level 的方法训练出的模型性能明显优于 Network-Level 方法。

本小节提出分层筛选机制的动机就是来源于此。同时，大量实验发现，这些观察结果并非偶然现象，在研究其他主流 DNN 模型（包括小网络 LeNet，大网络 AlexNet 和 Inception）时，也发现了类似的实验结论，由于篇幅有限，并未列出所有遵循相同规律的结果。

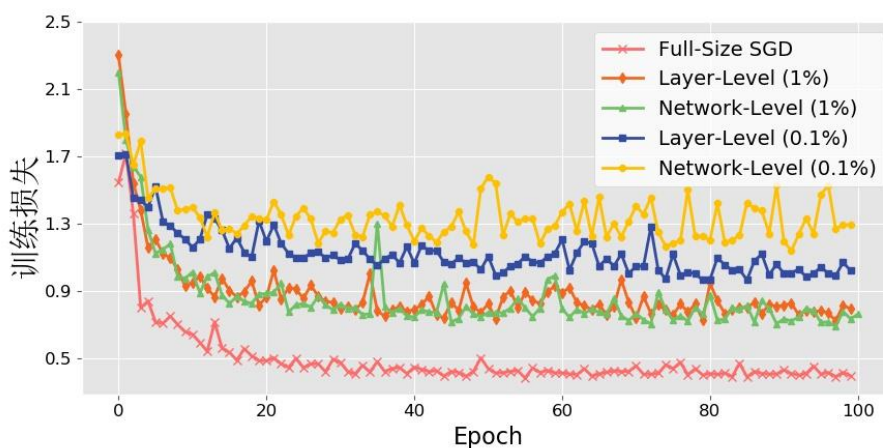


图 3-2 不同的筛选粒度对模型训练性能的影响

注：记录了在 ResNet-20（cifar10 数据集，batch size=64）上不同梯度稀疏率（1% 和 0.1%）时的训练损失曲线。

解决思路二：基于误差反馈的近似梯度累加机制。虽然梯度稀疏机制可以减少通信等待开销，提升训练速度，但是由梯度稀疏引起的梯度误差却变得十分显著。这主要是由于值较小的那些历史梯度被延迟更新，并被不断累加到后续迭代中，因此当这些梯度更新被采纳时，它们对于远程服务器端中收集到的最新版本的全局模型来说，已经过时或过于陈旧了。根据实验统计，当稀疏度设为 0.1% 时，大多数的参数梯度每 500 到 1000 次迭代更新一次，与每个 epoch 的迭代总次数相比来说，这个间隔时间也是非常长的。

总的来说，梯度稀疏导致了累加历史梯度的滞后性，在训练过程中引入了额外的随机噪声，即稀疏误差，这会减慢收敛速度，甚至导致模型无法收敛。这一结论与图

3-2 中的观察结果一致。例如，在 0.1% 的梯度稀疏度时，训练 ResNet-20 的验证损失曲线仅收敛到 0.9 左右，该次训练没有执行额外的梯度误差补偿。与此相比，未对梯度进行稀疏处理的基线可以很容易的在 100 个 epoch 后收敛到 0.4。因此，有必要对历史梯度进行深入研究，并对其进行适当的补偿和反馈，这将在第 3.4 节中进行具体的讨论。

解决思路三：高效的梯度编码压缩机制。经过梯度稀疏筛选和历史累加之后，梯度通信集合中的非零值变得非常少且质量很高。此时优化目标为尽可能减少通信编码时间来提升训练效率。在传统的方法中，稀疏后的梯度通信集合将以键值对的数据格式进行参数更新。然而，由于对每个键值对进行编码和解码的开销很高，在实际的工作流中使用该类方法是比较低效的。此外，通信集合的数据特征与传统通信集有显著的不同。经过 Top-k 筛选机制之后，通信集合是顺序的、单维度的、且高度稀疏。因此，希望能够寻找一个比稀疏表征方法^[92]（如 LIL, CSR, CSC 等）更为简化的稀疏矩阵表征方法，来专门解决稀疏梯度压缩问题。所提出的方法应该重点关注：（1）结合梯度数据的特性，（2）达到更好的压缩比、同时保证压缩与解压缩的速度。

然而，单一的策略并不能同时保证分布式训练质量并且有效减少通信集规模，因此有必要提出一套三阶段的梯度通信方案能够同时兼顾上述问题，如图 3-3 所示，包含三种对应的策略，能够解决上述的三个挑战，且能够相辅相成、兼顾彼此。具体来说，首先尝试探索梯度筛选的粒度问题，通过设置分层的（Layer-Level）阈值筛选策略，获得稀疏后的梯度，只保留当前层中值最大的梯度。接下来，为解决梯度陈旧问题，引入了陈旧梯度的泰勒近似，通过结合本地梯度与全局参数，对陈旧历史梯度进行了近似补偿。随后，为了解决通信带宽有限的问题，提出了一种简化的稀疏梯度表征方法，在压缩比例和压缩速度这两者中达到平衡。

总的来说，提出了一套三阶段的梯度通信机制 GradSA，在 GPU 集群上执行分布式训练任务时，对稀疏梯度通信模块进行管理，能够高效的进行梯度交换、并且保证模型训练性能。作为一套全栈式的系统支持，GradSA 可以在不同的深度学习训练系统上进行实现，包括 MXNet^[55]和 PyTorch^[93]等。GradSA 的几个重要优化步骤可以简要描述如图 3-3。

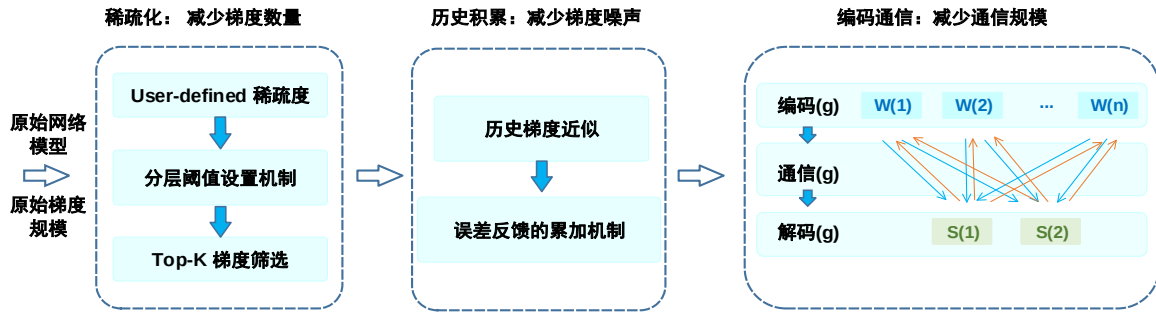


图 3-3 三阶段梯度通信机制

3.2 分层稀疏筛选机制

为了改善参数梯度稀疏上传而引起的更新质量问题，本节提出分层筛选机制，在 3.2.1 小节对稀疏通信过程相关的基本概念进行定义，在 3.2.2 小节详细描述了分层的梯度阈值筛选方案。

3.2.1 稀疏筛选基本定义

降低分布式训练通信量的一个关键点是对梯度进行合理稀疏化，通过 Top-k 方法设置适当的阈值^[64,88]，筛选出最显著的梯度子集并将其作为通信集合进行梯度交换。虽然未被传递的梯度元素值很小，然而，为了避免丢失信息，它们仍然应该被累积到一个称为“历史梯度累加”的数据池中，在下次迭代中作为待筛选元素，进行下一轮梯度稀疏。在累加得到的梯度值达到预先设定的阈值之前，该历史梯度的累加过程是持续的。对这一过程做出如下的定义：

定义 3.1: 用 $R: \mathbb{R} \rightarrow \mathbb{C}$ 表示基于阈值的稀疏函数，它将 d 维向量中的每个分量按降序排序，并选择 Top-k 个梯度分量（最大的前 k 个值）进行通信。稀疏率 θ 可以定义得非常低（如 0.1%），稀疏处理后的梯度向量通常只包含有限数量的非零梯度元素，所以稀疏向量可以有效地被编码压缩。在每个迭代中，每个 worker 在传递梯度之前对它的本地梯度进行稀疏处理：

$$\tilde{g}_t^{(k)} = R(g_t^{(k)}; \theta) \quad (3-1)$$

其中 $g_t^{(k)}$ 为第 k 个 worker 在第 t 次迭代时的本地梯度， $\tilde{g}_t^{(k)}$ 为其对应的稀疏梯度。

定义 3.2: $\hat{g}_t^{(k)}$ 被定义为下一个迭代的历史梯度, 由公式 (3-2) 完成掩码操作 Op , 该操作类似于相关工作[64]中的做法, 便于后续计算。

$$\hat{g}_t^{(k)} = Op(g_t^{(k)}, \tilde{g}_t^{(k)}) \quad (3-2)$$

定义 3.3: 当 server 端接收到其他 worker 发送的所有局部梯度时, 计算全局梯度并通过以下方式更新全局模型参数副本:

$$w_{t+1} = w_t - \eta \cdot \tilde{g}_t = w_t - \frac{\eta}{K} \sum_{k=1}^K \tilde{g}_t^{(k)} \quad (3-3)$$

3.2.2 分层稀疏筛选

本节提出使用一种分层阈值筛选方案, 与原始版本的梯度稀疏机制相比, 引入了分层 Top-k 筛选方案。在每个迭代中, 每个 worker 在上传梯度之前按照逐层顺序的稀疏处理局部梯度, 公式 (3-1) 可以细化为:

$$\tilde{g}_{(l,t)}^{(k)} = R(g_{(l,t)}^{(k)}; \theta) \quad (3-4)$$

其中 $g_{(l,t)}^{(k)}$ 是由第 t 次迭代的第 k 个节点计算上模型第 l 层的局部梯度, $\tilde{g}_{(l,t)}^{(k)}$ 是其稀疏对应。当 server 接收到其他 worker 发送的所有本地梯度时, 借助逐层更新法则 (大多数框架均采用), 去计算全局梯度并更新全局模型参数副本。这里稀疏率 θ 的取值范围为 $0 < \theta < 1$ 。具体来说, 在任意的 worker 中, 梯度的筛选阈值由 Top-k 和 θ 的取值共同确定, 并根据对应稀疏比例选取前 k 个最大的梯度。

这里值得注意的是, 在筛选粒度的选择上, 除了分层筛选之外, 仍然存在更细粒度的筛选维度, 即从每个过滤器 (filter) 或每个过滤器组 (filter group) 中筛选梯度。但是, 考虑到搜索空间过大, 搜索成本高昂, 而取得的性能收益却很有限, 这里暂不考虑引入这种激进的策略。此外, 不适当的稀疏粒度会导致不确定的稀疏率, 难以在低带宽环境下成功部署。根据实验发现, 以 ResNet-20 为例, 实现了过滤器级别筛选策略的模型的执行时间要比实现分层筛选策略的模型的执行时间长 9 倍, 并且前者只能达到近 10% 的稀疏率。

3.3 基于误差反馈的近似梯度累加机制

为了解决梯度稀疏误差问题，本节提出减缓历史梯度老化的方法：不再对原始的梯度进行累加，而是引入泰勒近似方法，在最新迭代的梯度上进行泰勒展开，以此来对这些历史梯度进行近似。换句话说，累加过程中陈旧的历史梯度可以被在时间上最接近它们的较新梯度所取代。

3.3.1 历史梯度近似

为了恢复陈旧老化的历史梯度，使其在目标函数优化中发挥更大的作用，这里引入了一个有趣的视角，即使用前瞻性（Look-ahead）的方式来实现历史梯度累积，不再直接使用之前迭代计算得到的历史梯度，而是将其替换为当前迭代计算得到的最新梯度。实现这一思路最直接的方法就是使用当前迭代的梯度，如 $g_{t+\tau}^{(k)}$ ，来近似之前迭代的历史梯度，如 $g_t^{(k)}$ 。本节的后续部分会展开讨论对历史梯度进行近似的过程。

利用泰勒展开得到梯度近似：可以用泰勒公式对梯度 $g_t^{(k)}$ 进行展开^[94]，

$$g_t^{(k)} = g_{t+\tau}^{(k)} - \nabla g_{t+\tau}^{(k)}(w_{t+\tau} - w_t) + O((w_{t+\tau} - w_t)^2) \quad (3-5)$$

这里的 ∇g 表示梯度矩阵的一阶导数，对应元素 $\nabla g_{ij} = \frac{\partial^2 f}{\partial w_i \partial w_j}$ 位于矩阵的第 i 行第 j 列， $O((w_{t+\tau} - w_t)^2)$ 表示泰勒公式的高阶余数。由于残差涉及到无限项的和而无法求得，因此忽略某些截断项，只考虑最简单的近似，即仅在泰勒展开式中考虑梯度的一阶项^[94]，如公式（3-6）所示：

$$(g_t^{(k)})' = g_{t+\tau}^{(k)} - \nabla g_{t+\tau}^{(k)}(w_{t+\tau} - w_t) \quad (3-6)$$

这里用 $(g_t^{(k)})'$ 来表示原始梯度 $g_t^{(k)}$ 的近似值， $\nabla g_{t+\tau}^{(k)}$ 对应于损失函数 $f(w_t; x_t)^{(k)}$ 的二阶导数，也就是 $f(w_t; x_t)^{(k)}$ 的海森矩阵（Hessian Matrix），可以被定义为 H_t 。

为更好的理解泰勒公式在这里的意义，借助一个反例来进行解释，假设并未在历史梯度累加中引入泰勒展开，那么当一个 worker 试图将自身累加的历史梯度集合

$\sum_{i=t}^{t+\tau} \hat{g}_i^{(k)}$ 更新到全局模型参数 $w_{t+\tau}$ 中, 此时可能已有其他一些 worker 将自身的本地梯度累加到全局模型中, 而全局模型参数可能已经得到更新。由此可以推测 $\sum_{i=t}^{t+\tau} \hat{g}_i^{(k)}$ 由以下部分组成: (1) 当前最新版本的梯度 $g_{t+\tau}^{(k)}$: 由最新版本的模型快照 $w_{t+\tau}$ 计算所得; (2) 积累的历史梯度: 由历次迭代 (从时间点 t 到 $(t+\tau-1)$ 时间点) 得到的模型快照计算所得 (τ 在这里被称为陈旧因子)。然而实际上, 将模型 w_t 对应的梯度叠加到另外一个模型 $w_{t+\tau}$ 中, 这本身很可能会导致数学错误, 而这也是导致梯度稀疏误差的根本原因, 会严重影响训练的收敛轨迹。因此, 提出对陈旧梯度进行一阶泰勒近似展开, 借助随后时间点的本地梯度和全局参数对陈旧梯度进行恢复。

利用对角海森矩阵得到梯度近似: 在得到泰勒近似展开公式之后, 如何能够对海森矩阵进行快速计算成为新的挑战。究其原因主要是由于针对参数规模较大的深度神经网络, 一阶梯度对应海森矩阵的精确计算开销非常大。因此希望能够通过进一步的对海森矩阵进行近似, 为此采用了方差有界的对角海森近似方法, 该方法借鉴了相关工作[95]中所使用的近似方法, 如引理 3.1 所示, 文献[95]指出对于特定损失函数 (如负对数似然函数), 二阶导数可以通过一阶导数的外积进行无偏估计。这种方法可以在理论上证明无限接近海森矩阵, 并且能够很容易地计算而不需要引入额外的复杂计算, 仅仅使用在前一个训练过程中已经获得的梯度和参数来得到海森矩阵的近似值。

引理 3.1: 用 P_t 表示海森矩阵 H_t 的无偏估计^[95], 作为第 t 次迭代时第 k 个 worker 的梯度的外积, 公式如下:

$$P_t^{(k)} = \left(\frac{\partial f}{\partial w_t^{(k)}} \right) \left(\frac{\partial f}{\partial w_t^{(k)}} \right)^T = g_t^{(k)} (g_t^{(k)})^T \quad (3-7)$$

证明: 考虑到神经网络模型的最顶层一般属于 Softmax 分布, 交叉熵损失函数使用负对数似然估计, 即 $P(Y=k|x, w_t) \triangleq \sigma_k(x; w_t)$ 。文献[96]依据计算 fisher 信息矩阵的两种等价方法, 可以得出梯度外积 $g_t^{(k)} (g_t^{(k)})^T$ 是海森矩阵 H_t 的渐近无偏估计, 即 $\varepsilon_t \triangleq E\|P_t - H_t\| \rightarrow 0, t \rightarrow \infty$ 。上述等价条件的前提假设是底层分布等同于模型参数最优

解 $\mathbf{w}^*$ 的分布规律，也就是说在神经网络的假设空间中不会存在近似误差，神经网络的模型参数 $\mathbf{w}_t$ 将会随着训练过程逐渐收敛到模型最优解 $\mathbf{w}^*$ 。这一假设符合深度神经网络的近似特性及局部最优化理论。

根据文献[95]可知，虽然无偏估计的均值相同，但是方差可能会存在较大的情况。考虑到较大方差会导致无偏估计 $\mathbf{P}_t$ 存在潜在的近似误差，为进一步提升海森近似的精度，通过最小化均方误差（Mean Square Error, MSE）的方法来控制方差，引入权重因子 λ ，得到 $MSE(\lambda \mathbf{P}_t) \leq MSE(\mathbf{P}_t)$ ，用以平衡近似估计的方差与偏置。

此外，为了降低计算成本，文献[95]采用二阶学习理论中常用的对角逼近方法^[97]来进一步近似海森矩阵。具体来说，如引理 3.2 所示，矩阵对角化方法只存储近似矩阵 $\lambda \mathbf{P}_t$ 的对角元素，并将所有其他元素设置为零， $\odot$ 表示 element-wise 的乘法运算。该对角化方法已被证明是一个计算复杂度比较低的海森矩阵近似方法，同时能够较好的保证近似精度^[97]。

引理 3.2: 用 $Diag(\lambda \mathbf{P}_t^{(k)})$ 表示海森近似矩阵^[95]，具体如下：

$$Diag(\lambda \mathbf{P}_t^{(k)}) = \lambda \mathbf{g}_t^{(k)} \odot \mathbf{g}_t^{(k)} \quad (3-8)$$

文献[97]给出海森矩阵对角化误差的上界设为 ε_D ，即 $\|Diag(\mathbf{H}_t) - \mathbf{H}_t\| \leq \varepsilon_D$ ，可求证 $Diag(\lambda \mathbf{P}_t)$ 的 MSE 存在上界。

证明：假设损失函数符合 L_1 -Lipschitz 条件，这里设 B_1 为 $\mathbf{P}_t$ 的方差，求证得到 $Diag(\lambda \mathbf{P}_t)$ 的 MSE 上界，

$$MSE(Diag(\lambda \mathbf{P}_t)) \leq 4\lambda^2 B_1 + 4(1-\lambda)^2 L_1^4 + 4\varepsilon_t^2 + 4\varepsilon_D \quad (3-9)$$

根据以下不等式可得：

$$\begin{aligned} MSE(Diag(\lambda \mathbf{P}_t)) &\leq E\|Diag(\lambda \mathbf{P}_t) - \mathbf{H}_t\|^2 \\ &\leq 4E\|Diag(\lambda \mathbf{P}_t) - E(Diag(\lambda \mathbf{P}_t))\|^2 + 4\|E(Diag(\lambda \mathbf{P}_t)) - E(Diag(\mathbf{P}_t))\|^2 \\ &\quad + 4\|E(Diag(\lambda \mathbf{P}_t)) - E(Diag(\mathbf{H}_t))\|^2 + 4\|E(Diag(\mathbf{H}_t)) - E(\mathbf{H}_t)\|^2 \\ &\leq 4\lambda^2 B_1 + 4(1-\lambda)^2 L_1^4 + 4\varepsilon_t^2 + 4\varepsilon_D \end{aligned}$$

最终得到历史梯度近似：因此，在公式（3-6）中的原始本地梯度 $g_t^{(k)}$ 能够被近似为如下：

$$(g_t^{(k)})' = g_{t+\tau}^{(k)} - \lambda g_{t+\tau}^{(k)} \odot g_{t+\tau}^{(k)} \odot (w_{t+\tau} - w_t) \quad (3-10)$$

值得注意的是，与原始的梯度交换机制相比，本节所提出的近似梯度方法是没有额外的通信成本的，对 server 端也没有额外的计算需求。公式（3-10）增加的额外计算只针对于本地 worker，对其引入了一个轻量级的计算开销。

3.3.2 历史梯度累加

在第一步尝试减轻历史梯度的陈旧性之后，随后对近似得到的历史梯度进行累积，并在后续推理得出最终的稀疏误差。这样在每一次迭代开始时，都会先用前若干次迭代累积得到的稀疏化误差反馈到当前迭代的本地梯度，对本地梯度进行修正，然后将其送入一个基于阈值筛选的梯度稀疏函数中，然后进入下一轮的三阶段稀疏通信过程，具体流程如 3.1.2 节所述。

与原始的梯度稀疏化方法相比，增加了历史梯度累加的步骤，对稀疏化阶段的对象做了替换，用近似后的累积梯度代替了原始梯度。具体来说，这里使用 $h_t^{(k)}$ 来表示第 k 个 worker 在第 t 次迭代时累积的历史梯度，它累积了所有以前迭代中未发送的历史梯度：

$$h_t^{(k)} = \sum_{i=1}^{t-1} (\hat{g}_i^{(k)})' \quad (3-11)$$

使用三个连续的时间序列来展开上述的积累公式，用以下的时间集合来表示，即 $\{(1, t-\tau-1), t-\tau, (t-\tau+1, t-1)\}$ ：

$$h_t^{(k)} = \sum_{i=1}^{t-\tau-1} (\hat{g}_i^{(k)})' + (\hat{g}_{t-\tau}^{(k)})' + \sum_{i=t-\tau+1}^{t-1} (\hat{g}_i^{(k)})' \quad (3-12)$$

注意，累积的历史梯度 $h_t^{(k)}$ 可以通过增量的方式进行更新 $\hat{g}_{t-1}^{(k)}$ ：

$$h_t^{(k)} = h_{t-1}^{(k)} + \hat{g}_{t-1}^{(k)} \quad (3-13)$$

将式（3-1）中基于阈值的稀疏化函数应用于累加梯度，计算得到稀疏本地梯度：

$$\tilde{g}_t^{(k)} = R(g_t^{(k)} + \alpha h_t^{(k)}) \quad (3-14)$$

其中 α 是历史梯度积累系数 ($\alpha > 0$)。

随后将稀疏误差定义为补偿后的累加梯度及其本地原始梯度之间的差距，在公式 (3-15) 用 $\varepsilon_t^{(k)}$ 表示：

$$\varepsilon_t^{(k)} = \tilde{g}_t^{(k)} - (g_t^{(k)} + \alpha h_t^{(k)}) \quad (3-15)$$

3.4 高效的稀疏编码压缩机制

在提出的三阶段通信机制中，在第一阶段的 Top-k 稀疏筛选、第二阶段的历史梯度恢复之后，待通信的梯度集合已经具备以下特性：顺序的、一维的和高度稀疏的。因此，希望选择一种兼顾以下三个要求的压缩算法，即（1）高压缩比，（2）高编码和解码速度，以及（3）对梯度数据特征的兼容性。

这里使用主流的梯度压缩方法进行详尽的实验性研究，如文献[64]中的 RLE，文献[67]中的 Huffman，以及谷歌的加速库 Snappy¹。文献[67]使用 Huffman 编码表示共享的梯度矩阵和基于距离差的索引。这两种数据类型都具有高频的数据分布，比较适合使用 Huffman 编码。但是，由于本章所描述的稀疏梯度的数据类型与此不同，存在着大量连续的零值和少量的非零值，且属于无偏分布，并不适合使用 Huffman 进行编码。从表 3-3 可以看出，Huffman 的压缩比例比其他的对比方法要差很多。文献[64]引入 RLE 来编码稀疏梯度，这里的数据类型与本文的很相似，实验可知 RLE 具有较好的压缩比，但是由于其压缩时间更长，因而并未被本文采用。而 Snappy 作为最著名的加速库具有理想压缩速度，但是其压缩比例较差，这一点在表 3-4 中也得到了验证。

综上所述，针对本实验中特定的数据类型、数据规模以及压缩比例，现有的压缩方法都不能对其充分实现高效通信传递这一目的。究其原因，主要是由于上述的这些方法并未结合稀疏梯度的数据特征，进而无法兼顾高压缩比与压缩速率的设计要求，因此无法实现高效通信的目标。

¹ <https://github.com/google/snappy>

表 3-3 压缩比例对比

稀疏度	模型	梯度规模(MB)	压缩比例 (×)			
			Huffman	RLE	Snappy	GradSA
0.1%	LeNet	3.47	4.81	254.14	9.09	462.04
	ResNet-50	98	4.80	255.46	8.47	494.77
	AlexNet	232.5	5.32	340.56	13.40	514.01
1%	LeNet	3.47	4.73	33.80	10.13	48.80
	ResNet-50	98	4.72	26.22	7.94	49.13
	AlexNet	232.5	5.23	34.59	12.46	51.85

表 3-4 编码解码时间对比

稀疏度	模型	梯度规模(MB)	编码解码时间 (秒)			
			Huffman	RLE	Snappy	GradSA
0.1%	LeNet	3.47	9.40	0.15	0.05	0.07
	ResNet-50	98	533.99	8.32	3.11	4.02
	AlexNet	232.5	404.55	17.60	6.29	8.05
1%	LeNet	3.47	9.37	0.14	0.05	0.07
	ResNet-50	98	180.92	8.29	3.16	4.11
	AlexNet	232.5	382.13	16.98	6.17	8.18

基于索引的梯度编码机制：为了解决上述问题，提出了一种基于索引的编码方法，该方法主要是从上述实验观察中得到启发。与传统的稀疏矩阵压缩方法相比，尝试寻找一种更简单的稀疏表示方法以满足所提出的三个标准。因此，提出了一种类似于 CSR/CSC 的基于索引的稀疏压缩方法。在每个迭代中，只从每个工作节点中上传稀疏梯度矩阵的非零元素，也就是说，工作节点完成上传操作时所使用的通信集合里仅仅包含了矩阵非零元素的实际值及对应的索引位置。

如图 3-4 所示，随梯度数组进行一次遍历可以收集到相应的非零值，索引(Index)记录了梯度数组中非零绝对值的位置。算法 3-1 描述了实现细节。当梯度稀疏度较低的情况下，如表 3-3 中所示的稀疏度为 0.1%，此时算法的空间复杂度与原始梯度的规模相比降低了两个数量级，由此得知，基于索引的编码方式是最有效的方法。因此，在本章使用这种方式为通信优化阶段的特定数据类型提供合适的压缩方法。

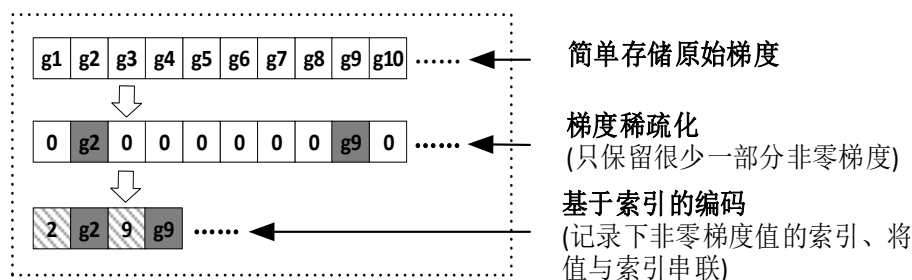


图 3-4 基于索引的梯度编码

算法 3-1: 高效编码-解码方法

```

1.  function ENCODE(grad list, encode list)
2.      //工作节点 worker 对梯度矩阵进行编码
3.      j=1 //对索引进行计数
4.      for i in grad list.size() do
5.          if grad list[i] != 0 then
6.              encode list[j++] = i //存储索引
7.              encode list[j++] = grad list[i] //存储梯度值
8.          end if
9.      end for
10. end function
11. function DECODE(grad list, encode list)
12.     //服务节点 server 初始化一个 full-size 的全零矩阵
13.     for k in worker.size() do //工作节点的数量
14.         Allgather(encode list all, encode list)
15.     end for
16.     for i in encode list.size() do
17.         grad list[encode list all[i]]=encode list all[i+1]
18.         //在 server 端对全尺寸矩阵进行恢复
19.         i+=2 //多线程执行
20.     end for
21. end function
    
```

3.5 有效性评价与分析

本节评估 GradSA 在深度学习系统中的性能表现, 选择了具有代表性的深度神经网络模型和训练数据集, 目前这些模型已经广泛应用于计算机视觉等研究领域。

3.5.1 实验设置

在这一节中通过大量的实验来评估所提出的 GradSA 的效率。选取了代表性的深度神经网络模型，如 AlexNet、Inception、ResNet，这些模型在图像分类等机器视觉任务中得到了广泛的应用，同时使用了两个著名的数据集 Cifar-10 和 ImageNet。实验是在一个由 16 个节点组成的 GPU 集群上进行的，这些节点通过 InfiniBand 和以太网相互连接。每个节点有两个 Tesla P100 GPU。在 MXNet 的基础上实现了 GradSA，MXNet 版本是从 Apache 中获取到的。在训练过程中使用卷积神经网络模型的原始文献提供的默认超参数（学习率、动量等），采用 Nesterov 动量 SGD 优化器^[98]。注意，关于其他的超参数（包括 λ ， α 和 τ ），利用网格搜索的方法进行简单的尝试，最终确定选择 $\alpha=0.9$ ， $\lambda=0.05$ ， $\tau=10$ ，以此获得较好的测试性能。

实验对比对象主要包括两个部分。首先，为了验证 GradSA 的训练效率，使用了原始的 SGD 训练方式作为基线，梯度为全精度类型且没有做任何稀疏化或通信优化。这些实验旨在测量在梯度优化前后模型精度的波动情况、以及训练收敛行为。其次，为了展示 GradSA 在各个方面的潜力，同时与其他相关工作进行比较，包括 8 比特位梯度量化策略^[99]（简称 8-bit）、TernGrad^[87]和 DGC^[64]，其开源版本均从 GitHub 获得。除此之外，为了验证 GradSA 在移动或边缘设备部署训练时的优势，简单统计并模拟了当前主流移动蜂窝通信网络（包括从 3G 到 5G 的不同版本^[100,101]），如表 3-5 所示。

表 3-5 现有蜂窝移动网络制式总结

网络标准	制式	下行带宽	上行带宽	网络延迟
3G	UMTS	128-364Kb/s	60-128Kb/s	800ms-2s
3.5G	HSPA	56Mb/s	22Mb/s	100-600ms
4G	LTE	100-300Mb/s	50Mb/s	100ms
4G+	LTE-A	1Gb/s	500Mb/s	10-50ms
5G	NR	5-10Gb/s	1-5Gb/s	< 1ms

注：大致列出了现无线蜂窝标准的规定^[100,101]，并对其统计数据进行了汇总。

3.5.2 训练性能评估

本节研究 GradSA、8-bit 量化、TernGrad 和 DGC 的整体训练性能，针对一些代表性的模型进行训练，并记录相应模型的精度、梯度压缩比和收敛行为。

模型精度对比：表 3-6 给出了各个模型在 4 个 worker 上训练 200 个 epoch 后的 top-1 测试精度。DGC 和 GradSA 的梯度稀疏度都是 0.1%（这意味着只有 0.1%的梯度是非零的）。与未作梯度稀疏优化的训练过程相比，GradSA 几乎能够在所有模型的训练中保证测试精度的稳定，这证实了本章提出的近似方法和误差补偿机制可以有效解决因梯度老化所引起的精度下降问题。相比之下，8-bit 量化和 TernGrad 由于缺乏有效的梯度补偿机制，因此只能获得较差的训练精度。值得注意的是，尽管 GradSA 和 DGC 都试图积累历史梯度来补偿梯度的陈旧老化，但是由于 GradSA 引入了前瞻性的历史梯度近似累加策略，基于泰勒展开的近似策略从本地 worker 中引入较新版本的梯度，并利用其他 worker 的更新值来增强本地梯度，因此要比 DGC 表现得更好。

表 3-6 梯度优化策略的训练精度对比

训练数据	模型	测试精度 (top-1; %)					
		基线	8-Bit	TernGrad	DGC (refer.)	DGC	GradSA
Cifar10	Inception-v2	89.33	86.91	87.90	-	89.04	90.40
	ResNet-50	92.23	90.16	90.58	-	92.10	93.12
	ResNet-56	93.03	90.91	91.23	-	91.96	93.84
	ResNet-110	93.75	89.75	92.69	93.87	93.54	94.09
ImageNet	ResNet-50	76.36	74.04	75.29	76.15	75.90	77.20
	AlexNet	57.15	55.96	56.28	58.20	56.19	58.05

注：列出了不同模型的测试 top-1 精度，ResNet 系列模型的批大小 batch size 为 128，Inception -v2 和 AlexNet 的总批大小为 64。

表 3-7 梯度优化策略的压缩比对比

训练数据	模型	压缩比例			
		8-Bit	TernGrad	DGC (refer.)	GradSA
Cifar10	Inception-v2	2×	16×	278×	490×
	ResNet-50	2×	16×	277×	498×
	ResNet-56	2×	16×	255×	495×
	ResNet-110	2×	16×	229×	415×
ImageNet	ResNet-50	2×	16×	269×	508×
	AlexNet	2×	8×	270×	514×

注：压缩比在每次迭代中略有变化，因此在 20 次迭代后取平均值。每个模型基线的压缩比为 1。

梯度压缩率的比较：表 3-7 展示了上述梯度稀疏方法相应的梯度压缩比。当分布式训练过程的梯度稀疏度设置为 0.1%时，通信成本将显著降低，DGC 和 GradSA 提供的压缩比例分别超过了 255 倍和 415 倍。相比之下，其他基于低精度定点化的梯度

量化方法只能获得非常有限的压缩率。值得注意的是，在使用 3.4 节提出的基于索引的编码方法之后，在互联网中进行传输的稀疏梯度的规模更小，这将使 GradSA 在移动或边缘训练场景中更具竞争力。具体来说，与基线相比，GradSA 的梯度压缩比为 514 倍，而 DGC 和 Terngrad 在 AlexNet 上的压缩比分别为 270 倍和 8 倍。

收敛行为对比：图 3-5 描述了三种策略在不同训练节点数量（分别为 16 个 worker 和 32 个 worker）下收敛曲线的对比。所有的方法都在 160 个 epoch 后终止训练，并使用相同的超参数配置。结果表明，与基于无稀疏压缩的全精度梯度（Full-Size Gradient）SGD 基线相比，GradSA 具有良好的收敛性。由于梯度老化的问题，与基线及 GradSA 训练方法相比，DGC 的学习曲线表现得较差。值得注意的是，当 worker 数量增加到 32 时，由于全精度梯度 SGD 使用的一些超参数并不适合梯度稀疏的 SGD 方法，例如学习率和权重衰减（Weight Decay），因此 GradSA 的训练损失略高于基线。但是在 32 个工作节点时，GradSA 的测试精度可以保持与基线方法的基本相同，如图 3-6（b）所示，因此超参数选择不当所带来的副作用几乎可以忽略不计。

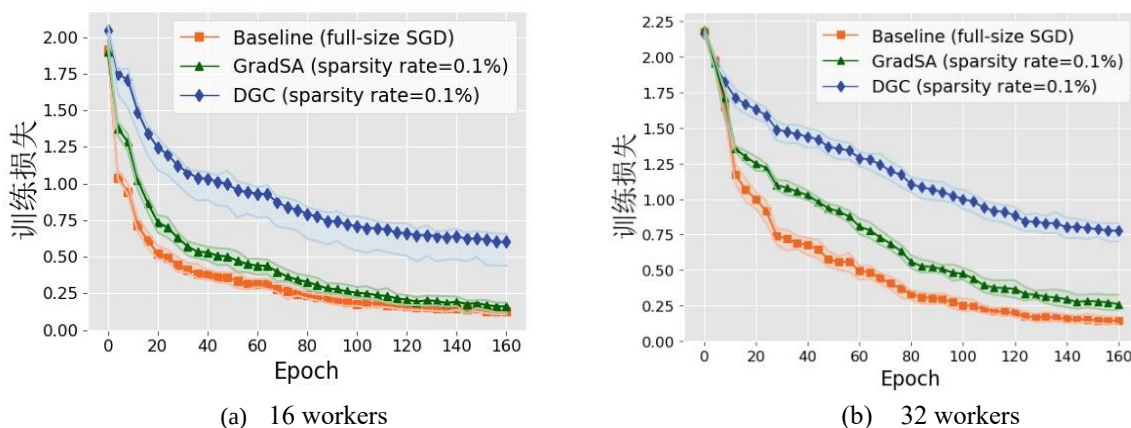


图 3-5 ResNet-56 学习曲线

除此之外，GradSA 另一个潜在的优势是可以加速模型的收敛，即在运行相同的 epoch 时能够有效减少整体的训练时间。图 3-6 为 top-1 测试精度与训练时间的对比图。为了在 ResNet-56 上达到类似的测试精度，在 32 个 worker 的情况下，基线方法的训练时间比 GradSA 多 6 倍，而 DGC 消耗的时间比 GradSA 多 1.74 倍。将梯度减小到极端尺寸可以充分解决分布式训练的通信瓶颈问题，显著加快模型训练收敛速度。

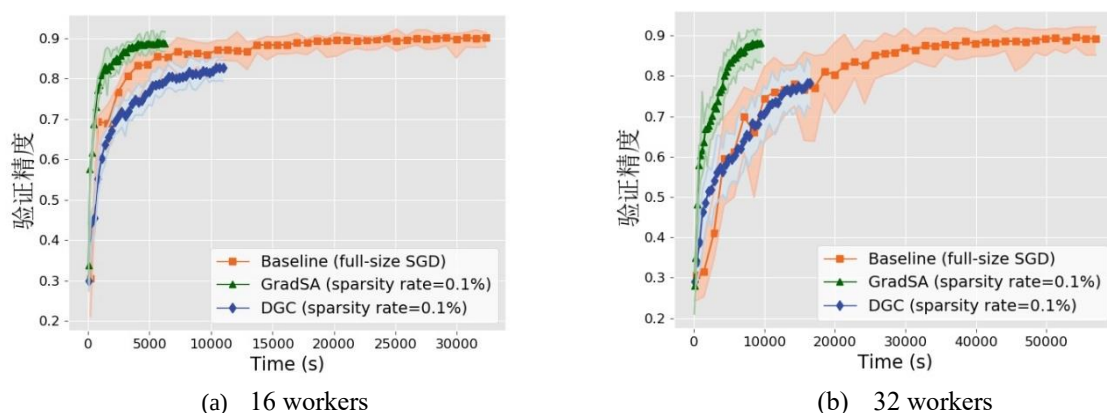


图 3-6 ResNet-56 测试精度与训练时间对比

3.5.3 时间效率评估

为了更好地展示 GradSA 的有效性，以 16 个 worker 为例，设置 0.1% 的梯度稀疏率，将一次迭代的时间分为四部分：（1）GPU 计算时间 t_{compu} ，即前向计算与后向计算时间之和 $t_{forward} + t_{backward}$ ；（2）局部梯度稀疏处理时间 t_{top-k} ，即梯度稀疏 Top-k 筛选的时间开销；（3）梯度编码时间 $t_{encoding}$ ，即将稀疏梯度编码为稠密梯度所花费的时间；（4）通信时间 t_{commu} ，即从参数服务器端拉取和推送梯度更新所需的时间。

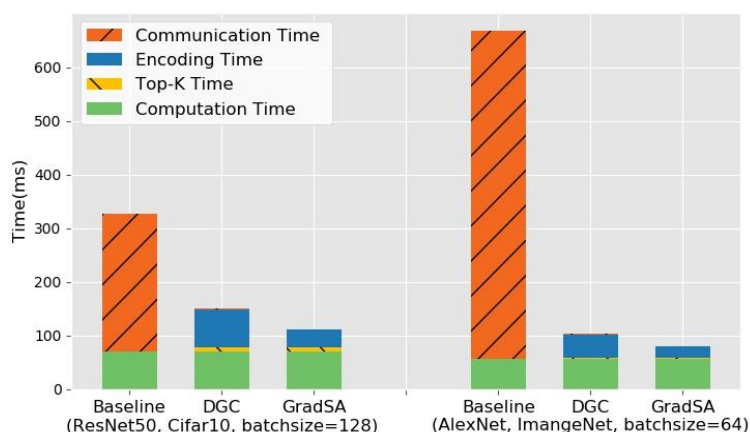


图 3-7 不同模型的时间开销对比

注：这里使用 16 个工作节点，以及 100Gbps 的 InfiniBand 网络。

图 3-7 统计了经过 100 次迭代后三种优化方法的时间消耗情况。可以看到即使当前网络带宽很高，比如 100Gbps，通信成本过高这一问题仍然严重阻碍着分布式训练

的效率。与使用全精度梯度的 SGD 基线相比, GradSA 的通信开销显著减少, 整体的运行时间也有明显减少。与使用 RLE 作为压缩策略的 DGC 相比, GradSA 消耗了更少的编码压缩时间。此外, GradSA 的通信交换时间也比 DGC 短, 这是由于在 0.1% 的极端稀疏率下, 基于索引方法的压缩比例远远高于 RLE 方法, 这一点在表 3-4 有所体现。然而, 与各个神经网络模型前后向计算所消耗的时间相比, 梯度编码消耗掉时间的规模仍然相当可观, 这可能是由于基于 GPU 的编码压缩方法并行性较差, 在未来使用更高效的并行编程方法将有助于解决这一问题。

3.5.4 系统可扩展性评估

为了评估 GradSA 的可扩展性, 本节调整了文献[102]中提出的性能建模方法, 把单工作节点的性能监测与基于参数服务器架构分析建模方法结合起来, 将其扩展到基于多机多卡的 GPU 深度学习系统中。这里以基线系统 MXNet 为例, 具体阐述性能建模过程。为合理估计整个多机多卡训练系统的可扩展性 (吞吐量), 首先定义集群中有 i 台机器, 每台机器上有 j 张 GPU 卡, 单 GPU 卡上训练数据的批量处理尺寸为 $N_{minibatch}$, 通常意义上的“计算节点”是指单个 GPU 卡, 这里 $N_{worker} = i * j$ 。然后定义两种主要的通信模式: 1) PS 通信模式^[84]: 即每个 worker 节点均与 server 节点进行通信, 直到将所有节点参数梯度汇总至 server 节点上; 2) Allreduce 通信模式^[103]: 即每个 worker 节点仅与邻居节点进行通信, 直到所有节点参数梯度都能够汇总到指定的某一 worker 节点上。同时, 由于 MXNet 训练系统中跨主机的节点通信和主机内部的节点通信时所采用的通信模式各不相同, 因此需要对这两者进行区分对待。具体来说, 主机内部节点通信采用 Allreduce 通信模式, 而同时跨主机通信则可以选择采用 PS 模式或者采用 Allreduce 模式。这里将对训练系统的计算过程和通信过程两个阶段进行建模得到各自的时间开销。

计算阶段: 不再对计算阶段的多个步骤 (即前向计算、后向计算、局部梯度稀疏处理、梯度编码以及参数更新计算等除参数通信以外的计算阶段) 分别进行建模, 而是直接对这一阶段的时间开销进行测量统计。注意这里仅统计一个计算节点上的批量处理规模为 $N_{minibatch}$ 时的计算时间开销, 并将其定义为 $U_{f,b,w}(m)$, 其中 m 表示当前的

参数模型。

通信阶段：对于主机内部节点通信，使用 Allreduce 通信模式并对其进行性能建模，对于主机内的 j 张 GPU 卡，其时间复杂度为 $O(\log_2 j)$ 。用 C_{gbw} 表示 j 张 GPU 卡之间的通信带宽，用 C_{cbw} 表示主机 CPU 和 GPU 之间的通信带宽，用 $W(m)$ 表示参数梯度规模，则主机内部通信开销建模如公式（3-16）所示：

$$M_{within} = \frac{W(m)}{C_{gbw}} \times \log_2 j + \frac{W(m)}{C_{cbw}} \quad (3-16)$$

对于跨主机节点通信，使用 PS 通信模式并对其进行性能建模，当系统扩展到更多机器时，对应的通信开销将会明显增加并导致系统可扩展性较差。最糟糕的情况发生在所有 worker 节点同时将参数梯度聚合到同一个 server 节点上，这时 worker 节点与 server 节点的通信带宽将会变得非常小。用 C_{ncost} 表示多机之间互联网络的延迟， C_{nbw} 表示多机互联网络的带宽，用 $W(m)$ 表示参数梯度规模，跨主机节点通信的成本建模如公式（3-17）所示：

$$M_{across} = C_{ncost} + \frac{i \times W(m)}{C_{nbw}} \quad (3-17)$$

综上，整体多机多卡通信阶段的成本可建模为公式（3-18）所示：

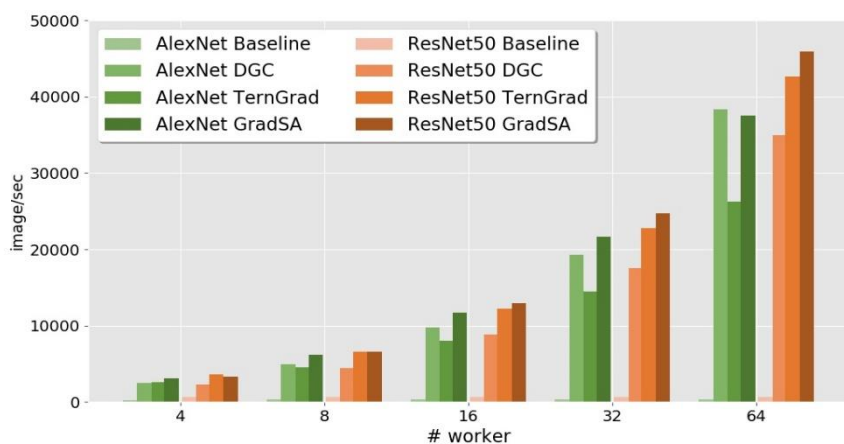
$$M_{across,within} = C_{ncost} + \frac{i \times W(m)}{C_{nbw}} + \frac{W(m)}{C_{gbw}} \times \log_2 j + \frac{W(m)}{C_{cbw}} \quad (3-18)$$

这里不再考虑将计算节点计算时间开销与通信时间开销进行重叠，用 N_{sample} 表示整个训练集的样本数目，并且对整个 epoch 的迭代时间进行估计，可得到一个 epoch 的整体时间开销建模，如公式（3-19）所示：

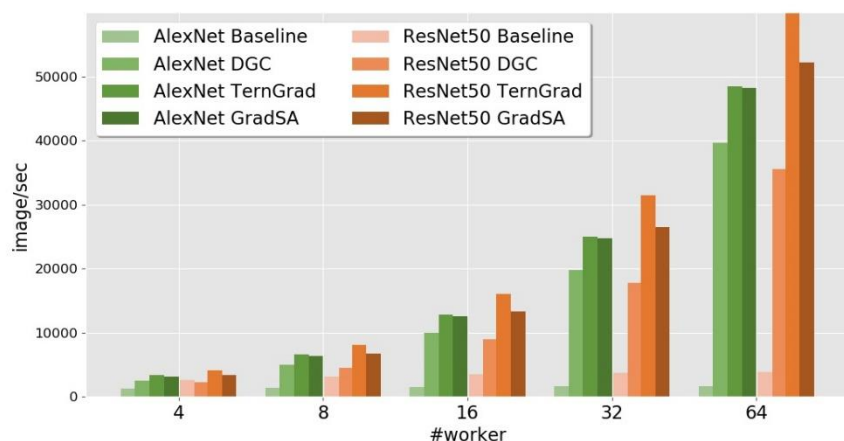
$$T_{epoch} = \frac{M_{across,within} \times N_{sample}}{N_{mini-batch}} + \frac{U_{f,b,w}(m) \times N_{sample}}{N_{mini-batch}} \quad (3-19)$$

本小节使用所提出的性能建模方法，在一个 NVIDIA P100 GPU 集群上将 4 个工作节点模拟扩展到 64 个工作节点，并在此配置下训练 AlexNet 和 ResNet-50 两个模型，统计对应的时间开销，如图 3-8 所示。观察得知，通信开销随着工作节点数量的增加而线性增加，这最终导致了系统扩展效率较低的问题。实验结果表明，在不同的

带宽环境下, GradSA 均能够显著降低网络通信成本, 获得最佳的吞吐量。这在很大程度上归功于 GradSA 的稠密通信集合 (Compact Communication Set), 由于其梯度的稀疏特性和较高的梯度编码效率, 使得该通信集合的规模比其他方法要小得多。



(a) 10Gbps 的以太网环境中 GPU 集群训练时吞吐量对比



(b) 100Gbps 的 InfiniBand 网络环境中 GPU 集群训练时吞吐量对比

图 3-8 不同 GPU 集群训练吞吐量对比

在 10Gbps 的以太网环境中, GradSA 通常比包括 TernGrad 和 DGC 在内的其他方法表现得更好。在时间性能方面, 在训练 AlexNet 时, GradSA 比 TernGrad 快 1.5 倍, 在 ResNet-50 的训练过程中, GradSA 比 TernGrad 快 1.09 倍。在系统吞吐率方面, 在训练 ResNet-50 时, 与 DGC 相比 GradSA 至少增加了 31% 的吞吐量。与此同时, 在 100Gbps 的 InfiniBand 网络环境中, GradSA 的性能略差于 TernGrad, 这主要是因为当网络带宽达到 100Gbps 后, 通信瓶颈这一问题得到了有效缓解, 超大规模的通信集合不再是高带宽环境下影响通信效率的主要因素, GradSA 的优势无法发挥。此外,

与 GradSA 和 DGC 相比, TernGrad 在梯度稀疏化节点没有额外的计算开销,这也是 TernGrad 表现良好的另一原因。但是与 DGC 相比, GradSA 仍有一定的优势,在 ResNet-50 的训练过程中提高了至少 46.7%的吞吐量。

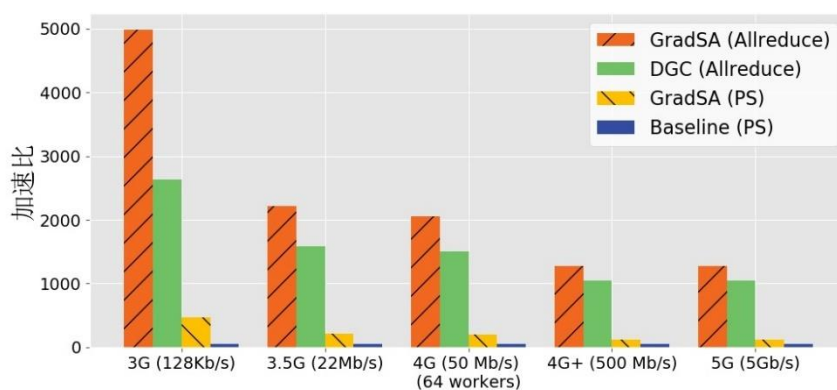
对低带宽环境的支持: 由于本节主要目标是在保障深度学习训练过程精确无损前提下,能够将其扩展到更多可能的移动设备或边缘设备。因此这里所做的尝试不仅包括考虑扩展到更多的节点,更应该考虑到各种移动应用所在的低带宽环境。值得注意的是,在图 3-8 (b) 中, GradSA 在 AlexNet 上的性能表现有所下降。这是由于在实验中 GradSA 使用的是参数服务器 (PS) 通信模式,所以在扩展到更多工作节点时性能表现稍逊。与去中心化的 Allreduce 模式相比,集中式的 PS 架构会导致通信负载量爆炸性的增长,总体通信负载的增加与工作节点数量的增加呈线性关系,通信成本线性增长。这也解释了当工作节点数量增加到数百个时 GradSA 表现较为一般的原因。与 PS 拓扑结构相比, Allreduce 更适合于低带宽环境。因此,本小节后续部分中 GradSA 采用 Allreduce 拓扑结构作为通信模式,可以将通信成本下降到次线性的数量级。

图 3-9 展示了 GradSA (Allreduce) 方法在低带宽环境下训练深度模型的优势。实验结果表明,在移动网络带宽环境中(网络带宽从 5Mb/s 逐渐降至 128Kb/s),由于通信规模及总体通信负载的显著降低,在 64 个工作节点上该方法仍然能够有效加速训练过程。当移动网络带宽降低到 128Kb/s 时,对比未使用任何通信优化策略的基线系统,在训练 ResNet-50 模型时, GradSA 的性能加速比提高了近 5000 倍,而 DGC 只提高了 2600 倍。由此可以证明, GradSA (Allreduce) 方法能够在较低带宽环境下保证分布式迭代训练的效率,为移动环境下部署训练任务提供了系统级支撑。

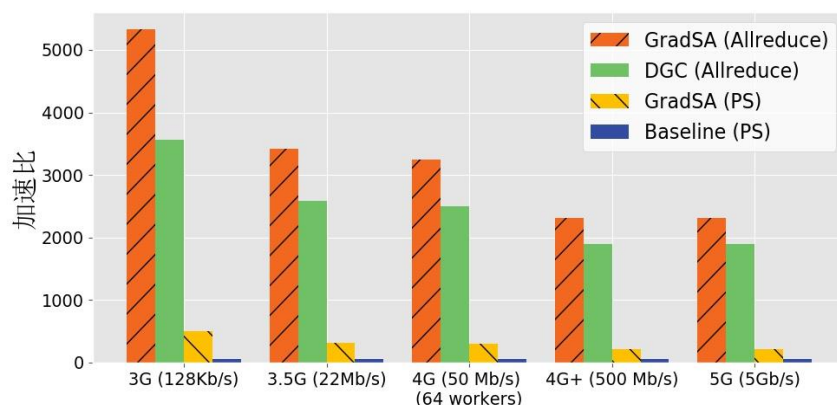
3.5.5 系统整体性评价

如第 3.1.2 节所述, GradSA 使用连续三阶段的稀疏梯度通信优化机制,这里将讨论这三个优化阶段各自对分布式通信过程的影响。具体地,这些技术包括:(1) 分层稀疏化 (Layer-Level Sparsification, LLS),其目的是筛选出更少的梯度来简化通信集;(2) 陈旧补偿积累 (Staleness-Compensated Accumulation, SCA),通过对历史梯度进行近似和积累来补偿过时梯度,加速收敛;(3) 高效编码通信 (Efficient Encoding

Communication, EEC): 其目标是压缩编码稀疏梯度, 进一步精简通信集。然而, 只使用单一的某一个策略并不能彻底解决通信问题, 应考虑如何在保持分布式训练的质量的前提下, 降低通信集合的规模。本章通过在 LLS 的基础上构建 SCA 和 EEC, 可以较为彻底地解决上述问题。LLS 和 SCA 这两种技术对训练质量产生了相当大的影响, 通过错误补偿机制帮助深度神经网络训练收敛到理想的测试精度。LLS 和 EEC 这两种技术帮助分布式系统减少了大约 500 倍的梯度通信规模, 有效缓解了系统通信瓶颈, 加速训练过程。可以说, GradSA 提出的三个策略彼此之间存在互补关系, 在优化过程中相互配合、发挥作用, 这也是本章的核心贡献之一。



(a) ResNet-50 的训练加速比



(b) AlexNet 的训练加速比

图 3-9 低带宽环境下训练加速对比

3.6 本章小结

分布式训练中工作节点间的梯度通信往往成为性能瓶颈, 在低带宽环境下, 梯度

通信延迟进一步阻碍了分布式训练。本章针对训练时梯度通信受限问题进行研究，该问题目前存在三大主要挑战：第一是梯度稀疏优化带来的梯度老化问题，可能导致模型训练无法收敛；第二是当前工作仅仅考虑对整个深度网络进行稀疏化，只从整个网络中筛选出一小部分梯度，而忽略了筛选策略本身对训练效率的影响；第三是如何高效地对稀疏后的梯度集合进行编码和解码这一问题还没有得到解决。为解决上述问题，本章提出了一种梯度稀疏优化机制，用于分布式深度学习系统，以提升通信效率。GradSA 关注的是如何在训练过程中显著地减小梯度的通信规模，同时又不会对 DNN 模型的训练精度产生明显的影响。实验结果表明，即使在通信网络带宽较低的环境下，所提出的梯度优化算法都能获得良好的吞吐量和加速性能。通过与 8-bit 量化、TernGrad 和 DGC 等方法进行比较，进一步证明 GradSA 在各种 DNN 模型和数据集上的优势。值得注意的是，本章所讨论的通信优化技术仍然存在一定的局限性，实际上各类的梯度稀疏优化处理方法均是以一定程度的计算代价来换取通信效率的提升，Top-k 的排序操作及编码解码操作都会带来相当比例的时间开销。但是，由于计算节点不断增加，通信开销的比例远远超过额外计算开销，包含迭代计算在内的整体计算时间均被通信时间所掩盖，因此 GradSA 更适合于通信瓶颈较为严重的训练任务中，能够在多节点、低带宽的环境下显著地加速训练进程。

4 深度学习系统运行时内存与通信协同优化机制

在深度神经网络的分布式训练过程中，参数、特征映射等中间数据占据着大量的 GPU 内存空间，与此同时，在计算节点之间进行参数梯度通信也在消耗着大量的互联带宽资源，然而，当前加速硬件的存储及带宽资源并不足以满足大规模神经网络训练的需求。因此，多种硬件资源与模型架构之间的矛盾已经成为限制深度学习领域发展的主要瓶颈。当前已有研究工作针对单一硬件资源受限的问题进行优化，但是由于神经网络模型对资源的需求往往来自多方面，单一地从某一角度优化并不能彻底改善训练运行时资源受限的问题。为此，提出对内存与通信资源进行协同优化的思路 LaySA，首先对内存优化策略进行数据对象增广与重新定义，解决由稀疏通信引起的内存膨胀问题，其次提出镜像参数更新机制，解决参数数据的内存优化难以与稀疏通信优化共存的矛盾，最终实现内存与通信两类优化方案的深度融合、协同执行。实验表明，与仅采用 Layrub 和 GradSA 的深度学习系统相比，LaySA 最高能够节约 80.5% 的系统内存使用量，神经网络模型在单 GPU 上整体训练时间缩短了 12.25% 左右，除此之外，LaySA 能够使用更大批量数据进行分布式训练，系统吞吐量整体提升 150% 以上，优于单独使用稀疏通信优化机制的分布式训练系统。

本章内容组织如下：第 4.1 节分析了分布式训练系统在多重资源优化方面存在的空缺，并阐述了研究的思路；第 4.2 节详细介绍了 LaySA 的具体策略和实施步骤；第 4.3 节介绍了算法实现及系统工作流程；第 4.4 节介绍了实验环境和评估设置，通过全面的实验对 LaySA 的性能进行验证；第 4.5 节对本章研究内容进行总结。

4.1 研究动机

本节首先分析训练运行时合并使用内存优化与分布式通信优化两个策略的必要性，随后分析了合并使用两个优化策略时的难点，即二者共存时所产生的矛盾，最后提出解决该矛盾的基本思路 and 设计方案。

4.1.1 问题提出

第2章分析了深度学习系统中卷积神经网络模型的内存分配行为特征,对特征映射数据使用基于空间复用的内存管理策略 Layrub,实现了单机训练运行时高内存利用率的数据组织方式;第3章分析了深度学习系统中分布式训练过程的参数数据交换特征,并基于分析结果对参数梯度数据进行合理稀疏以及高效编码,实现了训练系统中高时效的分布式数据通信机制 GradSA。上述两项优化工作针对训练系统中单一的资源受限问题,对深度学习系统的内存开销和网络带宽开销分别做出了优化。但是,实际上在计算机视觉、自然语言处理等典型应用中,资源受限往往同时体现在多个维度上,单一从某角度进行优化,仍然无法从根本上改善训练运行效率,难以为神经网络算法设计和进一步应用研究提供系统级的支撑。以自然语言处理领域为例,2018年谷歌公司提出了基于神经语言模型的预训练网络 BERT,该网络在当年一枝独秀、独领风骚,以一己之力在众多下游任务达到领先水平,刷新了各大自然语言处理类竞赛的历史记录。但是,由于其分布式训练过程中对内存、计算、通信带宽等资源需求都比较大^[36],为了完成训练任务、保证分布式训练效率,需要同时完成运行时的内存优化和通信优化工作,因此有必要在分布式训练阶段提出内存和通信的协同优化机制,以达到提升训练效率、高效利用系统资源的目的。

然而,直接对两类优化策略进行简单合并,并不能实际解决多种资源受限的问题,甚至还会阻碍训练效率的提升。例如,在某些神经网络模型中使用参数通信优化策略会进一步的增加内存压力,而内存优化策略的引入可能会影响分布式迭代计算的正确性。因此如何能够应对神经网络训练任务所带来的内存资源、通信带宽资源等方面的挑战,对内存与通信两类优化方案进行深度融合、协同执行,是目前有待解决的问题。

4.1.2 解决思路

回顾第1.1节中对中间数据的分类与特性总结,结合第2章和第3章提出的内存优化及分布式优化策略,本小节分析研究不同模型在优化前后的内存分配行为规则,并提供两个重要观察结果。根据这些观察结论获得启发,进而提出解决思路,即保障

内存高效的分布式参数稀疏通信机制，兼顾不同类型的神经网络，支持超大规模网络的训练，且能够在一定程度上对训练性能有所保证。

观察结论一：当前基于历史梯度累加的稀疏通信方法往往造成内存膨胀的问题。分析可知，虽然基于历史梯度累加的稀疏通信方法能够有效减轻历史梯度的陈旧程度、纠正历史梯度的累积偏差，从而保障模型训练精度、加速训练收敛，但是，迭代累加的历史梯度被作为反馈误差，长期驻留在内存中，因而需要为其分配适当的内存空间，此时的训练过程如图 4-1 所示，历史累加梯度的数据规模与参数及其梯度完全一致，因此稀疏通信机制下系统运行时参数及梯度所需的内存空间增加了 50%，加速硬件的内存压力明显增加

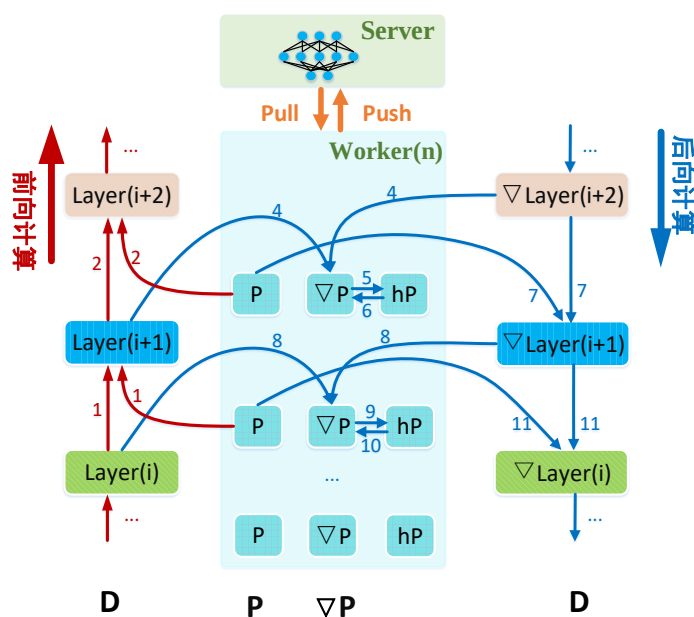


图 4-1 基于稀疏通信的训练过程

以自然语言处理领域为例，词向量作为研究基础被多种下游研究与应用广泛使用。然而在基于神经语言模型得到词向量的训练过程中，词向量被看作待更新的模型参数，具备维度高、数量大等特点。随着词表和维度的增加，参数规模甚至高达 2GB 以上，这时对其实现基于稀疏梯度优化的分布式加速时，参数及其梯度数据的规模将会增长至 6GB 以上，运行时的整体内存开销将会超过一些主流 GPU 的容量限制。实验统计了多个不同种类的神经网络模型在使用通信优化机制前后训练运行时中间数据所需的内存开销，如图 4-2 所示，可知 BERT 等一类基于 Transformer 的通用语言模型在

预训练过程中，也同时存在着参数等中间数据规模过大导致内存超载的问题。因此，如何能够在分布式稀疏通信优化的基础上，对包含参数、参数梯度、以及历史累加梯度三者在内的中间数据进行合理放置和内存优化分配，解决因通信优化而造成的内存膨胀问题，是当前分布式训练相关研究应当解决的首要问题。

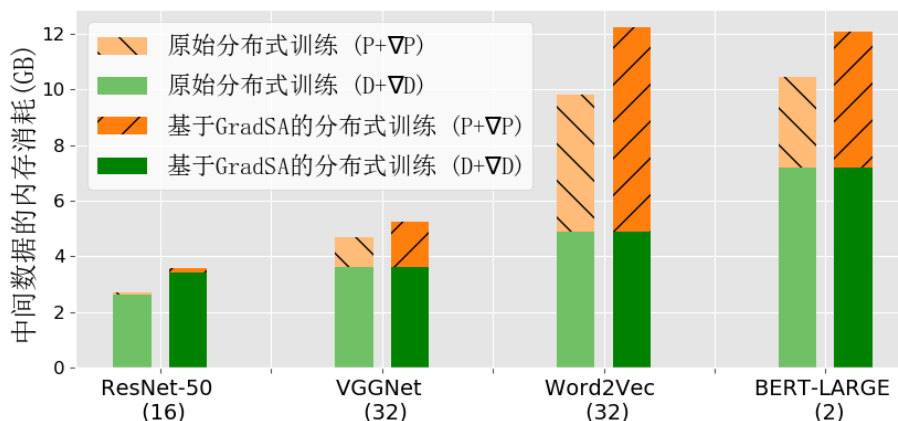


图 4-2 内存分配情况对比

注：这里的 P 和 ∇P 指参数及其梯度， D 和 ∇D 指特征映射数据及其梯度。训练测试时 VGGNet 使用 Cifar10 数据集，ResNet 使用 ImageNet 数据集，Word2Vec 和 BERT 使用 Wikipedia 数据集，模型名称下面括号内的数字表示训练时使用的批量处理尺寸，Word2Vec 模型配置：词向量维度=300，词表长度=1,115,011；BERT 模型配置（fine-tuned 训练）：序列长度=256，token 数量=512。

观察结论二： 直接对参数及其梯度数据应用当前的内存优化策略可能会阻碍正常的分布式参数通信过程。根据第一个观察结论可知，在神经网络训练任务中，通信优化的开展需要内存优化作为支撑。这里最直接的想法是将第 2、3 章所提出的内存优化和通信优化策略进行简单地合并。然而，由于数据不一致问题，两个策略无法直接集成到一个训练系统中，这将会对参数更新阶段产生直接影响。具体来说，当针对神经网络的参数及其梯度使用内存优化策略时，系统自动对这两者执行基于层结构的空闲复用，将驻留在 GPU 内存中的参数及梯度按照层的粒度适时迁移到 CPU 主存中。与此同时，当针对参数及其梯度使用稀疏通信优化策略时，将会在本节点对梯度进行分层的稀疏削减及累积，并及时上传至中心节点，同时拉取中心节点的全局参数进行更新。值得注意的是，当两个优化策略同时使用时，GPU 内存中驻留的参数及梯度仅为神经网络模型当前层所有，而梯度上传与参数拉取时需要网络模型所有层的数据，这时将会引起梯度数据缺失及参数写入越界的问题。梯度数据存在缺失的上传操作会导致训练收敛速度减慢，参数更新时出现内存越界则会影响整个程序的正确性。因此，

如何能够在完成参数及梯度数据内存优化的前提下,保障分布式参数通信的正确性和高效性,对梯度上传和参数拉取两个阶段的数据访问进行合理重定向,是当前分布式训练加速相关研究的工作重点。

总的来说,受上述一系列观察结论的启发,提出一种分布式深度学习系统运行时内存一通信协同优化机制,简称 LaySA。首先解决内存优化与通信优化共存所导致的数据缺失及内存越界问题,然后实现针对参数及梯度等中间数据的内存优化策略,完成针对内存资源与通信带宽资源的协同优化,最终达到兼顾系统资源利用效率与神经网络训练效率的目的。LaySA 的几个重要设计目标简要描述如下:

支撑超大规模神经网络完成分布式训练任务:为减轻模型自身及参数通信优化给系统内存资源带来的双重压力,重新实现 Layrub 的跨层复用策略。对 Layrub 的内存复用策略进行重定义,对内存优化的数据对象进行增广,将内存复用的策略应用在更多数据对象中。将迭代计算及稀疏通信所需的中间数据从 GPU 内存迁入 CPU 主存,并在随后时间点上按需迁回 GPU,减轻层间内存消耗。在 CPU-GPU 数据合理迁移的基础上,实现 Layrub 的层内复用策略,挖掘层内计算和内存访问操作的独立性,尽可能的减轻层内内存消耗。该内存复用策略旨在应对多种形态的大规模神经网络的分布式训练任务,显著改善了极限规模的网络无法在分布式环境下高效运行的现状。

保障系统运行时能够对内存、通信协同优化:在重新实现内存优化策略 Layrub 的基础上,实现分布式参数通信优化机制 GradSA,对分布式参数更新过程进行重新定义,将 CPU 主存作为镜像访问区域,对参数更新阶段的数据访问进行合理重定向,解决梯度数据缺失及参数写入越界的问题。在完成内存一通信协同优化的同时,保证相关数据的正确性,改善分布式稀疏通信的内存开销问题,保障分布式训练加速效率。在单 GPU 内部保证内存高效,在分布式环境下保证训练加速,最终达到兼顾系统资源利用效率与神经网络训练效率的目的。

4.2 内存高效的参数稀疏通信机制

基于 4.1 节的研究分析,通过将 Layrub 与 GradSA 进行集成得到 LaySA,能够实现深度学习系统运行时内存管理与稀疏通信的有机结合,使得内存效率与分布式效率

得到进一步提升。集成后的 LaySA 技术以 Layrub 为主要内存管理策略，以 GradSA 为主要通信传递机制，在保障分布式参数通信流程的正确性的前提下，实现系统内存的高利用率。

本小节内容组织如下：4.2.1 节对 Layrub 的内存优化策略进行重新定义，将待迁移数据及空间复用的数据对象进行增广，尽可能的提升内存效率，为运行时参数通信优化奠定基础。4.2.2 节通过引入镜像参数更新的方式来确保分布式训练程序 GradSA 的正确执行。4.2.3 节展示了 LaySA 的协同优化过程，在完成内存优化的同时，协同优化分布式参数通信的效率。

4.2.1 内存复用策略重定义

在第 2 章中，由于卷积神经网络中占据最大体量的特征映射数据，Layrub 仅仅针对该类数据展开优化，并未将参数及其梯度考虑在内，而实际上在另一大类基于语言模型的神经网络中，参数数据占据了更大体量。与此同时，根据观察结论一可知，在进行稀疏通信优化的过程中，会对应产生更多参数梯度类型的中间数据，这两者的共同作用给系统内存负载造成了双重压力。因此在神经网络训练任务中，通信优化工作的开展需要内存优化策略作为支撑。有必要对 Layrub 的内存复用策略进行重定义（Memory Reuse Refinement），对内存优化的数据对象进行增广，将内存复用的策略应用在更多数据对象中，用以应对不同应用领域的各类神经网络模型的训练任务，尽可能减轻因为模型自身及稀疏通信给系统内存带来的双重压力，保障分布式参数通信的高效优化，为各类应用研究的进一步扩展提供系统级的支撑。

数据对象增广：与 Layrub 相比，LaySA 进行内存管理的数据对象增加了三种，总计包含五种数据类型，分别为特征映射数据、特征映射梯度、参数数据、参数梯度和历史累加梯度。

数据依赖分析：与 Layrub 所做的数据依赖分析过程相比，LaySA 增加了对稀疏梯度通信优化相关计算序列的分析，并得到解耦数据依赖的规则集。具体包含：（1）任意一个中间层的参数梯度计算操作应优先于同层的神经元特征映射对应梯度的计算操作；（2）当前层的特征映射梯度计算操作应优先于随后一层的参数梯度计算操作。

复用策略重定义：针对 LaySA 新增的数据对象，重新定义 Layrub 的层内复用及跨层复用的策略。如图 4-3 所示，在执行以层为中心的梯度计算任务时，一般只需要 5 个 GPU 的内存块（其中 2 块用来存储连续层的特征梯度，另外 2 块用来存储连续层的参数梯度，最后 1 块则用来存储当前层的历史梯度），训练所需的内存空间大小基本恒定。为实现这一优化目的，首先，通过对包含稀疏梯度优化在内的前后向计算序列进行数据依赖分析。然后，根据规则集在访存操作序列的适当位置插入同步 barrier 操作，保证计算过程的数据一致性。最后，根据重排后的操作执行序列，对各类数据实现自适应的分时放置和空间复用机制。与此同时，根据按层划分的操作执行序列，对相关位置的内存访问进行引用计数统计，划分更细粒度的访存操作序列，在适当位置插入数据迁移操作，根据引用计数器来确定何时从 CPU 主存提前预取及回迁相关数据。

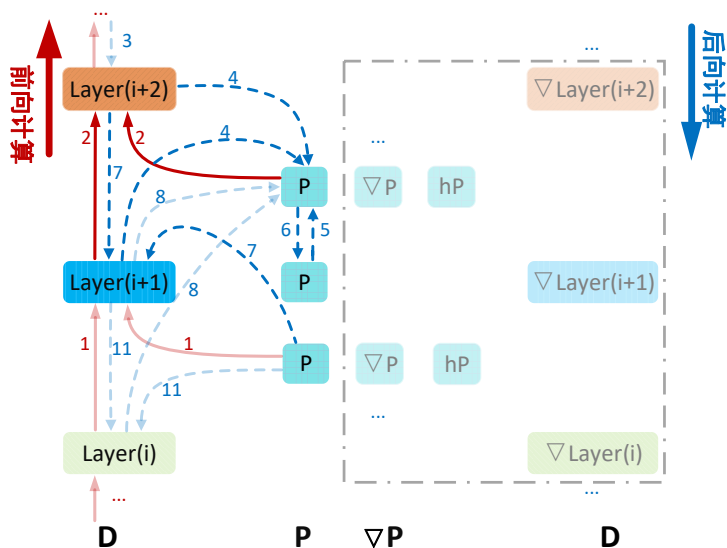


图 4-3 内存复用策略重定义

根据第 2 章的介绍，为了能够对训练性能有所保证，在实现包含稀疏梯度优化在内的前后向迭代计算时，使用 CUDA 流水线技术将其划分为多个计算流并且同时进行。在完成跨层的 CPU-GPU 数据迁移时，同样使用流水线技术，引入多个数据迁移流，在 GPU 内存和 CPU 主存之间按照层的粒度，同时根据计算序列的需要，提前将后续层的数据调度至指定位置，对计算任务和数据迁移任务实现并发执行，在前后向计算中隐藏数据迁移所带来的时间开销，达到最大程度的时间重叠。

4.2.2 镜像参数更新

作为分布式通信的算法核心，参数更新通常发生在远端中心节点与本地工作节点之间，主要包含两个阶段：本地梯度上传（push 操作），远端参数拉取（pull 操作）。与传统通信过程相比较，GradSA 的不同之处在于，当后向计算完成后，将会对梯度进行 Top-k 稀疏化、历史累加、及稀疏编码，在这之后，正常的梯度上传操作将会接管稀疏编码后的梯度，对其进行上传，并对应的拉取全局参数，对本地参数进行更新覆盖。因此，在参数更新过程中，GPU 内存作为主要的数据访问区域，梯度上传和参数拉取均在 GPU 内存中进行。

然而，根据观察结论二可知，由于内存优化策略 Layrub 的引入，GPU 内存空间中驻留的参数及梯度仅为网络模型当前层所有，而梯度上传与参数拉取时需要网络模型所有层的数据，这时执行参数更新操作将会导致梯度数据缺失及参数写入越界。而与此同时，Layrub 实际上将 CPU 内存作为迁移数据的缓冲区，在完成前后向计算之后，该区域已包含网络模型的全体参数及相关中间数据。因此，考虑将 CPU 内存作为参数更新过程的镜像访问区域，对参数更新过程进行重新定义，使用参数镜像更新（Weight Mirror Update）方式完成梯度上传和参数拉取过程。

镜像梯度上传：对原始参数更新阶段的数据访问位置进行重定向，从 GPU 内存区域重定向至 CPU 内存区域中。首先，将训练所需的全体中间数据（包含参数及优化后的梯度）所对应的内存空间完整映射到 CPU 指定内存区域中，将其作为镜像访问区域。然后，对 push 操作中涉及到访问梯度数据的部分进行重新定义，将其指向到 CPU 镜像对应位置。

分层镜像参数迁移：对原始参数更新阶段的数据访问时序进行重新排列，将镜像区域的模型参数按需、按层的迁移回 GPU，保障后向计算的正确性。首先，对 pull 操作涉及到参数拷贝的部分进行重新定义，将其指向到 CPU 镜像对应位置。然后，在远端中心节点通过 pull 操作拉取最新版本的全体参数。最后，在下一轮后向计算需要某一层的参数时提前预取，将其迁移回 GPU。

4.2.3 内存高效的稀疏通信策略

4.2 节分别介绍了重定义的内存优化策略以及镜像参数更新机制，使得 LaySA 在针对训练运行时各类中间数据完成内存优化的同时，能够对分布式参数梯度通信过程完成协同优化，保障分布式训练程序的正确执行，减轻因梯度稀疏优化及模型自身所带来的双重内存压力，在训练运行时获得较为显著的内存收益。

如图 4-4 所示，LaySA 在 worker 端完成内存优化及数据迁移操作，图中的 CPU 主存不仅作为参数更新阶段的镜像访问区域（即图中所示的 CPU Mirror Memory），同时作为内存优化阶段的中间数据缓冲区域（即图中所示的 CPU Buffer Memory），能够尽可能地减少 GPU 内存压力，为完成更大规模的神经网络训练任务提供可能性。在完成当次迭代的前向及后向计算后，LaySA 在 server 及 worker 端执行稀疏通信操作，这时 push 及 pull 操作均在镜像区域进行，能够在不影响性能的前提下，保障程序正确性。对比实验证明了该方法的有效性，能够在单 GPU 上为超大规模的训练任务获得最大程度的内存收益，同时在分布式 GPU 节点上达到较为理想的系统吞吐率。

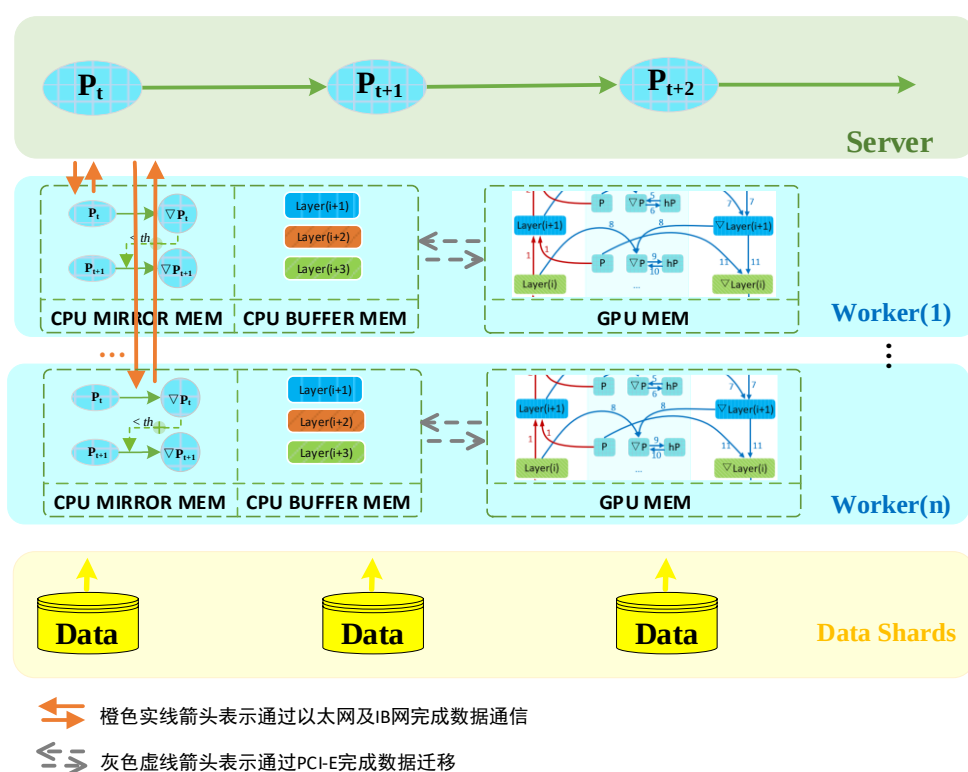


图 4-4 内存高效的参数通信

4.3 LaySA 原型系统设计与实现

本节首先描述 LaySA 的设计细节和算法实现，以深度学习系统 MXNet 为基础进行代码实现，然后用程序流程图的形式展示相应策略的实现过程，对内存复用策略进行适当改良，并完成分布式训练任务。

4.3.1 算法实现

4.2 节详细介绍了 LaySA 中内存复用策略和分布式通信优化策略的设计思想，在训练系统中执行内存高效的参数稀疏通信机制，本小节在此基础上说明 LaySA 的实现细节。神经网络模型同时部署在多个 GPU 计算节点上，对应的 worker 进程调用 CUDA 内核及库函数完成单个节点上的前后向计算，得到参数梯度；server 进程负责汇总多个计算节点的最新梯度。这个过程将会调用 LaySA 来管理内存空间的分配，并接管原始的前后向计算序列，同时完成分布式通信模块涉及到的算法优化及通信同步操作。

算法 4-1 给出一个使用 LaySA 进行训练迭代的示例。神经网络模型的单次迭代包含一次前向计算、一次后向计算、以及一次参数及梯度的双向通信。为对神经网络模型进行性能侧写，在模型初始化阶段采用与 2.3.1 小节类似的虚拟迭代方式获取相关信息。与 2.3.1 小节的算法 2-1 相比，算法 4-1 的不同之处在于，在正式训练阶段，模型按逐层顺序进行遍历，在对第 l 层执行前向计算之前需要调用 Loadin() 函数从 CPU 镜像区域预取参数及历史梯度，在对第 l 层执行后向计算之后需要调用 Offload() 函数将压缩后的梯度通信子集及历史梯度回迁至镜像区域，等待 worker 端执行参数更新操作。此外，为保证数据传输速率，在预留 CPU 端镜像区域内存时，选择使用页锁定内存的方式。在完成前后向计算后，将会对参数及梯度通信集合进行一次双向通信，算法 4-2 和 4-3 分别描述了 worker 端及 server 端在参数更新阶段完成双向通信的具体步骤。其中 worker 端的 push 操作从 CPU 镜像区域中把梯度通信集合传递到 server 端，同时通过 pull 操作拉取全局参数存储到镜像区域中，并对本地参数进行更新覆盖。

LaySA 执行训练迭代的实现细节如算法 4-1 的伪代码所示。

算法 4-1: LaySA 训练迭代

```
1. function Train(net)
2.    //L:一个网络(net)的所有层; T:总的迭代次数
3.    for l = 1 → L do
4.        Tracer[l].addRef() //初始化网络, 设置引用计数(Ref), 统计数据依赖关系
5.    end for
6.    for i = 1 → T do
7.        for l = 1 → L do //前向计算过程
8.            InterLayer[l].Loadin(weight_in_mirror) //从镜像区域预取参数及历史梯度
9.            layer[l].Forward(default_stream)
10.           Tracer[l].subRef()
11.           if Tracer[l].Ref == 0 then
12.               InterLayer[l].Offload(migrate_stream)
13.               Synchronize() //对 CUDA 流进行同步控制
14.               InterLayer[l].Reuse()
15.           end if
16.        end for
17.        for l = L → 1 do //后向计算过程
18.            layer[l].Backward(IntraLayer[l].Reuse(), default_stream)
19.            encode_list = ENCODE(sparse_topk(grad_arrays[l])) //对梯度进行稀疏及编码
20.            InterLayer[l].Offload(encode_list) //将压缩后的稀疏及历史梯度回迁至镜像区域
21.            InterLayer[l-1].Loadin(migrate_stream)
22.            Synchronize()
23.        end for
24.        update_on_worker(grad_arrays, kvstore) //worker 端执行参数更新
25.    end for
26. end function
```

在 worker 及 server 端完成参数更新的实现细节如算法 4-2 及 4-3 所示。

算法 4-2: worker 端的参数更新

```
1. function update_on_worker (grad_arrays, kvstore)
2.    for i = 1 → len(grad_arrays) do
3.        mirror_push(worker_encode_list) //push 操作将压缩后的梯度传递到 server
4.    end for
5.    server_grad_list = mirror_pull() //pull 操作从 server 端拉回全局参数
6.    update_in_mirror(server_grad_list) //在镜像区域中更新本地参数
7. end function
```

算法 4-3: server 端的梯度聚合

```
1. function update_on_server (grad_arrays, kvstore)
2.   for  $j = 1 \rightarrow \text{worker\_number}$  do
3.     for  $i = 1 \rightarrow \text{len}(\text{grad\_arrays})$  do
4.       worker_grad[i] = DECODE(worker_encode_list) //对 worker 端的通信集合解码
5.       aggre_grad(worker_grad[i], server_grad_list) //聚合梯度, 更新全局参数
6.     end for
7.   end for
8. end function
```

4.3.2 LaySA 系统流程

4.3.1 小节详细介绍了 LaySA 中内存复用策略和分布式通信优化策略的算法实现, 本小节将介绍 LaySA 内存复用和通信优化方法的工作流程, 整体的训练系统执行过程如图 4-5 所示。

(1) 在训练任务初始化阶段, 对指定的神经网络模型进行初始化, 虚拟执行前后向计算序列, 记录模型各层的数据依赖关系, 评估运行时各类中间数据的内存开销, 完成模型的性能侧写。

(2) 在模型的构建阶段, 首先, 判定当前训练任务是否在分布式环境下进行, 如果是, 则按照镜像参数更新方式在 CPU 主存中设置镜像区域, 为参数及梯度分配地址空间, 将梯度上传和参数拉取操作的访存位置定向到镜像区域中。随后, 实现计算操作序列的重排, 根据解耦数据依赖的规则集在计算序列的对应位置加入同步控制操作。同时, 根据模型侧写结果, 给中间数据预分配指定规模的内存空间。

(3) 在模型的训练起始阶段, 按预设的批量处理规模迭代式地加载训练数据。

(4) 在模型的前向计算阶段, 使用跨层空间复用策略, 在当前层前向计算完成后, 将特征映射及参数等中间数据逐层地、对应地迁移至 CPU 主存缓冲区及镜像区域。

(5) 在模型的后向计算阶段, 使用跨层空间复用策略, 提前将特征映射及参数数据回迁至 GPU 内存, 在当前层后向计算完成后, 使用层内空间复用策略, 将特征映射梯度、参数梯度写入对应内存位置。

(6) 在模型的参数更新阶段, 首先, 使用基于错误累加的稀疏梯度优化机制, 将

历史梯度累加到本地的最新梯度上，对梯度完成稀疏优化及压缩编码的操作，并将其迁移至 CPU 镜像区域，等待上传；随后，在镜像区域完成梯度上传及参数拉取操作；最后，等待下一轮的后向计算阶段，将镜像区域已更新的模型参数按需、按层地回迁至 GPU 内存。

(7) 如果符合迭代结束的判定条件，终止训练；否则继续循环步骤(3)~步骤(7)。

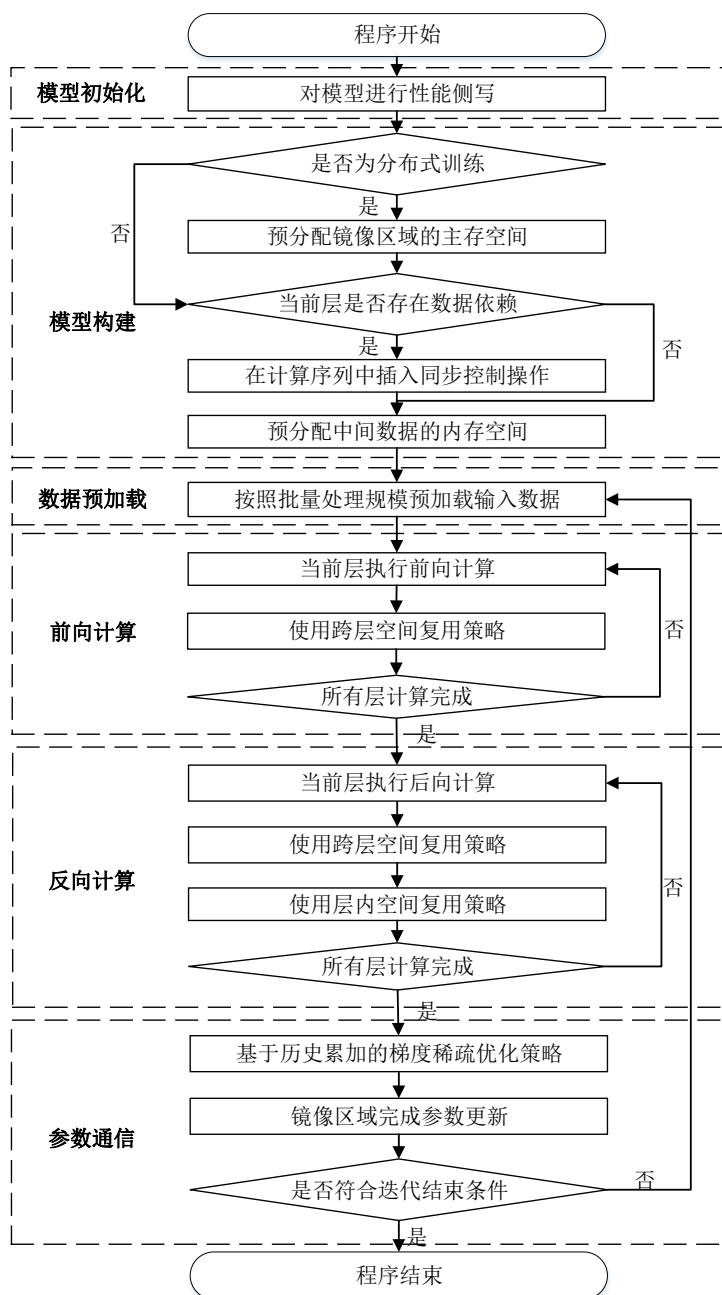


图 4-5 LaySA 训练执行流程图

4.4 有效性评价与分析

本节评估 LaySA 在深度学习系统中的性能表现，在 4.4.1 小节选择了具有代表性的深度神经网络模型和训练数据集，目前这些模型已经广泛应用于自然语言处理和计算机视觉等多个研究领域。在 4.4.2 到 4.4.4 小节主要对实验结果进行详细介绍和分析。对比实验主要涉及内存优化效率、整体训练效率、以及在训练极限规模神经网络方面的探索。

4.4.1 实验设置

实验环境设置：实验是在两个分布式 GPU 集群上进行的，分别使用两种 GPU，具体型号与配置如表 4-1 所示。GPU 通过 PCI-E 总线（gen3）与主机 CPU 通信，该总线提供最高 16Gbps 的数据传输带宽。这些节点通过 InfiniBand 和以太网相互连接，InfiniBand 网络带宽为 100Gbps，以太网环境带宽为 10Gbps。LaySA 的实现和部署均在深度学习框架 MXNet 上进行，MXNet 版本是从 Apache 中获取到的。除另有说明外，所有实验均在 NVIDIA P100 GPU 上进行。

表 4-1 系统配置描述

GPU 配置	机器配置	软件环境
NVIDIA Tesla P100-PCIE GPU, 16276MB	Intel Xeon E5-2682 (2.50GHz 24-core), 256GB RAM	64-bit Ubuntu 18.04, CUDA toolkit 10.2, cuDNN v7.6

模型和数据集：实验选取了代表性的 DNN 模型，如 AlexNet、VGGNet、ResNet、BERT，这些模型均在机器视觉及自然语言处理等任务中得到了广泛的应用。模型的所有配置都遵循默认超参数（学习率、动量等）。此外，使用了图像分类数据集：Cifar-10 和 ImageNet；以及语料文本数据集：Wikipedia。

实验对比方案：实验对比主要包括两个部分。首先，为验证所提出内存复用策略的内存效率，实验测量前后向计算及稀疏优化计算所需中间数据的内存开销。使用基于稀疏梯度的分布式训练方式 GradSA 作为基线，对比参数对应中间数据优化前后的内存开销，展示 LaySA 协同优化策略的优势。其次，为验证 LaySA 在其他各个方面的性能优势，实验测量整体模型的训练时间，并且探索单个 GPU 上可能训练的神经

网络模型规模的极限，即训练时单 GPU 所能支持的最大训练数据批量规模。由于在大多数系统（如 Caffe 和 MXNet）中，在开始迭代计算之前会默认给所有中间数据以静态方式预分配内存空间，因此可以准确地测算出中间数据及其相应梯度的内存消耗。在实验中使用 NVIDIA 系统管理接口（nvidia-smi）测算运行时的内存使用情况。

4.4.2 内存效率评估

LaySA 的思想是在分布式通信优化的基础上，通过复用神经网络模型训练过程中的多种中间数据的内存空间，来尽可能地提升系统整体内存效率。为了验证基于数据增广的内存复用策略的有效性，选择了第 2 章的内存优化方法 Layrub 作为对比对象，在完成梯度稀疏通信优化 GradSA 的分布式训练环境中，对多个神经网络模型（包括 AlexNet、VGGNet、ResNet、Word2Vec、BERT-BASE 和 BERT-LARGE）进行了运行时内存测试对比。实验结果如图 4-6 所示，图中原始分布式训练方式是指未使用任何内存优化方法的原始 MXNet 系统；GradSA 是指未使用内存优化方法的稀疏通信训练系统；LaySA（M）是指使用 Layrub 作为内存优化方法的稀疏通信训练系统，这里使用参数镜像更新方式解决 Layrub 和 GradSA 无法共存的矛盾；LaySA（M+R）是指使用数据增广的内存复用策略后的稀疏通信训练系统。

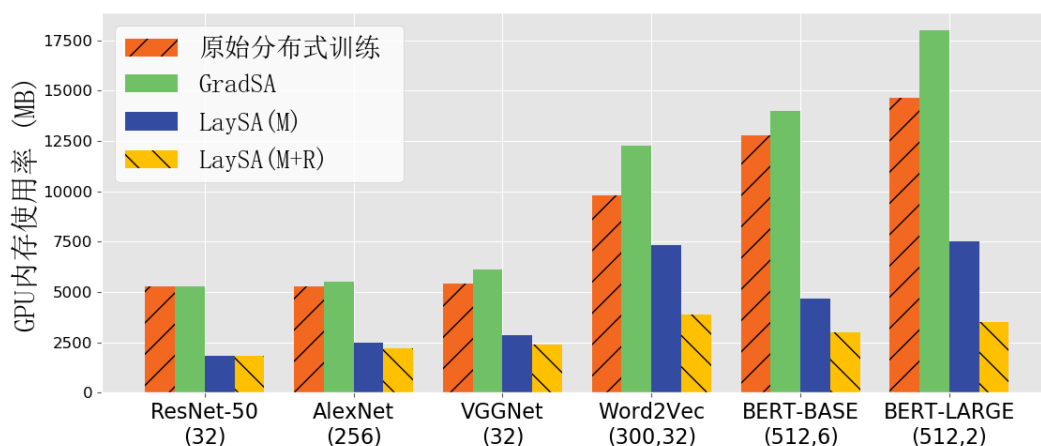


图 4-6 训练运行时内存分配情况

注：M：镜像参数更新，R：内存复用策略重定义。训练测试时 VGGNet 使用 Cifar10 数据集，ResNet 及 AlexNet 使用 ImageNet 数据集，Word2Vec 和 BERT 使用 Wikipedia 数据集，模型名称下面括号内的数字表示训练时使用的批量处理尺寸，Word2Vec 模型配置：词向量维度=300，词表长度=1,115,011；BERT 模型配置（fine-tuned 训练）：序列长度=512，token 数量=512。

图 4-6 展示了各类神经网络在训练过程中进行内存优化后的内存使用量。其中,提出的 LaySA 在分布式稀疏通信优化的基础上,在所有神经网络模型上均能够实现最大的内存节约比例,并且神经网络的参数等中间数据规模越大, LaySA 的内存效率越高。例如在 AlexNet 和 ResNet-50 模型训练中, LaySA 方法的 GPU 内存使用量仅仅比 Layrub 方法降低了 1.4%,这种程度的差异几乎被忽略不计。然而,随着模型结构的变化以及参数规模的增长,在 BERT-LARGE 模型训练时, LaySA 方法的 GPU 内存使用量比 Layrub 降低了 53%,比未使用内存优化策略的 GradSA 方法降低了 80.5%。

分析得知, LaySA 方法在不同神经网络结构中性能表现不同的主要原因在于,特征映射数据或者参数数据的规模在各类神经网络中并非始终占据主体部分,例如,在卷积神经网络 AlexNet 和 ResNet-50 中,特征映射数据大约占总数据量的 80%以上,这时 LaySA 方法退化为 Layrub 方法。而在另外一大类基于语言建模的网络模型 Word2Vec 和 BERT 中,参数规模相对较大,甚至在某些模型配置时参数和特征映射各自占据总数据量的一半左右。因此使用 Layrub 方法针对特征映射数据进行内存优化,仅仅能够减轻卷积神经网络这一类模型的内存压力,无法兼顾另一大类面向自然语言领域的神经网络模型。而 LaySA 在解决 Layrub 和 GradSA 共存矛盾的同时,针对参数、梯度、累加梯度三者进行合理的内存复用,进一步降低了各类神经网络训练运行时的内存消耗。

小结: LaySA 针对训练运行时的多种中间数据进行内存管理,能够应对各种不同形态、结构的神经网络,在大规模神经网络的训练任务中达到较为理想的内存效率。

4.4.3 超大规模训练任务

4.4.2 小节在各类神经网络模型上验证了提出的 LaySA 方法在内存效率方面的优势,解决了内存复用策略 Layrub 与稀疏通信优化方案 GradSA 无法共存的矛盾,表明 LaySA 能够显著降低模型训练时的内存消耗,减轻由稀疏通信优化带来的内存压力,特别是在语言建模类神经网络模型上,其内存优化效果更为明显,这得益于 LaySA 方法能够处理前后向计算及稀疏优化两阶段所产生的大多数中间数据,分别对这些数据类型实现内存空间的复用。

在完成内存节约之后，本小节将探索 LaySA 方法能够完成的训练任务的最大规模，也就是神经网络训练运行时能够同时处理的最大数据批量规模（batch size）。在单个 GPU 内存容量有限的情况下训练大规模网络模型，如 ResNet、BERT 等，可容纳的数据批量越大，则 GPU 的内存资源利用率就越高。与此同时，在单个 GPU 内部使用更大的批量处理规模，可以保证训练过程中收敛行为的稳定性，进而保障分布式训练的迭代效率。因此，在包括 BERT 在内的许多语言建模类模型设计过程中均对批量大小（batch size）和序列长度（sequence length）有着一定要求，例如，在 BERT-BASE 中为保证训练精度，需要在设置 sequence length=512 的情况下 batch size 不小于 16。然而当 batch size 设置为 8 以上时，已经超出主流 GPU 16GB 的容量限制。因此，在计算集群中单个 GPU 内存资源受限的情况下，通过内存优化策略扩展批量处理规模非常必要，能够为分布式训练扩展提供有力的保障。

表 4-2 批量尺寸及内存消耗对比

网络模型	最大序列长度	最大批量尺寸	运行时内存消耗		
			MXNet	Layrub	LaySA
BERT-BASE	512	8	15433MB	5578MB	4250MB
		10	OOM	6510MB	5850MB
		30	OOM	15834MB	10850MB
		48	OOM	OOM	16109MB
BERT-LARGE	512	2	14667MB	7500MB	3500MB
		4	OOM	8400MB	4050MB
		12	OOM	15600MB	11250MB
		22	OOM	OOM	15250MB

注：这里对 BERT 模型进行 fine-tuned 训练测试，OOM 表示内存不足（Out of Memory）。

为了探究 LaySA 在单 GPU 上可处理批量规模的极限情况，对比测试了不同深度学习系统在 Tesla P100 GPU 上能够成功训练的 BERT 最大批量尺寸。实验结果如表格 4-2 所示，LaySA 在所测试的两个 BERT 模型上都有较好的扩展性。特别是对于层数较深（24 层）的 BERT-LARGE 模型，在序列长度设置为 512 时，LaySA 所能支持的最大批量大小可达到 22，已经完全满足模型在单个 GPU 上的推荐配置，能够在训练过程中达到预期的收敛效果。相比之下，原生的 MXNet 系统仅能在单个 GPU 上处理批量大小为 2 的训练任务，当批处理规模继续加大，MXNet 的训练任务已经无法再

进行下去。而同时地，由于 Layrub 方法无法在分布式环境下对参数及其中间数据进行内存优化，因此该方法的可扩展性非常有限，仅能处理批量大小为 12 的训练任务，影响分布式训练迭代效率。

此外，在增加批量处理规模的同时，测试对比了批量规模缩放前后模型的训练时间开销。实验结果如表格 4-3 所示，在一系列不同种类的神经网络模型上(包括 AlexNet、VGGNet、ResNet 和 BERT)进行了测试对比，并记录了数据批量从正常尺寸(BS_{norm})缩放到极限尺寸(BS_{extreme})时，分别迭代完成一个 epoch 所使用的训练时间。实验结果表明，提出的内存重用策略 LaySA 在扩展批处理大小后并不会对训练过程造成明显的性能损失。相反地，在批量规模不断增加的同时，VGGNet、ResNet 及 BERT 等模型的训练时间消耗明显降低。

表 4-3 批量处理规模与训练耗时

模型	最大批量处理规模			LaySA 训练耗时		
	BS _{norm}	BS _{extreme}	Ratio	BS _{norm}	BS _{extreme}	Ratio
AlexNet	128	2100	+1541%	1h 23m	1h 16m	-8.43%
VGGNet	64	336	+425%	18h 39m	16h 58m	-9.03%
ResNet-50	32	272	+850%	23h11m	21h34m	-6.91%
ResNet-152	32	336	+950%	52h23m	49h55m	-4.71%
WRN-28-10	128	1523	+1090%	29m	27m	-6.89%
BERT-BASE	8	48	+500%	49m	43m	-12.25%

注：训练测试时 VGGNet 及 WRN-28-10 使用 Cifar10 数据集，ResNet 及 AlexNet 使用 ImageNet 数据集，BERT 使用 Wikipedia 数据集，BERT 模型配置(fine-tuned 训练)：序列长度=512，token 数量=512。

如表 4-3 所示，在 P100 GPU 上训练 BERT-BASE 模型时，批处理大小能够从 8 增加到 48，而相比之下，单个 epoch 的整体训练迭代时间减少了 12.25%。分析可知，整体迭代时间减少主要出于以下原因：内存优化策略的引入使得 GPU 的计算及内存资源被更大批量的数据充分利用，由内存优化引起的数据迁移延迟能够被隐藏在更耗时的前后向计算过程中。

小结：LaySA 的优势在于不仅能够协助大规模网络模型完成正常训练，而且能够支撑中等规模网络模型的大批量训练。实验证明，在完成大批量训练时，所使用的内存复用策略对时间性能没有影响，甚至在部分模型中起到加速训练的效果。

4.4.4 整体训练效率评估

为验证 LaySA 在分布式训练阶段的高效性，对比实验选择未作内存及通信优化的原始 MXNet 系统作为基线（baseline），同时选择未使用内存优化策略的 GradSA 作为对比对象，在 BERT-BASE 模型上进行系统吞吐率的测试。由于未使用内存优化策略，在基线系统和 GradSA 系统中，最大序列长度为 512 的 BERT-BASE 模型在单个 P100 GPU 上的批量处理尺寸最大可以设置为 8，而同一模型在 LaySA 中可以将批量尺寸增加至 48。使用 3.5.4 节提出的性能建模方法，在 P100 GPU 集群上将 4 个工作节点模拟扩展到 64 个工作节点，统计得到对应的系统吞吐率。如图 4-7 所示，在 10Gbps 的以太网环境中，与 GradSA 相比，LaySA 至少增加了 150% 的吞吐量。分析可知，LaySA 在 BERT-BASE 模型上表现良好的主要原因在于，单 GPU 同时处理的数据批量显著增加，训练速度始终保持相对稳定，多节点分布式迭代训练的整体吞吐量能够随之提升。

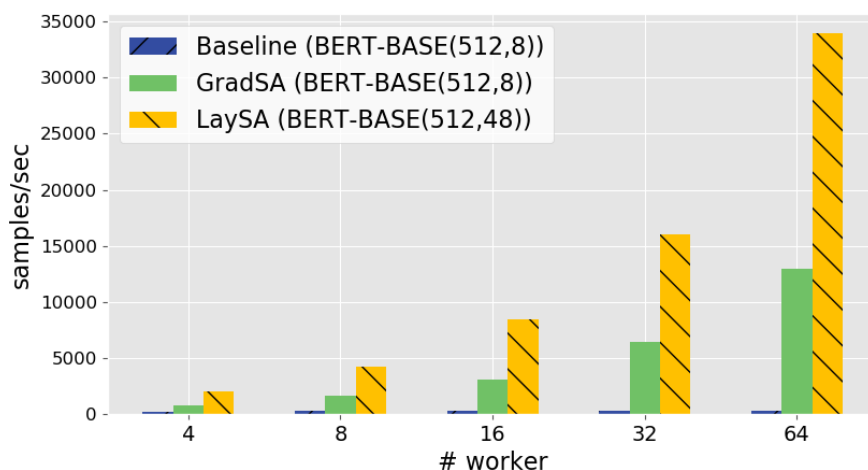


图 4-7 训练系统吞吐量对比

注：训练系统使用 10Gbps 的以太网环境。

小结：在分布式训练任务中，LaySA 方法通过对参数等中间数据进行镜像更新，能够保障基于 GradSA 的稀疏通信优化能够顺利实现，有效降低梯度通信规模，减少通信时间开销，同时在单计算节点内部实现内存管理，使用更大批量数据进行训练，最终实现系统吞吐量的整体提升。

4.5 本章小结

虽然当前众多的神经网络模型形态各异，应用实践的领域各不相同，但是其对深度学习系统多种资源的渴求程度却是类似的，然而当前计算、内存、通信带宽等资源的优化方法仅仅从单一角度出发，尚未能够互相协同、互相融合。直接将第 2、3 章提出的两类优化策略进行简单合并，并不能实际解决多种资源受限的问题，甚至还会阻碍训练效率的提升。因此提出对内存与通信资源进行协同优化的思路 LaySA，首先对内存优化策略进行数据对象增广与重新定义，解决由稀疏通信引起的内存膨胀问题，其次提出镜像参数更新机制，解决参数数据的内存优化难以与稀疏通信优化共存的矛盾，最终实现内存与通信两类优化方案的深度融合、协同执行。实验结果表明，提出的 LaySA 能够兼顾多种类型的神经网络，特别是对参数规模较大的 BERT 类模型有着较好的支持，能够将单个 P100 GPU 上 BERT 模型的批量处理规模扩展至 48，可以满足模型的配置需求，能够在分布式环境下有力保障模型快速收敛。

从算法通用性层面上看，本章将第 2、3 章节的优化策略进行抽象封装，将两者有机结合、并增加新的参数更新方式，达到了较好的通用性和跨平台特性，对相应的策略接口进行简单修改，即可将其部署到包含 TensorFlow 在内的多个深度学习系统中。从算法本质上看，本章所使用的内存优化及通信优化技术，均是以空间或时间为代价换取内存资源及通信资源的高效利用，虽然能够轻松应对大规模神经网络模型，并且尽可能地提升系统资源利用率，但是不可避免地带来一定程度的时间性能开销。因此如何能够进一步地在空间、时间以及系统效率之间达到平衡状态，是有待继续深耕的研究方向。

5 总结与展望

深度学习是一门综合了多个学科内容的前沿领域。随着近年来高性能硬件与分布式理论的发展,深度学习再一次迎来了全面复兴的研究热潮。由于深度学习在训练与推理阶段的计算体量庞大,同时也将产生大量的中间数据。这些特性使得大多数的深度学习应用对中间数据的存储空间及计算平台的高可扩展性有着较大需求。因此,针对深度学习训练过程中的中间数据进行内存管理和通信优化已经成为研究热点。本文针对大规模神经网络模型训练运行时的内存压力大、分布式环境下参数梯度交换导致通信瓶颈、以及单一优化方法无法兼顾多重硬件资源利用率等的挑战,做了深入的研究与广泛的探索。主要工作及创新性研究成果包括以下 3 个方面:

1. 深度学习系统运行时内存管理机制

针对高性能加速硬件内存资源受限的问题,考虑以神经网络的层结构为中心的空间重用思想,提出一种基于空间复用的模型特征映射数据放置机制 Layrub。首先对反向计算的执行序列进行细粒度的重排,然后提出神经网络层敏感的内存复用策略,实现高内存利用率的特征数据组织方式,通过对特征数据在加速硬件与主存之间进行合理迁移与放置,训练运行时的内存消耗明显减少,能够对深度神经网络的进一步设计研究提供系统级支撑。实验表明与原生的 Caffe 相比, Layrub 显著降低了运行时的内存使用率,而优化内存所带来的额外时间开销较小。结果还表明, Layrub 优于一些主流的具备内存优化能力的深度学习系统,并且可以完成极限规模的训练任务。

2. 深度学习系统运行时稀疏通信优化机制

针对分布式训练中节点间梯度通信时带宽资源受限的问题,提出了一种基于陈旧补偿的参数梯度稀疏优化方法 GradSA。首先提出分层的梯度稀疏化思想,符合神经网络的层结构特性,能够显著减少通信开销,其次,利用近似梯度进行历史梯度的积累,以此来加快收敛速度。随后,针对历史梯度累积提出高效的编码机制。实验结果表明,即使在通信网络带宽较低的环境下,所提出的梯度通信优化机制也能获得良好的吞吐量和加速性能,能够在保证性能不下降的前提下,显著减少参数梯度通信量。

通过与 8-bit 量化、TernGrad 和 DGC 等方法进行比较,进一步证明了 GradSA 在各种神经网络模型和数据集上的优势。

3. 深度学习系统运行时内存与通信协同优化机制

由于神经网络模型对加速硬件资源的需求往往来自多方面,单一地从某一角度优化并不能彻底改善训练运行时资源受限的问题。因此提出了对内存与通信资源进行协同优化的思路 LaySA,首先对内存优化策略进行数据对象增广与重新定义,解决由稀疏通信引起的内存膨胀问题,其次提出镜像参数更新机制,解决参数数据的内存优化难以与稀疏通信优化共存的矛盾,最终实现内存与通信两类优化方案的深度融合、协同执行。实验表明,与单独采用 Layrub 和 GradSA 的训练系统相比,LaySA 能够显著节约系统运行时的内存使用量,模型在单 GPU 上整体训练时间也能够明显缩短。除此之外,LaySA 能够使用更大批量的数据进行分布式训练,整体的系统吞吐量提升显著,优于单独使用稀疏通信优化机制的分布式训练系统。

总的来看,从深度学习系统运行时优化研究的发展历程可知,作为两个主要研究分支,内存管理与分布式高效通信始终备受关注。随着学界与产业界的不断探索,内存优化与分布式通信优化二者的关系也从过去各自展开研究,逐渐需要共存、相互促进、协同发展,对二者深度融合的应用需求也在不断涌现。随着深度学习应用实践的不断深入,训练数据体量与神经网络的模型规模进一步扩张。内存优化能够为单机完成超大规模网络的训练任务提供契机,分布式优化能够进一步加速这一训练任务的执行过程,训练系统运行时优化需要两者协同完成。放眼未来,内存优化与分布式通信优化将保持当下这种互相配合、互相驱动的关系,更加深入地融合并协同发展,为未来神经网络模型设计与持续扩展的应用研究提供系统层面的支撑。

但是,围绕深度学习的优化技术方面,仍然有许多工作需要进一步深入研究,可从以下几个方面展开:

(1) 面向移动环境的高效运行机制研究:计算体量和访存体量是用来衡量和评判深度学习模型性能的两大重要指标。相对于本文所专注的 GPU 等加速计算硬件,移动环境下典型的嵌入式设备不管是计算性能,还是内存带宽、片上的缓存,都与高性能硬件非常悬殊。因此在嵌入式设备中部署深度学习模型应更加重视上述两大指标。

如何能够在已知硬件基本情况时，对模型的计算量和访存量进行预估和合理调整，是保证嵌入式深度学习高效运行的必要条件。

（2）基于性能分析的运行时超参数自动配置研究：分布式深度学习系统中的高吞吐率并不能完全代表训练过程的高有效性。这是由于系统软硬件层面上，有众多的配置参数制约着模型的训练速度和训练质量，然而目前所有的训练系统，包括本文所提出的系统在内，这些配置参数均需要用户手动进行设置。因此需要设计一个运行时超参数自动配置机制，对系统硬件及软件层面进行性能分析，以此提升多机多卡系统在训练不同模型时的有效性。

（3）基于新型非易失性存储硬件的深度学习读写能耗优化：在移动及嵌入式环境下，众多深度学习应用需要更大的片上存储。新型非易失性存储硬件（PCM、STT-RAM、RRAM 等）由于其存储密度高、零泄漏等优势，将其作为嵌入式等移动设备的片上或片外存储将有很大潜力。研究深度学习应用的可近似性（高容错性），有效降低基于新型存储硬件的深度学习应用的能耗开销。近似读写虽然能够显著降低新型存储硬件的能耗，但是会给应用的计算精度带来影响。如何能够在保证深度学习应用性能的情况下，对新型存储硬件写能耗进行优化，是有待解决的一个新问题。

致 谢

站在求学生涯的最尾端回头看，一切从懵懂未知，到不懈尝试，再到收获采撷，此刻千头万绪，感慨良深。自 2015 年入学至今，已在喻家山下生活学习了近 5 个年头，这期间，有幸得到了诸位师长亲朋同门的提携和支持，谢意在心，从长道来。

首先要对我的导师金海教授致以深深的鞠躬，这不仅是学子对恩师的谢意，更是晚辈对长辈的敬爱，同样也是自身对学者的崇拜。为人师表的道德模范，严谨端正的治学态度，纯真洒脱的大家风范，皆由金老师言传身教，他给予的宝贵财富，值得思索余生。可以说博士求学期间的每一步成长与发展，都离不开金老师的指导与帮助。虽然事务繁多、身兼数职，但是金老师做到了真正的为学生着想，每每回想一桩桩小事，内心都无比温暖。得此良师，乃人生之大幸。希望今后的我能够努力向恩师看齐，做真正有意义的研究，做一个对祖国有用、对小家有爱的人。

感谢石宣化教授与蒋文斌教授。在两位老师的课题组里，我从初出茅庐的科研菜鸟，逐渐学习到了如何开展科研工作以及团队合作。感谢两位老师让我有机会参与到项目申报、举办会议等的许多学术活动中，在得到锻炼的同时又开阔了眼界，为未来的科研工作打下了坚实的基础。两位老师详尽周全的指导让我获益颇多，十分感激。

感谢新加坡国立大学何炳胜副教授，何老师对学术研究的远见卓识总是让我折服。何老师教会了我如何思辨与表达，这在论文写作过程中不可或缺，使我受益匪浅。感谢何老师总是不厌其烦的跟我讨论论文的方方面面，提出无数宝贵的意见。与何老师的数次合作是我博士生涯中最宝贵的学习时光，我会牢记何老师的谆谆教诲，力争做出有影响力的工作。

感谢刘方明教授，刘老师总是亲切而平易近人，我视他亦师亦友，不断在科研工作中为我答疑解惑，并在博士论文的准备阶段给我很多指引和帮助，不仅如此，更是在生活和未来规划上给我无数真心的建议。刘老师是我眼中无限膜拜的大神，同时也是我心中可以依靠的兄长，他在学术上的高深造诣，在科研上的严谨认真，在生活中对同学们的关爱和照顾，对大局的掌控，对细节的把握，一切的一切，都值得我去不断去学习和揣摩。

华中科技大学博士学位论文

感谢廖小飞教授、吴松教授、华强胜副教授、郑龙副教授、徐鹏副教授、刘海坤副教授等等所有实验室老师们对我的鼓励、支持和帮助。同时感谢办公室的刘英书老师、耿聪老师、庞瑜聪老师、陶莉老师、刘芹老师。这几位美丽睿智的女性老师，辛勤处理着繁杂众多的科研管理事务，认真负责地维持着实验室的正常运转，同时也给这个充满理工气息的实验室带来了无尽的温情。感谢她们细微的关怀，让紧张工作的日常里面充满了人情的暖意，使这个大家庭的成员们和睦可亲，共同发展。

感谢徐慧师姐、陆路师兄、李珍师姐、吴尧师妹、李文珂师妹、杨莹师妹、姚鹏程同学、刘志强师兄、罗毅师兄、张伟师兄、王孝远师兄、罗璇同学、张雄同学、戴小海同学、李志师弟、陈聘师弟等小伙伴们。同在一片屋檐下的我们，在相互的陪伴中，共同进步，共渡难关，分享喜悦、分担烦忧。与他们讨论学术问题总是使我收获满满，同他们一起聊天八卦是我最愉快的课余时间。如果没有他们的存在，那么这些年的求学生活一定会少了很多美景、美食与美好回忆。

最后我要感谢我的家人。感谢我的双亲，他们是这世上我永远的后盾和避风港，无论疾病、困苦、失意，他们从来没有放弃过我，一如既往的爱我、支持我、鼓励我。我从婴儿爬行的第一步，到求学独立的每一步，都离不开他们的注视和关切，我的父母是全世界最好的父母，他们给了我一切的一切，我爱他们。感谢我的舅舅舅妈，犹记舅舅带我去各地求学的不懈与寄望，不忘舅妈在我病中的照料与泪光，他们陪伴着我人生转折的每个节点，他们装点了我的生命，使之多彩而饱满。感谢我的外公外婆与爷爷奶奶，不管远在天堂还是近在床榻，他们视我为骄傲与珍宝，是他们的独爱和珍视，使我自信自爱自强，不畏困难，做我自己。

最后的最后，我要感谢我人生中最重要的人。他是我的灵魂伴侣，我的结发爱人，我的同门师兄，我的同袍战友，我的最好朋友，太多称谓集于一身的人，赵少锋先生。在我们携手同行的8年时光里，他给予了我最多的耐心和温柔。在近5年的博士求学生涯里，他提供给我最大的帮助和支持。没有他的无私付出，没有他的轻声鼓励，就不会有如今收获满满的我。多少个奋斗不息的晨昏，多少个难以成眠的深夜，他见证着我的成长，陪伴着我的迷茫，鼓舞着我的汗水，擦拭去我的眼泪。在这个青春即将结束的而立之年，感恩有他。情深无言，唯有余生相伴。

华中科技大学博士学位论文

最后，感谢所有关心帮助我的亲人、朋友、老师和同学，感谢拨冗垂阅我的论文并提出宝贵意见的各位专家！

参考文献

- [1] McCulloch S, Pitts W. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*. 1943, 5(4): 115-133
- [2] Rosenblatt F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958, 65(6): 386-408
- [3] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998, 86(11): 2278-2324
- [4] Sutskever I, Hinton G, Taylor G. The recurrent temporal restricted boltzmann machine. In *Proceedings of the 23rd Annual Conference on Neural Information Processing Systems (NIPS'09)*. Curran Associates, Inc., Vancouver, BC, Canada, 1601-1608
- [5] Suykens J, Vandewalle J. Least squares support vector machine classifiers. *Neural Processing Letters*, 1999, 9(3): 293-300
- [6] Pal M. Random forest classifier for remote sensing classification. *International Journal of Remote Sensing*, 2005, 26(1): 217-222
- [7] 焦李成, 杨淑媛, 刘芳, 等. 神经网络七十年:回顾与展望. *计算机学报*, 2016, 39(8): 1697-1716
- [8] 周飞燕, 金林鹏, 董军. 卷积神经网络研究综述. *计算机学报*, 2017, 40(06): 1229-1251
- [9] Hinton G, Salakhutdinov R. Reducing the dimensionality of data with neural networks. *Science*, 2006, 313(5786): 504-507
- [10] 方荣强, 王晶, 姚治成, 等. 多层神经网络算法的计算特征建模方法. *计算机研究与发展*, 2019, 56(6): 1170-1181
- [11] Krizhevsky A, Sutskever I, Hinton G. ImageNet classification with deep convolutional neural networks. In *Proceedings of the 25th Annual Conference on Neural Information Processing Systems (NIPS'12)*. Curran Associates, Inc., Lake Tahoe, NV, USA, 1097-1105
- [12] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the 3rd International Conference on Learning*

- Representations (ICLR'15). San Diego, CA, USA
- [13] Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions. In Proceedings of the 28th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'15). IEEE, Boston, MA, USA, 1-9
- [14] Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the inception architecture for computer vision. In Proceedings of the 29th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'16). IEEE, Las Vegas, NV, USA, 2818-2826
- [15] Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, inception-ResNet and the impact of residual connections on learning. In Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI'17). AAAI, San Francisco, CA, USA, 4278-4284
- [16] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. In Proceedings of the 29th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'16). IEEE, Las Vegas, NV, USA, 770-778
- [17] Huang G, Sun Y, Liu Z, et al. Deep networks with stochastic depth. In Proceedings of the 14th European Conference on Computer Vision (ECCV'16). Springer, Amsterdam, Netherlands, 646-661
- [18] 吴建鑫, 高彬彬, 魏秀参, 等. 资源受限的深度学习:挑战与实践. 中国科学: 信息科学, 2018, 48: 501-510
- [19] Tarditi D, Puri S, Oglesby J. Accelerator: Using data parallelism to program GPUs for general-purpose uses. In Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'06). ACM, New York, NY, USA, 325-335
- [20] Bergstra J, Bastien F, Breuleux O, et al. Theano: Deep learning on GPUs with python. In Proceedings of the 2011 NIPS Workshop on BigLearning. Microtome Press, Granada, Spain, 1-48
- [21] 卢丽强, 郑思泽, 肖倾城, 等. 面向卷积神经网络的 FPGA 设计. 中国科学: 信息科学, 2019, 49: 277-2941
- [22] Wang C, Gong L, Yu Q, et al. DLAU: A scalable deep learning accelerator unit on FPGA. IEEE Transactions on Computer-Aided Design of Integrated Circuits and

- Systems, 2016, 36(3): 513-517.
- [23] Li B, Qin Y, Yuan B, et al. Neural network classifiers using stochastic computing with a hardware-oriented approximate activation function. In Proceedings of the IEEE International Conference on Computer Design (ICCD'17). IEEE, Boston, MA, USA, 97-104
- [24] 王慧丽, 郭阳, 屈婉霞. 基于通用向量DSP的深度学习硬件加速技术. 中国科学: 信息科学, 2019, 49: 256-276
- [25] 陈云霄, 陈天石. 人工神经网络处理器. 中国科学: 生命科学, 2016, 46(002): 223-224
- [26] 余凯, 贾磊, 陈雨强, 等. 深度学习的昨天, 今天和明天. 计算机研究与发展, 2013, 50(9): 1799-1804
- [27] 李宏亮, 郑方, 郝子宇, 等. 面向智能计算的国产众核处理器架构研究. 中国科学: 信息科学, 2019, 49: 247-255
- [28] Shafiee A, Nag A, Muralimanohar N, et al. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In Proceedings of the 43rd International Symposium on Computer Architecture (ISCA'16). IEEE, Seoul, Korea, 14-26
- [29] Chi P, Li S, Xu C, et al. PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory. In Proceedings of the 43rd International Symposium on Computer Architecture (ISCA'16). IEEE, Seoul, Korea, 27-39
- [30] 张顺, 龚怡宏, 王进军. 深度卷积神经网络的发展及其在计算机视觉领域的应用. 计算机学报, 2019, 42(3): 453-482
- [31] Bengio Y, Simard P, Frasconi P. Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 1994, 5(2): 157-166
- [32] Bengio Y, Ducharme R, Vincent P. A neural probabilistic language model. Journal of Machine Learning Research, 2003, 3(Feb): 1137-1155
- [33] Mnih A, Hinton G. A scalable hierarchical distributed language model. In Proceedings of the 21st Annual Conference on Neural Information Processing

- Systems (NIPS'08). Curran Associates, Inc., Vancouver, British Columbia, Canada, 1081-1088
- [34] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space. In Proceedings of the International Conference on Learning Representations (ICLR'13). Scottsdale, AZ, USA
- [35] Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality. In Proceedings of the 26th Annual Conference on Neural Information Processing Systems (NIPS'13). Stateline, NV, USA, 3111-3119
- [36] Devlin J, Chang M, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL'19). ACL, Minneapolis, MN, USA, 4171-4186
- [37] Peters M, Neumann M, Iyyer M, et al. Deep contextualized word representations. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL'18). ACL, New Orleans, LA, USA, 2227-2237
- [38] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000-6010
- [39] Bottou L. Large-scale machine learning with stochastic gradient descent. In Proceedings of 19th International Conference on Computational Statistics (COMPSTAT'10). Springer Science & Business Media, Paris, France, 177-186
- [40] Werbos P. Backpropagation through time: What it does and how to do it. Proceedings of the IEEE, 1990, 78(10): 1550-1560
- [41] Gruslys A, Munos R, Danihelka I, et al. Memory-efficient backpropagation through time. In Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS'16). Curran Associates Inc., Barcelona, Spain, 4125-4133
- [42] Milne D, Witten I. An Effective, low-cost measure of semantic relatedness obtained from Wikipedia links. In Proceeding of 2008 AAAI Workshop on Wikipedia and Artificial Intelligence: An Evolving Synergy, AAAI, Chicago, IL, USA, 25-30
-

- [43] Sak H, Senior A, Beaufays F. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In Proceedings of the 15th Annual Conference of the International Speech Communication Association (INTERSPEECH'14). Singapore, 338-342
- [44] Lopez-Moreno I, Gonzalez-Dominguez J, Plchot O, et al. Automatic language identification using deep neural networks. In Proceedings of the 2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'14). IEEE, Florence, Italy, 5337-5341
- [45] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images. Technical Report. University of Toronto. 2009
- [46] Deng J, Dong W, Socher R, et al. ImageNet: A large-scale hierarchical image database. In Proceedings of the 22nd IEEE Conference on Computer Vision and Pattern Recognition (CVPR'09). IEEE, Miami, FL, USA, 248-255
- [47] Qiu Z, Yao T, Mei T. Learning spatio-temporal representation with pseudo-3D residual networks. In Proceedings of the 2017 IEEE International Conference on Computer Vision (ICCV'17). IEEE, Venice, Italy, 5534-5542
- [48] Karpathy A, Toderici G, Shetty S, et al. Large-scale video classification with convolutional neural networks. In Proceedings of the 27th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'14). IEEE, Washington, DC, USA, 1725-1732
- [49] 马立伟, 深度学习驱动领域专用架构, 中国科学: 信息科学, 2019, 49(3): 334-341
- [50] 向陶然, 叶笑春, 李文明, 等. 基于细粒度数据流架构的稀疏神经网络全连接层加速. 计算机研究与发展, 2019, 56(6): 1192-1204
- [51] Dean J, Corrado G, Monga R, et al. Large scale distributed deep networks. In Proceedings of the 25th Annual Conference on Neural Information Processing Systems (NIPS'12). Curran Associates, Inc., Lake Tahoe, NV, USA, 1223-1231
- [52] Chilimbi T, Suzue Y, Apacible J, et al. Project adam: Building an efficient and scalable deep learning training system. In Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation (OSDI'14). USENIX

- Association, Broomfield, CO, USA, 571-582
- [53] Ooi B C, Tan K L, Wang S, et al. SINGA: A distributed deep learning platform. In Proceedings of the 23rd ACM International Conference on Multimedia (MM'15). ACM, New York, NY, USA, 685-688
 - [54] Chen T, Li M, Li Y, et al. MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems. In Proceedings of Workshop on Machine Learning Systems at the 28th Annual Conference on Neural Information Processing Systems (LearningSys'15). Montreal, Canada, 1-6
 - [55] Chen T, Xu B, Zhang C, et al. Training deep nets with sublinear memory cost. arXiv preprint arXiv:1604.06174, 2016
 - [56] Wang L, Ye J, Zhao Y, et al. SuperNeurons: Dynamic GPU memory management for training deep neural networks. In Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'18). ACM, Vienna, Austria, 41-53
 - [57] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16). IEEE, Taipei, China, 1-13
 - [58] Cui H, Zhang H, Ganger GR, et al. GeePS: Scalable deep learning on distributed GPUs with a GPU-specialized parameter server. In Proceedings of the 11th European Conference on Computer Systems (EuroSys'16). ACM, London, UK, 1-16
 - [59] Ben-Nun T, Hoefler T. Demystifying parallel and distributed deep learning: An in-depth concurrency analysis. ACM Computing Surveys, 2019, 52(4): 1-43.
 - [60] Zhang H, Zheng Z, Xu S, et al. Poseidon: An efficient communication architecture for distributed deep learning on GPU clusters. In Proceedings of the 2017 USENIX Conference on Annual Technical Conference (ATC'17). USENIX Association, Santa Clara, CA, USA, 181-193.
 - [61] Denil M, Shakibi B, Dinh L, et al. Predicting parameters in deep learning. In Proceedings of the 26th Annual Conference on Neural Information Processing Systems (NIPS'13). Curran Associates, Inc. Lake Tahoe, NV, USA, 2148-2156
 - [62] Strom N. Scalable distributed DNN training using commodity GPU cloud computing.
-

- In Proceedings of the 16th Annual Conference of the International Speech Communication Association (INTERSPEECH'15), Dresden, Germany, 1488-1492
- [63] Aji A, Heafield K. Sparse communication for distributed gradient descent. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP'17). ACL, Copenhagen, Denmark, 440-445
- [64] Lin Y, Han S, Mao H, et al. Deep gradient compression: Reducing the communication bandwidth for distributed training. In Proceedings of the 6th International Conference on Learning Representations (ICLR'18). Curran Associates, Inc., Vancouver, BC, Canada, 1-10
- [65] Dryden N, Jacobs S, Moon T, et al. Communication quantization for data-parallel training of deep neural networks. In Proceedings of the Workshop on Machine Learning in High Performance Computing Environments (MLHPC'16). IEEE, Salt Lake City, UT, USA, 1-8
- [66] Han S, Liu X, Mao H, et al. EIE: Efficient inference engine on compressed deep neural network. In Proceedings of the 43rd International Symposium on Computer Architecture (ISCA'16). ACM, Seoul, Korea, 243-254
- [67] Han S, Mao H, and Dally W. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In Proceedings of the 4th International Conference on Learning Representations (ICLR'16). San Juan, Puerto Rico, 1-14
- [68] Micikevicius P, Narang S, Alben J, et al. Mixed precision training. arXiv preprint arXiv:1710.03740. 2017
- [69] Seide F, Fu H, Droppo J, et al. 1-Bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In Proceedings of the 16th Annual Conference of the International Speech Communication Association (INTERSPEECH'14). Singapore, 1058-1062
- [70] Alistarh D, Grubic D, Li J, et al. QSGD: Communication-efficient SGD via gradient quantization and encoding. In Proceedings of the 31st Annual Conference on Neural Information Processing Systems (NIPS'17). Curran Associates, Inc., Long Beach, CA, USA, 1709-1720
- [71] Zagoruyko S, Komodakis N. Wide residual networks. In Proceedings of the 27th

- British Machine Vision Conference (BMVC'16). BMVA, York, UK, 87.1-87.12
- [72] Xie S, Girshick R, Dollár P, et al. Aggregated residual transformations for deep neural networks. In Proceedings of the 30th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'17). IEEE, Honolulu, HI, USA, 5987-5995
- [73] Jia Y, Shelhamer E, Donahue J, et al. Caffe: Convolutional architecture for fast feature embedding. In Proceedings of the 22nd ACM International Conference on Multimedia (MM'14). ACM, Orlando, FL, USA, 675-678
- [74] Abadi M, Barham P, Chen J, et al. TensorFlow: A system for large-scale machine learning. In Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI'16). USENIX Association, Savannah, GA, USA, 265-283
- [75] Jiang J, Yu L, Jiang J, et al. Angel: A new large-scale machine learning system. National Science Review, 2018, 5(2): 216-236.
- [76] Chetlur S, Woolley C, Vandermersch P, et al. cuDNN: Efficient primitives for deep learning. arXiv preprint arXiv:1410.0759, 2014
- [77] NVIDIA. CUDA toolkit 8.0 documentation: profiler. Retrieved December 13, 2019 from <http://docs.nvidia.com/cuda/profiler-users-guide/index.html#axzz4p7v1j ezC>
- [78] Glorot X, Bordes A, Bengio Y. Deep sparse rectifier neural networks. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS'11). JMLR, Lauderdale, FL, USA, 315-323.
- [79] NVIDIA. Introduction to cuBLAS. Retrieved December 13, 2019 from <http://docs.nvidia.com/cuda/cublas/index.html#axzz4oxZMgelu>
- [80] Mikolov T, Karafiát M, Burget L, et al. Recurrent neural network based language model. In Proceedings of the 11th Annual Conference of the International Speech Communication Association (INTERSPEECH'10). Chiba, Japan, 1045-1048.
- [81] Donahue J, Anne Hendricks L, Guadarrama S, et al. Long-term recurrent convolutional networks for visual recognition and description. In Proceedings of the 28th IEEE Conference on Computer Vision and Pattern Recognition (CVPR'15). IEEE, Boston, MA, USA, 2625-2634
- [82] Soomro K, Zamir A R, Shah M. UCF101: A dataset of 101 human actions classes from videos in the wild. Technical Report CRCV-TR-12-01. University of Central

- Florida, Orlando, FL, USA, 2012
- [83] Jiang W, Ma Y, Liu B, et al. Layup: Layer-adaptive and multi-type intermediate-oriented memory optimization for GPU-based CNNs, *ACM Transactions on Architecture and Code Optimization*, 2019, 16(4), 39:1-23
 - [84] Li M, Andersen D, Park J, et al. Scaling distributed machine learning with the parameter server. In *Proceedings of the 11st USENIX Symposium on Operating Systems Design and Implementation (OSDI'14)*. USENIX Association, Broomfield, CO, USA, 583-598
 - [85] NVIDIA. NVIDIA V100 TENSOR CORE GPU. Retrieved December 12, 2019 from <https://www.nvidia.com/en-us/data-center/v100/>
 - [86] Zhang C, Haddadi H. Deep learning in mobile and wireless networking: A survey. *IEEE Communications Surveys & Tutorials*, 2019, 3(21): 2224-2287
 - [87] Wen W, Xu C, Yan F, et al. Terngrad: Ternary gradients to reduce communication in distributed deep learning. In *Proceedings of the 31st Annual Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates, Inc., Long Beach, CA, USA, 1509-1519
 - [88] Wangni J, Wang J, Liu J, et al. Gradient sparsification for communication-efficient distributed optimization. In *Proceedings of the 32nd Annual Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates, Inc., Montral, Canada, 1299-1309
 - [89] Wu J, Huang W, Huang J, et al. Error compensated quantized SGD and its applications to large-scale distributed optimization. In *Proceedings of the 35th International Conference on Machine Learning (ICML'18)*. PMLR, Stockholmsmssan, Stockholm Sweden, 5325-5333
 - [90] Tang H, Lian X, Zhang T, et al. DoubleSqueeze: Parallel stochastic gradient descent with double-pass error compensated compression. In *Proceedings of the 36th International Conference on Machine Learning (ICML'19)*. PMLR, Long Beach, California, USA, 6155-6165
 - [91] Lei Ba J, Kiros J, Hinton G. Layer normalization. *arXiv preprint arXiv: 1607.06450*, 2016
 - [92] Wikipedia. Sparse matrix. Retrieved December 13, 2019 from <https://en.wikipedia.org>

- g/wiki/Sparsematrix
- [93] Ketkar N, Introduction to PyTorch. Springer, 2017, 195-208
 - [94] Jeffreys H, Jeffreys B. Methods of mathematical physics. Cambridge University press, 1999, 62-66
 - [95] Zheng S, Meng Q, Wang T. Asynchronous stochastic gradient descent with delay compensation. In Proceedings of the 34th International Conference on Machine Learning (ICML'17). PMLR, Sydney, Australia, 4120-4129
 - [96] Hastie T, Tibshirani R, Friedman J. The elements of statistical learning, Springer, 2001, 139-189
 - [97] Becker S, LeCun Y. Improving the convergence of back-propagation learning with second order methods. In Proceedings of the 1988 Connectionist Models Summer School. Morgan Kaufmann, San Matteo, CA, USA, 29-37
 - [98] Sutskever I, Martens J, Dahl G, et al. On the importance of initialization and momentum in deep learning. In Proceedings of the 30th International Conference on Machine Learning (ICML'13). PMLR, Atlanta, GA, USA, 1139-1147
 - [99] Banner R, Hubara I, Hoffer E, et al. Scalable methods for 8-bit training of neural networks. In Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18). Curran Associates Inc., Montreal, Canada, 5151-5159
 - [100] Ma Z, Zhang Z, Ding Z, et al. Key techniques for 5G wireless communications: network architecture, physical layer, and MAC layer perspectives. SCIENCE CHINA Information Sciences, 2015, 58(4): 1-20
 - [101] David K, Berndt H. 6G vision and requirements: Is there any need for beyond 5G? IEEE Vehicular Technology Magazine, 2018, 13(3): 72-80
 - [102] Yan F, Ruwase O, He Y, et al. Performance modeling and scalability optimization of distributed deep learning systems. In Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'15). ACM, Sydney, NSW, Australia, 1355-1364
 - [103] NVIDIA. NVIDIA collective communication library (NCCL) documentation. Retrieved December 13, 2019 from <https://developer.nvidia.com/nccl>

附录 1 攻读博士学位期间发表的主要论文

- [1] Hai Jin, **Bo Liu**, Wenbin Jiang, Yang Ma, Xuanhua Shi, Bingsheng He, Shaofeng Zhao. Layer-Centric Memory Reuse and Data Migration for Extreme-Scale Deep Learning on Many-Core Architectures. *ACM Transactions on Architecture and Code Optimization*, 2018, 15(3), 37:1-26 (SCI 检索, CCF B 类期刊)
- [2] **Bo Liu**, Wenbin Jiang, Hai Jin, Xuanhua Shi, Yang Ma. Layrub: Layer-Centric Memory Reuse and Data Migration for Extreme-Scale Deep Learning Systems. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP'18)*. ACM, Vienna, Austria, 1-2 (CCF A 类会议, 短文)
- [3] **Bo Liu**, Wenbin Jiang, Shaofeng Zhao, Hai Jin, Bingsheng He. GradSA: Gradient Sparsification and Accumulation for Low-Bandwidth Communication in Distributed Deep Learning. In *Proceedings of the Green, Pervasive, and Cloud Computing (GPC'20)*. Springer, Xi'an, China (CCF C 类会议)
- [4] **Bo Liu**, Xuanhua Shi, Hai Jin. Measuring Directional Semantic Similarity with Multi-Features. In *Proceedings of the Asia-Pacific Web Conference (APWeb'16)*. Springer, Suzhou, China, 543-554 (CCF C 类会议)
- [5] Wenbin Jiang, Yang Ma, **Bo Liu**, Haikun Liu, Bingbing Zhou, Jian Zhu, Song Wu, Hai Jin. Layup: Layer-Adaptive and Multi-Type Intermediate-Oriented Memory Optimization for GPU-Based CNNs. *ACM Transactions on Architecture and Code Optimization*, 2019, 16(4), 39:1-23 (SCI 检索, CCF B 类期刊)
- [6] Xuanhua Shi, Zhigao Zheng, Yongluan Zhou, Hai Jin, Ligang He, **Bo Liu**, Qiang-Sheng Hua. Graph Processing on GPUs: A Survey. *ACM Computing Surveys*, 2018, 50(6), 81:1-35 (SCI 检索, 影响因子 6.748)

(注: 以上论文第一署名单位均为华中科技大学)

附录 2 攻读博士学位期间申请发明专利

申请国家发明专利

- [1] 蒋文斌, 金海, 祝简, 马阳, 刘博、彭晶, 刘湃. 一种分布式深度学习参数量化通信优化方法及系统, 专利申请号 201811382970.7, 专利申请日 2018 年 11 月 20 日, 申请单位: 华中科技大学
- [2] 蒋文斌, 金海, 祝简, 马阳, 刘博、彭晶, 刘湃. 一种用于图像分类的深度神经网络的训练方法, 专利申请号 201811389548.4, 专利申请日 2018 年 11 月 21 日, 申请单位: 华中科技大学

附录 3 攻读博士学位期间参与的主要科研项目

- [1] 移动网络环境下云端融合的关键技术合作研究 (No. 2015DFE12860), 国际科技合作与交流专项, 2015.04-2017.12
- [2] 基于弱同步策略的分布式深度学习并行优化理论与方法研究 (No. 61672250), 国家自然科学基金面上项目, 2017.01-2020.12
- [3] 面向大数据高效处理的体系结构与关键技术 (No. 61832006), 国家自然科学基金重点项目, 2019.01-2023.12
- [4] 面向图计算的通用计算机技术与系统 (No. 2018YFB1003500), 国家重点研发计划项目, 2018.5-2021.4